



ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא

מערכת סל קניות חכם דיגיטלית



שם הסטודנט: דויד בן גיגי

ת"ז: 329235535

מנחה: אילן פרץ

תאריך הגשה: י"ח סיוון תשפ"ו 03/06/2026

המכללה: כנפי רוח, קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

3	1. הצעת הפרויקט שאושרה ע"י משרד החינוך
4	2. מבוא
4	2.1 הרקע לפרויקט
5	2.2 תהליך המחקר
5	2.3 רקע תיאורטי וסקירת ספרות מקצועית
6	2.4 אתגרים מרכזיים במחקר
7	3. מטרות ויעדים לפיתוח המערכת
8	4. אתגרים בבניית המערכת
12	5. הגדרת מדדי הצלחה למערכת
13	6. תיאור פתרונות קיימים לבעיה וניתוח חלופות
15	7. תיאור החלופה הנבחרת ונימוקים לבחירתה
16	8. אפיון המערכת המוצעת
16	8.1 ניתוח הדרישות מהמערכת
17	8.2 מודלי המערכת
18	8.3 אפיון פונקציונלי וביצועים עיקריים
19	8.4 אילוצי המערכת
20	9. תיאור הארכיטקטורה של המערכת
20	9.1 ארכיטקטורת המערכת
22	9.2 מודל שרת לקוח
23	9.3 תהליכים של מע' ההפעלה
24	9.4 תיאור פרוטוקולי תקשורת
24	9.5 שירותים חיצוניים
25	10. ניתוח תרחישים וזרימת המידע במערכת המוצעת
25	10.1 תרשימי תרחיש Use case Diagram
27	10.2 תרשימי רצף Sequence Diagram
36	11. תיאור מסכים וממשק משתמש
36	11.1 תרשים היררכיית המסכים
37	11.2 תיאור המסכים
64	12. תיאור התוכנה
64	12.1 סביבת הפיתוח של התוכנה
66	12.2 תיאור מבנה הפרויקט והמחלקות
94	12.4 מבני נתונים בהם נעשה שימוש
96	12.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
104	13. תיאור מסד הנתונים
106	14. מדריך למשתמש
109	15. בדיקות והערכה
109	16. מסקנות
110	17. פיתוחים עתידיים
111	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

תיאור הנושא הפרויקט

פרויקט "סל חכם" (SmartCart) הינו מערכת אינטרנטית בתחום האופנה, אשר פותחה במטרה לספק מענה טכנולוגי לבעיית הצפת המידע והקושי בקבלת החלטות שתוארו בסעיף הקודם. המערכת משמשת כחנות וירטואלית מקיפה, המאגדת בתוכה את כלל הכלים הסטנדרטיים המצופים מאתר מסחר מודרני, לרבות מודול לניהול משתמשים המאפשר הרשמה והתחברות לשמירת פרופיל אישי, קטלוג מוצרים דינמי ועשיר הכולל תמונות ומפרטים, מנועי חיפוש וסינון, וניהול סל קניות ידני לביצוע רכישות שגרתיות. מעבר לפונקציונליות הבסיסית, חדשנותו של הפרויקט באה לידי ביטוי בשילוב רכיב "הסל החכם", המשמש כעוזר קנייה אישי המבוסס על אלגוריתמיקה מתקדמת. רכיב זה נועד להחליף את תהליך החיפוש הידני והמייגע, ומאפשר למשתמש להגדיר בצורה פשוטה סט של אילוצים והעדפות אישיות, כגון תקציב מקסימלי, טווח כמות פריטים רצויה וסגנון לבוש מועדף. ברגע שהמשתמש מזין את נתוניו, המערכת מבצעת עיבוד נתונים מהיר ובונה עבורו באופן אוטומטי את סל הקניות האופטימלי ביותר העומד בכל הדרישות שהוגדרו, ובכך מגשרת על הפער בין ההיצע העצום לבין יכולת העיבוד המוגבלת של הצרכן. שילוב זה בין חנות אונליין מסורתית לבין מנוע אופטימיזציה חכם מעניק למשתמש חווית קנייה יעילה, חוסך זמן חיפוש יקר ומבטיח ניצול כלכלי מיטבי של התקציב העומד לרשותו.

הגדרת הבעיה האלגוריתמית

כפי שתואר בפסקה הקודמת (תיאור הנושא הפרויקט), האתגר המרכזי בפרויקט הוא לגשר על הפער בין הרצון של המשתמש למקסם את התמורה שהוא מקבל מבחינת איכות והתאמה אישית, לבין המגבלות הפיננסיות והכמותיות שלו.

מבחינה מתמטית וחישובית, לב המערכת מגדיר ומממש וריאציה מורחבת ותלת-ממדית של בעיית התרמיל הרב-ממדית הקלאסית (MDKP), המסווגת בספרות המקצועית כבעיה קשה חישובי (NP-Hard).

בהינתן מאגר דינמי של N מוצרים, כאשר לכל מוצר i קיים מחיר קבוע P_i וציון התאמה דינמי S_i המחושב בהתאם להעדפות המשתמש, המערכת נדרשת לקבל החלטה בינארית $x_i \in \{0, 1\}$ האם לכלול את הפריט בסל או לא.

פונקציית המטרה מוגדרת כמקסום הניקוד המצטבר של הפריטים שנבחרו (כמוצג למטה).

$$\text{Maximize } \sum_{i=1}^N S_i \cdot x_i$$

תחת כפיפות למערכת אילוצים קשיחים המוזנים על ידי הלקוח, כדלהלן:

- **אילוץ תקציבי חסום (W):** סך עלות המוצרים שנבחרו לא יעלה על התקציב שהוגדר.

$$\sum_{i=1}^N P_i \cdot x_i \leq W$$

- **אילוץ כמותי מוגדר ($K_{\min}$, $K_{\max}$):** מספר הפריטים הכולל בסל חייב להתכנס במדויק בתוך טווח כמויות מוגדר מראש.

$$K_{\min} \leq \sum_{i=1}^N x_i \leq K_{\max}$$

2. מבוא

פרק זה מציג את המבוא והרקע הכללי לפרויקט "הסל החכם". במסגרת הפרק נסקור בקצרה את המצב הקיים בשוק המסחר האלקטרוני ואת הסיבות שהובילו לצורך במערכת חכמה. כמו כן, נפרט על האתגרים והמטרות המרכזיות של הפרויקט, ונציג את המושגים הטכנולוגיים והתיאורטיים שעליהם המערכת מתבססת.

2.1 הרקע לפרויקט

כיום, חנויות בגדים באינטרנט מציגות קטלוגים רחבים ומסתמכות על סינונים פשוטים בלבד. לקוחות שמחפשים להרכיב לעצמם חבילת לבוש נאלצים לעבור מוצר-מוצר, לחשב מחירים ידנית ולנסות להתאים את הכל למגבלת התקציב שלהם.

מערכת SmartCart פותחה כדי לפתור את הקושי הזה באמצעות "הסל החכם" – דף ייעודי שבו המשתמש מגדיר את התקציב שלו, כמויות הפריטים הרצויות (מינימום ומקסימום) ועונה על שאלון סגנון. המערכת מריצה אלגוריתם אופטימיזציה מתמטי שמחשב ומציג לו מיד את חבילת הלבוש בעלת ההתאמה הגבוהה ביותר לתקציב ולטעם שלו.

בנוסף, כפיצ'ר עזר משני לחוויית המשתמש, שולב בדף הבית באנר המלצות קטן שמושך נתוני טמפרטורה בסיסיים ומציע פריטים קלים לעונה הנוכחית, ללא קשר לאלגוריתם הראשי של הסל.

2.2 תהליך המחקר

תהליך המחקר בפרויקט זה התבצע בארבעה שלבים אסטרטגיים, תוך אימוץ מתודולוגיות פיתוח תוכנה מודרניות [5], במטרה להבטיח מערכת יציבה וניתנת להרחבה.

להלן פרוט השלבים:

1. **מיפוי צרכים:** שלב זה כלל ניתוח מעמיק של עולם ה-E-Commerce. המחקר התמקד בקושי הצרכני המכונה "עייפות בחירה", המתרחש כאשר לקוחות מוצפים במבחר פריטים ללא כלים לסנן אותם.
2. **סקירה אלגוריתמית:** ביצעתי מחקר השוואתי על מודלים מתחום האופטימיזציה הקומבינטורית [1]. המטרה הייתה למצוא אלגוריתם שאינו מסתפק בחיפוש בסיסי, אלא מסוגל לבצע "סינון חכם" המשלב העדפות אישיות עם אילוצי תקציב וכמות פריטים.
3. **בחירת טכנולוגיות:** נבחר ה-Stack המודרני של Spring Boot לניהול הלוגיקה וה-Backend, בשילוב עם מסד הנתונים MongoDB [6]. שילוב זה מאפשר גמישות בניהול אובייקטים מורכבים וביצועים גבוהים בהשוואה למסדי נתונים רלציוניים קשיחים [4].
4. **תיקוף:** בניית אב-טיפוס איטרטיבי. בשלב זה בחנתי את האלגוריתם תחת עומסים שונים כדי לוודא שזמן החישוב נשאר פולינומיאלי ואינו פוגע בחוויית המשתמש.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

הבעיה הניצבת בבסיס המערכת היא גרסה מתקדמת של בעיית תרמיל הגב הקלאסית, המכונה **MDKP (Multidimensional Knapsack Problem)**. בעוד שהבעיה הקלאסית (Knapsack 0/1) עוסקת באילוף יחיד, המערכת שלנו נדרשת לאופטימיזציה רב-ממדית הכוללת: תקציב מוגבל, כמות פריטים מקסימלית/מינימלית, וציון התאמה (Match Score).

הספרות האקדמית מגדירה בעיה זו כ-NP-Hard [2], מה שמשמעותו התיאורטית היא כי קיים קושי למצוא פתרון אופטימלי בזמן ריצה קצר ככל שמרחב הבעיה גדל. עם זאת, השימוש בשיטת **תכנות דינמי (Dynamic Programming)** נמצא בספרות כגישה היעילה ביותר לפתרון בעיות אלו. מחקרים מראים כי באמצעות טבלאות מצבים, ניתן להשיג פתרון

אופטימלי בזמן ריצה סביר [3], מה שהופך את ה-DP לסטנדרט התעשייתי במערכות הממקסמות משאבים תחת אילוצים מרובים [1].

2.4 אתגרים מרכזיים במחקר

במהלך שלבי הפיתוח, נתקלתי בדילמות תכנוניות המוגדרות כאתגרים הנדסיים מורכבים, שכן הם דרשו איזון בין דרישות ביצועים לבין עקביות נתונים. להלן האתגרים המרכזיים והמענה ההנדסי שניתן להם:

1. **מורכבות אלגוריתמית:** האתגר המרכזי היה המעבר מתיאוריה מתמטית למימוש בשפת Java, תוך הבטחת זמן תגובה נמוך ב - UI (ללא נעילת ה Thread הראשי) לצורך כך, בוצעה הפרדת שכבות, [7] (Layered Architecture) כאשר הלוגיקה האלגוריתמית הופרדה לחלוטין משכבת התצוגה.
2. **ניהול נתונים:** השילוב בין מנוע אלגוריתמי מורכב לבין מסד נתונים מונחה מסמכים (NoSQL) דרש תכנון זהיר של מבנה הנתונים כדי למנוע יתירות [6]. נעשה שימוש בטבלאות תכנות דינמי (DP Tables) השומרות את מצבי הביניים, טכניקה המבטיחה עבודה יעילה ומונעת חישובים חוזרים. [3]
3. **אמינות היסטורית:** האתגר בשמירה על "צילום מצב" (Snapshot) של הנתונים, כך שהיסטוריית הרכישות של המשתמש תישאר אמינה ועקבית גם אם המוצרים המקוריים בקטלוג ישתנו בעתיד. פתרון זה הבטיח אמינות נתונים חשבונאית במערכת.

2.4.1 התמודדות עם הבעיה

ההתמודדות בוצעה באמצעות ארכיטקטורת שכבות (Layered Architecture) [7]. הלוגיקה האלגוריתמית הופרדה לחלוטין משכבת התצוגה (UI), מה שמאפשר להריץ חישובים כבדים ב-Service Layer ללא פגיעה בחוויית המשתמש. כמו כן, השימוש בטבלאות תכנות דינמי איפשר שמירת מצבי ביניים, טכניקה שמונעת חישובים חוזרים ומבטיחה עבודה יעילה תחת עומס [3].

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

המוטיבציה היא הנדסית-יישומית. היכולת לתרגם בעיה מתמטית קלאסית ומרתקת לפתרון טכנולוגי שמשנה את חוויית המשתמש הקצה. המטרה היא להוכיח כי תכנון אלגוריתמי מדויק יכול לייעל משמעותית תהליכי קבלת החלטות במערכות מורכבות, ולהוות כלי עזר משמעותי במערכות מסחר דיגיטליות.

2.4.3 על איזה צורך הפרויקט עונה ?

הפרויקט עונה על הצורך של **אופטימיזציה צרכנית בזמן אמת**. המשתמש המודרני מוצף במידע ומתקשה לבנות סל מוצרים שמשלב בצורה אופטימלית בין תקציב לבין העדפות סגנון אישיות. הפרויקט מספק מנוע חכם המבצע את האופטימיזציה עבור המשתמש, חוסך לו זמן יקר ומבטיח כי התקציב שלו מנוצל בצורה הטובה ביותר, ובכך הוא משפר את המרת המכירות של בעל העסק.

3. מטרות ויעדים לפיתוח המערכת

פרק זה מגדיר את המטרות המרכזיות של מערכת "הסל החכם", הן מההיבט השימושי עבור הלקוח והן מההיבט הטכנולוגי והנדסי של פיתוח התוכנה.

מטרות פונקציונליות וצרכניות (עבור משתמשי האתר)

- **אוטומציה של הרכבת סל הקניות:** המטרה המרכזית היא לאפשר ללקוח להזין תקציב והעדפות לבוש, ולקבל באופן מיידי שילוב של בגדים שמתאים לדרישות שלו, מבלי שיצטרך לחשב ולחפש מוצרים אחד-אחד.
- **גמישות ושליטה של הלקוח:** לתת למשתמש אפשרות לבחור בעצמו אילו מוצרים מתוך החבילה המוצעת הוא מעוניין להוסיף בפועל לעגלה הכללית של האתר, במקום לחייב אותו לרכוש חבילה קבועה מראש.
- **חווית קנייה פשוטה ואינטראקטיבית:** לספק ממשק משתמש נוח וברור, הכולל שאלון העדפות דינמי, באנר המלצות בסיסי לפי מזג האוויר המקומי בדף הבית, וחיווי ויזואלי ברור בזמן שהמערכת מבצעת את החישובים.
- **ניהול פלטפורמת מסחר מלאה:** אספקת חנות וירטואלית שלמה המאפשרת למשתמשים לבצע את כל שלבי הרכישה המסורתיים: רישום והתחברות מאובטחת למערכת, דפדוף וסינון בקטלוג הבגדים הראשי, ניהול עגלת קניות ידנית רגילה, ומסך היסטוריית הזמנות אישי לעקוב אחר רכישות עבר.
- **ניהול מלאי ומוצרים בצד שרת:** מתן מענה מלא לצורכי העסק, הכולל תשתית המאפשרת שמירה, עדכון ושליפה של מוצרים ישירות מול מסד הנתונים, כך שהמלאי המוצג ללקוח תמיד מסונכרן ומעודכן בזמן אמת.

מטרות טכנולוגיות והנדסיות (עבור מנהל המערכת והפיתוח)

- **דיוק מתמטי באופטימיזציה:** מימוש אלגוריתם יציב (תכנות דינמי) שמסוגל לעבד את מלאי החנות ולמצוא את שילוב המוצרים הטוב ביותר תחת שלושה אילוצים במקביל (תקציב, מינימום פריטים ומקסימום פריטים), ללא שגיאות חישוב.
- **ניהול תוכן דינמי מלא לאדמין:** בניית מנגנון המאפשר לחשבון מנהל המערכת (Admin) להוסיף, למחוק או לערוך את שדות השאלון והאופציות שלהם ישירות מתוך ממשק האתר, מבלי לשנות את קוד התוכנה או את מבנה הטבלאות.

- **שימוש חוזר ברכיבי קוד:** עיצוב ופיתוח רכיבי ממשק מודולריים (כמו כרטיס המוצר) כך שיוכלו לשמש את האתר במסכים שונים (דף החנות, מנוע ההמלצות ודף הבית) תוך שינוי גודל ומאפיינים בלבד, כדי למנוע כפל קוד.
- **הפרדת תהליכים:** הרצת חישובי האלגוריתם הכבדים בתהליכון (Thread) נפרד מאחורי הקלעים, כדי למנוע את תקיעת הדפדפן של המשתמש ולאפשר עדכון אסינכרונית של מד התקדמות הריצה.
- **יישום ארכיטקטורת שכבות:** בניית האפליקציה בצורה מבוצרת המפרידה לחלוטין בין שלוש שכבות: שכבת המידע, שכבת הלוגיקה העסקית והשירותים ושכבת הממשק. הפרדה זו מבטיחה קוד נקי ויכולת לתחזק ולהרחיב את החנות בעתיד בקלות.
- **אבטחת מידע וניהול סשן:** מימוש מנגנון הרשאות מבוסס תפקידים המפריד בין לקוח רגיל לבין מנהל מערכת (Admin) תוך שמירה מאובטחת של נתוני המשתמש והעגלה בתוך ה VaadinSession כדי למנוע זליגת מידע בין דפדפנים שונים באתר.
- **ממשק ניהול ייעודי למנהל המערכת:** אספקת מסך ניהול מאובטח שבו מנהל האתר יכול לראות את כל המידע הקיים, להוסיף בגדים חדשים לקטלוג, לערוך ולשנות פרטים של בגדים קיימים (כמו עדכון מחיר, שינוי תיאור, החלפת תמונה או הוספת תגיות), ולמחוק מוצרים במידת הצורך.

4. אתגרים בבניית המערכת

במהלך פיתוח מערכת "הסל החכם", נתקלנו במספר אתגרים טכנולוגיים וארכיטקטוניים בשכבות השונות של האפליקציה (בסיס הנתונים, הלוגיקה וממשק המשתמש). פרק זה מתאר את האתגרים המרכזיים ואת הפתרונות ההנדסיים שיושמו כדי לפתור אותם.

התמודדות עם מורכבות הבעיה האלגוריתמית וסיבוכיות זמן הריצה

- **האתגר:** הבעיה העומדת בלב המערכת מורכבת יותר מבעיית תרמיל הגב הקלאסית (0/1 Knapsack), כיוון שהיא דורשת אופטימיזציה תחת מספר אילוצים בו-זמנית (גם מגבלת מחיר וגם מגבלת טווח כמויות של פריטים). בעיה זו מוכרת בספרות המדעית כ-MDKP (Multidimensional Knapsack Problem) ומסווגת כבעיה מסוג NP-Hard (SCIRP, 2018) המשמעות התיאורטית היא שלא קיים אלגוריתם המסוגל לפתור אותה בזמן פולינומיאלי מהיר לכל קלט, דבר שהיה עלול

להוביל לזמני חישוב ארוכים מאוד ולתקיעת השרת במידה ומשתמשים בגישות מסורתיות כמו כוח גס (Brute Force).

- **הפתרון:** בחרנו להתמודד עם אתגר הסיבוכיות באמצעות גישת תכנות דינמי (Dynamic Programming). במקום לבדוק את כל השילובים האפשריים בצורה מעריכית, האלגוריתם מפרק את הבעיה לתתי-בעיות קטנות ושומר את תוצאות הביניים בטבלת זיכרון תלת-ממדית (מטריצה). מבנה זה מאפשר למנוע חישובים חוזרים, ומוריד את סיבוכיות זמן הריצה לזמן פסאודו-פולינומיאלי מהיר ויעיל. הפתרון מאזן בצורה מושלמת בין דיוק מתמטי מוחלט של 100% לבין מהירות תגובה מיידיית במערכת זמן-אמת.

מניעת קפיאת ממשק המשתמש בזמן חישוב האלגוריתם

- **האתגר:** אלגוריתם ה - Knapsack התלת-ממדי שמחשב את סל הקניות האופטימלי מבצע פעולות מתמטיות רבות מול מלאי החנות במסד הנתונים. הרצת חישוב כזה ישירות על גבי תהליכון הממשק הראשי של Vaadin גרמה לקפיאה מוחלטת של דפדפן המשתמש עד לסיום החישוב, דבר שפגע קשות בחוויית המשתמש ומנע אפשרות להציג מד התקדמות.
- **הפתרון:** פיצלנו את תהליך החישוב באמצעות ניהול תהליכונים. עטפנו את פעולת האלגוריתם בתוך משימת Runnable נפרדת הרצה ב - Thread עצמאי מאחורי הקלעים. כדי לעדכן את רכיבי המסך ואת ה - ProgressBar בצורה בטוחה תוך כדי החישוב מבלי לייצר התנגשויות, השתמשנו במנגנון ui.access() של Vaadin המאפשר לתהליכון המשני לעדכן את ה - UI של המשתמש בזמן אמת ובאופן אסינכרוני.

מעבר משאלון קשיח למערכת שדות דינמית

- **האתגר:** בהתחלה, שדות שאלון הסטיל של המשתמש נבנו בצורה קשיחה בקוד באמצעות מחלקה קבועה בשם SurveyCriteria שהחזיקה משתנים קבועים מראש (כמו צבע, עונה וסוג בד). בעקבות דרישה לאפשר לחשבון האדמין לנהל, למחוק ולהוסיף שדות ואופציות חדשות לשאלון ישירות מהממשק, המבנה הקשיח הזה לא יכול היה לעבוד יותר ואיים לשבש את חישוב ציוני האלגוריתם.
- **הפתרון:** מחקנו לחלוטין את לוגיקת ה - SurveyCriteria הקשיחה והעברנו את ניהול השאלון למסמך דינמי במסד הנתונים בשם TagField שכתבנו את שירות חישוב הציונים (ScoringService) כך שיקבל רשימה דינמית של מחרוזות המייצגות את

התגיות. כעת המערכת רצה בלופ על הבחירות של המשתמש ומחשבת את אחוז ההתאמה הריאליסטי מול תגיות המוצר, מבלי להיות תלויה באף שדה קבוע מראש בקוד.

שימוש חוזר ברכיבי ממשק ושינוי מימדים

- **האתגר:** המערכת נדרשה להציג את כרטיסי המוצרים במספר מסכים שונים באתר, בדף החנות הראשי, בתוצאות האלגוריתם של הסל החכם, ובוידג'ט המלצות מזג האוויר בדף הבית. יצירת רכיב עיצובי ולוגי נפרד לכל מסך הייתה מייצרת כפל קוד משמעותי, מסרבלת את התחזוקה ומנפחת את הפרויקט שלא לצורך.
- **הפתרון:** יישמנו עקרון שימוש חוזר ברכיבים. שדרגנו את המחלקה ProductCard והוספנו לה קונסטרוקטור דינמי המקבל פרמטר בוליאני בשם compact. במידה והוא true (כמו במסך תוצאות הסל או במזג האוויר), הכרטיס מקטין את מימדיו אוטומטית, משנה את גודל הגופנים ומעלים פרטים ארוכים (כמו תיאור המוצר) כדי לחסוך מקום בממשק, אך שומר על אותה לוגיקת הוספה לסל המקורית של האתר.

סנכרון ותקשורת נתונים בין רכיבי UI מופרדים

- **האתגר:** כאשר המשתמש לוחץ על כפתור "הוסף לסל" מתוך רכיב כרטיס המוצר (ProductCard), המערכת נדרשת לעדכן את מספר הפריטים המוצג בתוך ה-Badge של העגלה. הבעיה היא שה-Badge הזה נמצא בכלל בתוך רכיב הניווט הראשי של האתר (MainLayout). מאחר ובפלטפורמת Vaadin הרכיבים מופרדים ומבודדים זה מזה, כרטיס המוצר לא מכיר את רכיב הניווט ולא יכול לפנות אליו ישירות, מה שהיה עלול לגרום לכך שהסל יתעדכן מאחורי הקלעים בסשן, אך המשתמש לא יראה את השינוי הוויזואלי במסך ללא ריענון ידני של הדף.
- **הפתרון:** ניצלנו את מנגנון שיתוף הנתונים של Vaadin באמצעות המחלקה ComponentUtil. בתוך מתודת הלחיצה של הכרטיס, הוספנו שורה השולפת בצורה בטוחה את המופע הפעיל של תפריט הניווט מתוך ה-UI הנוכחי MainLayout במידה והוא נמצא, המערכת מפעילה באופן מיידי את המתודה layout.refreshCartCount(). פתרון זה יצר תקשורת נקייה וישירה בין רכיבים מופרדים ושמר על סנכרון מלא של הממשק בזמן אמת.

התאמת מחירי מוצרים עשורניים לאינדקסים של מטריצת האלגוריתם

- **האתגר:** אלגוריתם תרמיל הגב (Knapsack) בגישת תכנות דינמי מחשב פתרונות על ידי שימוש בערכי המשקל/עלות כאינדקסים (מיקומים) בתוך מערך או מטריצה. בשל כך, האלגוריתם מחייב לעבוד אך ורק עם מספרים שלמים. מצד שני, בעולם האמיתי מחירי בגדים הם מספרים עשורניים המכילים אגורות (למשל: 169.90 ש"ח). אם היינו מעגלים את המחירים למספרים שלמים רגילים כדי להתאים אותם לאלגוריתם, המערכת הייתה מאבדת דיוק ומציגה תוצאות שגויות ללקוח, ומצד שני שימוש במספר עשורני כאינדקס במערך אינו אפשרי בקומפילציה.
- **הפתרון:** יישמנו מנגנון המרה וסנכרון של יחידות מחיר. בבסיס הנתונים שמרנו את מחירי הבגדים כמספרים שלמים מוכפלים (לדוגמה, מחיר של 169.90 ש"ח נשמר בקוד כ-1699). כאשר המוצר מוצג ללקוח ב- ProductCard, אנחנו מחלקים את המחיר ב-10.0 ומציגים את המחיר הריאלי בשקלים. מנגד, כאשר המשתמש מפעיל את האלגוריתם, המערכת לוקחת את התקציב שהוזן, מכפילה אותו ב-10 ומעגלת אותו למספר שלם. בצורה זו, האלגוריתם מסוגל לרוץ על אינדקסים שלמים ומדויקים בשרת, מבלי לאבד את הדיוק של האגורות בחישוב הסופי.

עקיפת אילוצי הרשאות מיקום בשילוב שירות מזג האוויר

- **האתגר:** כדי להציג את רכיב המלצות הלבוש לפי מזג האוויר בדף הבית, המערכת צריכה לדעת מה המיקום הגיאוגרפי של המשתמש. הגישה הסטנדרטית של שימוש ברכיב הדפדפן האוטומטי מקפיצה חלון אישור מציק, שמשתמשים רבים נוטים לחסום מטעמי פרטיות. חסימה כזו מחזירה שגיאה ומצריכה קוד טיפול מורכב, בנוסף לסרבול שבניסיון להעביר קואורדינטות (קו אורך ורוחב) מה - Frontend אל השרת בצורה אסינכרונית, עבור רכיב שהוא בסך הכל תוספת משנית באתר.
- **הפתרון:** העברנו את השליטה והבחירה בצורה נקייה לידי המשתמש. המערכת מאפשרת ללקוח להזין את שם העיר שלו - אם כחלק מתהליך הרישום והקמת פרופיל המשתמש באתר, ואם בצורה ישירה ומהירה בתוך שדה קלט ייעודי ברכיב ההמלצה עצמו. ה - Backend מקבל את מחרוזת הטקסט הפשוטה של העיר ומשבץ אותה ישירות בתוך כתובת ה - URL של בקשת ה - HTTP GET שנשלחת אל OpenWeather API. פתרון זה מבטיח שהשירות ישלוף נתונים ריאליים ויעבוד תמיד ב-100% מהפעמים, ללא שגיאות קוד וללא תלות בהרשאות משתנות של דפדפנים.

5. הגדרת מדדי הצלחה למערכת

פרק זה מגדיר את המדדים המעשיים והכמותיים שקובעים האם המערכת פועלת בצורה תקינה, ומאפשרת לוודא שהאפליקציה עומדת בכל דרישות הפיתוח וההנדסה שהוגדרו עבורה.

- **דיוק מתמטי ועמידה מוחלטת באילוץ התקציב:** בכל הרצה של אלגוריתם הסל החכם, סך העלות הכוללת של מוצרי הלבוש המוצעים לא יעבור בשום מצב את תקרת התקציב שהגדיר המשתמש בשדה הקלט, תוך שמירה על דיוק מלא ברמת האגורות (בזכות מנגנון הכפלת המחירים ב-10 במסד הנתונים).
- **התכנסות במגבלות הכמות:** מספר המוצרים שיוצעו ללקוח בסיום ריצת האלגוריתם יתכנס במדויק בתוך הטווח שהוגדר על ידו בשדות "מינימום פריטים" ו"מקסימום פריטים" (לדוגמה: אם הוגדר בין פריט אחד ל-3, המערכת לא תציע סל של 4 פריטים או סל ריק).
- **סנכרון מיידי של עגלת הקניות בממשק:** לחיצה על כפתור "הוסף לסל" מתוך כרטיס מוצר (ProductCard) תכניס את המוצר לסשן ותעדכן אוטומטית ובזמן אמת את מספר הפריטים ב - Badge של העגלה הכללית ב - MainLayout, ללא שגיאות וללא צורך בריענון ידני של הדף על ידי הלקוח.
- **ריצה אסינכרונית יציבה וחווית משתמש רציפה:** בזמן שהאלגוריתם מריץ את החישובים מאחורי הקלעים, מסך הדפדפן של המשתמש יישאר פעיל לחלוטין ולא ייקפא. מד ההתקדמות יציג חיווי ויזואלי זורם של שלבי הריצה השונים עד להצגת כרטיסי התוצאה המותאמים.
- **ניהול דינמי מלא (CRUD) בצד האדמין:** כל פעולת ניהול שיבצע האדמין במסך השליטה (הוספה/עריכה/מחיקה של בגדים בקטלוג, עדכון משתמשים, או שינוי שדות השאלון ב-MongoDB) תתבצע בצורה מאובטחת, ותשתקף מיד בממשק הלקוח ללא צורך בריענון השרת או בשינוי קוד.
- **אבטחת גישה והפרדת הרשאות:** מערכת הסינון וההגנה של הדפים (BeforeEnterObserver) תחסום ב-100% מהמקרים גישה של משתמשים רגילים או אורחים לדף הניהול הרגיש, ותפנה אותם אוטומטית לדף הבית.
- **זמינות ויציבות שירות מזג האוויר:** פנייה ל - OpenWeather API באמצעות העיר שהזין המשתמש (או ברישום או ברכיב עצמו) תחזיר נתונים ריאליים ותציג המלצה

ויזואלית בתוך ה- ProductCard הקומפקטי ב-100% מהפעמים, ללא תלות בהרשאות דפדפן וללא חסימות.

- **אבטחת סיסמאות המשתמשים ומניעת חשיפת נתונים:** סיסמאות המשתמשים אינן נשמרות בשום שלב כטקסט גלוי במסד הנתונים (MongoDB). מדד ההצלחה הוא ערבול והצפנה מלאה של 100% מהסיסמאות בשכבת ה- Backend לפני שמירתן, כך שגם במצב של גישה ישירה למסד הנתונים - הסיסמה המקורית אינה חשופה בשום צורה.
- **דיוק מרבי בהתאמה לסגנון המשתמש (מניעת רנדומליות):** אלגוריתם הצעת הסל אינו בוחר פריטים בצורה אקראית. מדד ההצלחה נקבע על פי התאמה מלאה בין הבחירות של המשתמש בשאלון לבין התגיות של המוצרים בקטלוג. המערכת מחשבת ציון התאמה ריאלי ומציעה אך ורק את חבילת הלבוש בעלת הניקוד הגבוה ביותר עבור הטעם הסגנוני של הלקוח, תחת מגבלות התקציב.

6. תיאור פתרונות קיימים לבעיה וניתוח חלופות

פרק זה סוקר את הפתרונות הקיימים כיום בשוק המסחר האלקטרוני להתאמת מוצרים עבור לקוחות, ומציג ניתוח חלופות שנעשה בשלבי אפיון המערכת.

פתרונות קיימים בשוק

כיום, ברוב אתרי האופנה והבגדים הגדולים (כמו עלי-אקספרס, אסוס או חנויות Shopify סטנדרטיות), פתרון התאמת המוצרים ללקוח מבוסס על שני מנגנונים עיקריים:

1. **מערכת סינונים קשיחה (Filtering):** המשתמש צריך לבחור בעצמו קטגוריה, טווח מחירים, מידה וצבע. האתר מציג לו רשימת מוצרים שעונים על הסינון, והלקוח צריך להתחיל לחשב ידנית אילו מוצרים לשלב כדי להגיע לחבילת בגדים שלמה שעומדת בתקציב שלו.
2. **באנרים סטטיים של "מוצרים מומלצים":** מערכות המציגות ללקוח מוצרים דומים למה שהוא צפה בהם, ללא קשר ריאלי לתקציב כולל של סל קניות מוגדר מראש וללא בדיקה של אילוצי כמויות פריטים.

ניתוח חלופות טכנולוגיות לפיתוח המערכת החכמה

כדי להתמודד עם מורכבות בעיית ה-MDKP באתר, נבחנו בספרות ארבע גישות אלגוריתמיות עיקריות. להלן טבלת השוואה המנתחת את הפתרונות הקיימים ומסבירה את הפער ביניהם:

שם הגישה / אלגוריתם	מנגנון הפעולה	יתרונות	חסרונות	הפער בפתרונות הקיימים
אלגוריתם חמדן	בחירה בכל צעד בפריט עם היחס עלות/תועלת הטוב ביותר באותו רגע.	זמן ריצה מהיר מאוד $O(N \log N)$ וקל למימוש תכנותי.	אינו מבטיח פתרון אופטימלי ועלול להשאיר "חורים" לא מנוצלים בתקציב.	חוסר דיוק: בבעיות פיננסיות, פתרון שאינו אופטימלי (קירוב) נתפס כחיסרון משמעותי שאינו ממצה את משאבי הלקוח.
כוח גס (Brute Force)	סריקת כל הקומבינציות האפשריות של מוצרים בקטלוג כדי למצוא את הטובה ביותר.	מבטיח ב-100% את מציאת הפתרון האופטימלי המוחלט (הטוב ביותר האפשרי).	זמן ריצה מעריכי שגדל בצורה קיצונית עם כל פריט נוסף.	חוסר יעילות: השיטה אינה ישימה במערכות זמן-אמת עם קטלוגים גדולים, שכן זמן החישוב יהיה ארוך מדי.
תכנות דינמי	פירוק הבעיה לתתי-בעיות קטנות ושמירת תוצאותיהן בטבלה למניעת חישובים חוזרים.	מבטיח פתרון אופטימלי מוחלט (כמו כוח גס) אך בזמן יעיל משמעותית (פסאודו-פולינומיאלי).	צריכת זיכרון התלויה בגודל הטבלה (מכפלת האילוצים).	זוהי הגישה היחידה המאפשרת דיוק מתמטי מושלם בזמן ריצה סביר עבור ערכים מספריים שלמים בלבד.

אלגוריתמי קירוב	משפחת אלגוריתמים המבטיחים מציאת פתרון הקרוב לאופטימלי עד כדי שגיאה ידועה מראש (למשל FPTAS).	זמן ריצה פולינומיאלי, מתאים מאוד לבעיות Big Data עם מיליוני פריטים.	הפתרון אינו אופטימלי ב-100% אלא רק "קרוב מספיק". המימוש לעיתים מורכב מאוד	כאשר הקלטים הם בטווח סביר (כמו במערכות מסחר רגילות), אין הצדקה להתפשר על דיוק לטובת מהירות, שכן ניתן להשיג את שניהם.
-----------------	---	---	---	--

לאור הסקירה המוצגת, הספרות האקדמית מצביעה על גישת התכנות הדינמי כפתרון המאוזן ביותר עבור בעיות MDKP בעלות אילוצים מספריים בדידים בטווחים סבירים. ניתוח זה מראה כי בעוד שגישות הקירוב או החמדתן מתפשרות על דיוק המתמטי, וגישת הכוח הגס מתפשרת על זמן הריצה, תכנות דינמי מאפשר להשיג הן דיוק מוחלט והן יעילות חישובית, ולכן הוא נחשב לסטנדרט המועדף לפתרון אתגרים מסוג זה במערכות זמן-אמת.

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

בהתבסס על ניתוח החלופות, הסקירה המדעית והפערים במצב הקיים שהוצגו בפרק 6, החלופה שנבחרה למימוש מנוע המלצות הלבוש במערכת "הסל החכם" היא גישת התכנות הדינמי (Dynamic Programming). להלן הנימוקים ההנדסיים והמעשיים לבחירת חלופה זו והיתרונות שלה על פני הפתרונות הקיימים בשוק:

- **פתרון פער "הסינון הקשיח":** בניגוד לאתרים הקיימים בשוק (כמו אסוס או עלי-אקספרס) המאלצים את המשתמש לבצע סינונים ידניים מתישים ולחשב בעצמו שילובים, החלופה הנבחרת מייתרת את העבודה הידנית. האלגוריתם מקבל את התקציב ואת דרישות הכמות, ובונה עבור המשתמש באופן אוטומטי ומיידית חבילת לבוש שלמה ומותאמת אישית, ללא צורך בניסוי וטעיה מצד הלקוח.
- **הכרח בדיוק מתמטי מוחלט (עמידה באילוץ התקציב):** כפי שהוגדר במדדי ההצלחה, המערכת מחויבת להציע סל בגדים שמנצל את תקציב המשתמש בצורה המקסימלית מבלי לחרוג ממנו אפילו באגורה אחת. חלופות כמו אלגוריתם חמדן או אלגוריתמי קירוב (FPTAS) אינן מבטיחות פתרון אופטימלי, דבר שהיה מוביל לתוצאות שגויות או חלקיות עבור הצרכן.

- **מתאים למגבלות הקלט בפרויקט (מספרים שלמים):** אלגוריתם תכנות דינמי דורש עבודה עם אינדקסים שלמים במטריצה. מאחר ופיתחנו מנגנון Scaling ייעודי ב- Backend המכפיל את מחירי הבגדים ב-10 ומעגל אותם, נתוני החנות הפכו לערכים שלמים בדידים המתאימים בצורה מושלמת למבנה הנתונים של האלגוריתם.
- **גודל קטלוג ומספר אילוצים סביר:** צריכת הזיכרון של תכנות דינמי תלויה בגודל המטריצה (מכפלת האילוצים: תקציב $\times$ כמות פריטים $\times$ מלאי). מאחר ומדובר בחנות בגדים בעלת קטלוג מנוהל וטווח תקציב הגיוני של משתמש ממוצע, ממדי המטריצה נשארים בטווח סביר לחלוטין. השרת מסוגל לבצע את החישוב בתוך מאיות השניה, כך שאין סכנה לקריסת זיכרון כפי שעלול לקרות בבעיות Big Data בקנה מידה עולמי.
- **זמן ריצה מהיר בזמן אמת:** בניגוד לחלופת הכוח הגס (Brute Force), שזמן הריצה שלה הוא מעריכי והייתה תוקעת את דפדפן המשתמש ככל שקטלוג הבגדים גדל, גישת התכנות הדינמי רצה בזמן פסאודו-פולינומיאלי יעיל, המאפשר להחזיר את תוצאת הסל החכם למשתמש באופן מיידי ולשמור על חוויית שימוש רציפה ומהירה באתר.

8. אפיון המערכת המוצעת

פרק זה מציג את אפיון התוכנה והנדסת הדרישות המלא של מערכת "הסל החכם". האפיון מפרט את הצרכים המעשיים, מודלי העבודה, דרישות הביצועים והאילוצים הטכנולוגיים המהווים את הבסיס לפיתוח המערכת.

8.1 ניתוח הדרישות מהמערכת

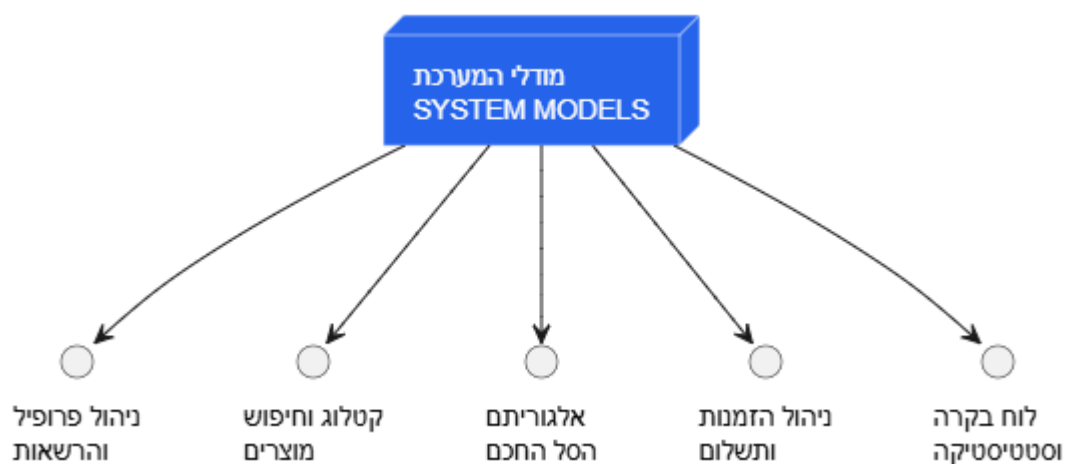
סעיף זה מגדיר את דרישות התשתית והאיכות הרוחביות הנדרשות לפעילותה התקינה של המערכת כפלטפורמה מודרנית:

- **ריבוי משתמשים:** המערכת מיועדת לעבודה מול משתמשים רבים במקביל. הארכיטקטורה מבוססת על ניהול ששנים נפרדים לכל לקוח. בנוסף, הרצת אלגוריתם האופטימיזציה מבוצעת על גבי תהליכון משני נפרד כדי למנוע חסימה של ממשק המשתמש הראשי, ולאפשר גלישה רציפה של משתמשים אחרים באתר בזמן חישוב האלגוריתם.
- **שמירת נתונים:** נתוני המערכת נשמרים במסד נתונים מסוג MongoDB. שמירת הנתונים מבטיחה את עקרון ה- Data Integrity (יושרת הנתונים), באמצעות מנגנוני

אימות בשכבת ה- Backend (מניעת אימיילים כפולים, בדיקת תקינות קלטים). כל מחירי המוצרים עוברים המרה והכפלה ב-10 כדי להישמר כמספרים שלמים בלבד, על מנת למנוע איבוד מידע פיננסי או שגיאות עיגול עשרוניות.

- **יעילות וניצול משאבים:** המערכת מנצלת את משאבי השרת ביעילות באמצעות גישת התכנות הדינמי. במקום לבצע חישובים חוזרים ונשנים או סריקה בזבזנית של כלל הקומבינציות בקטלוג, האלגוריתם שומר תוצאות ביניים בטבלת זיכרון (מטריצה), דבר המוריד בצורה דרסטית את צריכת ה- CPU והזיכרון של השרת.
- **מהירות זמן תגובה:** כאתר מסחר בזמן-אמת, מהירות התגובה היא קריטית. מנוע החישוב האלגוריתמי מתוכנן להחזיר תוצאה מיידית ללקוח ללא השהיה מורגשת (בזכות יעילות זמן הריצה של התכנות הדינמי). בנוסף, התקשורת בין רכיבי ה- UI השונים (כמו עדכון כמות הפריטים בעגלה) מבוצעת בצורה ישירה באמצעות ComponentUtil ללא צורך בריענון הדף, מה שמבטיח חוויית שימוש חלקה ומהירה.
- **ליבה פועלת:** הליבה המרכזית של המערכת משלבת בין מנוע החישוב האופטימלי (וריאציה תלת-ממדית של בעיית תרמיל הגב הרב-ממדית - MDKP), לבין מנוע חישוב ציוני התאמה (ScoringService). הליבה פועלת בצורה שמבטיחה התכנסות מלאה של התוצאות תחת אילוצי התקציב והכמויות של המשתמש.

8.2 מודלי המערכת



הסבר על תרשים מודלי המערכת (System Models)

תרשים זה מציג את חמשת עמודי התווך של מערכת **SmartCart** התרשים ממחיש את החלוקה הלוגית של המערכת למודיולים (Modules) עצמאיים, המבטיחים הפרדת רשויות וניהול ארכיטקטוני מסודר:

1. **ניהול פרופיל והרשאות:** המודול האחראי על אבטחת המידע וניהול זהות המשתמש. הוא מבטיח גישה מאובטחת ומבדיל בין הרשאות לקוח רגיל (סל חכם, רכישות) לבין הרשאות מנהל מערכת (ניהול תוכן).
2. **קטלוג וחיפוש מוצרים:** הממשק המאפשר הנגשת נתונים דינמית. מודול זה משתמש בחיפוש מבוסס תגיות (Tags) המאפשר למשתמש לצמצם את מרחב הבחירה שלו בהתאם להעדפות אישיות.
3. **אלגוריתם הסל החכם:** זהו ה "Core" האלגוריתמי של הפרויקט. מודול זה מממש לוגיקה מתקדמת של אופטימיזציה (תכנון דינמי), המבצעת שקלול של נתוני משתמש מול מאגר המוצרים כדי לייצר "חבילת ביגוד" אידיאלית.
4. **ניהול הזמנות ותשלום:** מודול ניהול והיסטוריית הרכישות. הוא מבטיח אמינות נתונים על ידי שמירת מצב המוצר בזמן הרכישה, כך שההיסטוריה הפיננסית נשארת עקבית.
5. **לוח בקרה וסטטיסטיקה:** המודול הניהולי. הוא מספק למנהל המערכת תובנות עסקיות מבוססות נתונים (כמו רווחים חודשיים וניתוח פופולריות מוצרים), מה שמשלים את המערכת והופך אותה מפרויקט פיתוח לכלי עבודה עסקי.

8.3 אפיון פונקציונלי וביצועים עיקריים

חלק זה מתאר את הפונקציות המרכזיות שהמערכת מספקת למשתמשי הקצה ואת רמת הביצועים המוגדרת עבורן:

- **ניהול גישה, הרשמה והתחברות:** המערכת מספקת ממשק מאובטח לרישום משתמשים חדשים ואימות משתמשים קיימים. תהליך זה מנהל את סשן העבודה של הלקוח ומבטיח כי רק משתמשים רשומים יוכלו לגשת לאזור האישי, לשמור את העדפות הסטייל שלהם ולבצע הזמנות באתר.
- **אוטומציה מלאה של בניית סל הלבוש:** המשתמש מזין תקציב וטווח כמותיות, והמערכת מייצרת עבורו חבילת בגדים אופטימלית בזמן תגובה מהיר וכמעט מיידי.

הפונקציה משיגה 100% דיוק מתמטי בניצול התקציב מבלי לחרוג ממנו, ומציגה את התוצאות ישירות על המסך ללא עיכובים.

- **ניהול שאלון סטייל דינמי:** המערכת מריצה בלופ את בחירות המשתמש מול תגיות המוצרים ומחשבת את אחוז ההתאמה הסטייליסטי ללא תלות באף שדה קשיח מראש בקוד.
- **המלצות מזג אוויר חכמות:** פנייה אסינכרונית ל - OpenWeather API על בסיס העיר שהזין המשתמש (בזמן הרישום או ברכיב עצמו), ושליפת המלצת לבוש מותאמת לטמפרטורה הנוכחית ללא צורך בהרשאות מיקום אוטומטיות של הדפדפן.
- **מנגנון רכישה ותיעוד היסטוריית הזמנות:** המערכת תומכת בתהליך סגירת הזמנה ורישום טרנזקציות מלא, תוך הבחנה מובנית בין שני תרחישי קנייה.
רכישה מבוססת אלגוריתם (שבה נשמר לצד הפריטים גם הציון שחושב עבורם בזמן אמת לצורך בקרה וניתוח) ורכישה ידנית עצמאית של הלקוח (שבה שדה הציון מסומן כידני). המערכת מבצעת "צילום מצב" מלא של הפריטים והמחירים ששולמו בפועל ברגע הרכישה, ושומרת אותם בצורה קבועה בהיסטוריית ההזמנות של המשתמש.
- **ממשק ניהול ובקרה:** מתן אפשרות למנהל האתר לבצע הוספה, עריכה, ומחיקה של בגדים ותגיות בזמן אמת, כאשר השינויים משתקפים מיד בממשק הלקוח ללא צורך בריענון השרת.

8.4 אילוצי המערכת

המערכת כפופה למספר אילוצים טכנולוגיים, עסקיים וסביבתיים המשפיעים על אופן מימושה:

- **אילוץ המורכבות האלגוריתמית:** מאחר והבעיה משלבת מספר אילוצים בו-זמנית (מחיר וכמויות), היא מסווגת כ - MDKP שהיא NP-Hard. האילוץ מחייב להשתמש בתכנות דינמי ומטריצות זיכרון ולא בכוח גס, כדי למנוע זמני חישוב מעריכיים ותקיעה של השרת.
- **אילוץ קלט של מספרים שלמים:** אלגוריתם התכנות הדינמי מחייב שימוש בערכים שלמים כאינדקסים במערך. אילוץ זה נפתר בקוד באמצעות הכפלת המחירים העשרוניים מהעולם האמיתי ב-10 ומעבר לעבודה עם שלמים בלבד ב-Backend.

- **אילוץ אבטחה והגנת פרטיות:** חל איסור מוחלט על שמירת סיסמאות משתמשים כטקסט גלוי במסד הנתונים. המערכת מאולצת לבצע תהליך ערבול והצפנה של כל סיסמה טרם שמירתה במסד הנתונים.
- **אילוץ אבטחת גישה:** גישה לדפי הניהול הרגישים מאולצת לעבור בדיקה מקדימה קפדנית באמצעות מנגנון BeforeEnterObserver של Vaadin, החוסם לחלוטין גישה אורחים או משתמשים רגילים ומפנה אותם אוטומטית לדף הבית.
- **אילוץ תלות בשירותים חיצוניים:** מערכת ההמלצות לפי מזג האוויר תלויה לחלוטין בזמינות של שרת ה- OpenWeather API. המערכת מאולצת לכלול מנגנוני טיפול בשגיאות תקשורת, על מנת להבטיח שהאתר לא יקרוס וימשיך לתפקד בצורה עצמאית גם במקרה של נפילת השירות החיצוני.

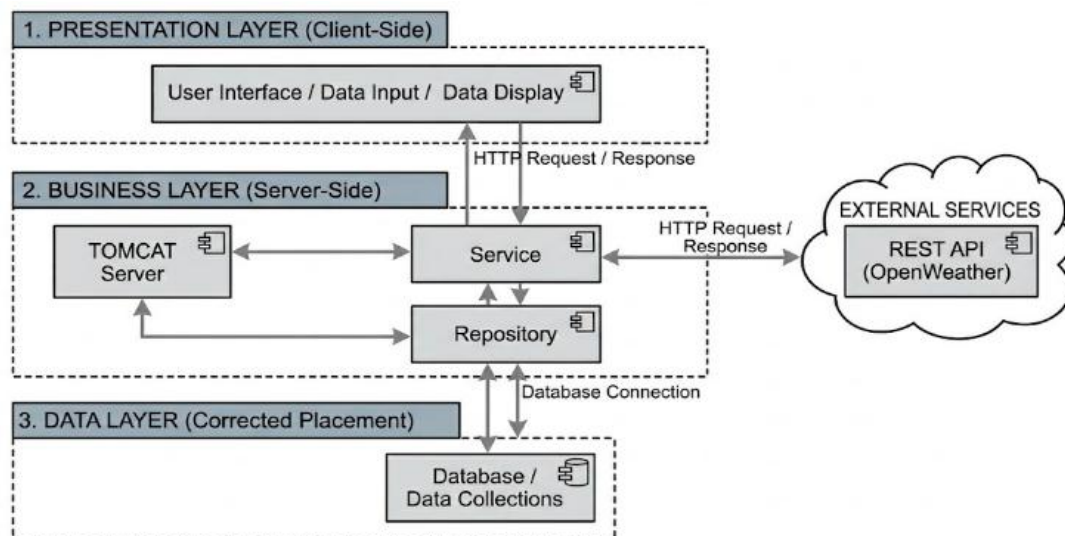
9. תיאור הארכיטקטורה של המערכת

בפרק זה נשלים את אפיון מערכת "הסל החכם" ונפרט על תשתיות התקשורת, תהליכי מערכת ההפעלה והממשקים החיצוניים המאפשרים לאפליקציה לפעול בסביבת רשת מודרנית ומאובטחת.

9.1 ארכיטקטורת המערכת

המערכת תוכננה ופותחה על בסיס **ארכיטקטורת שלוש השכבות** המבוססת על מודל **שרת-לקוח**. בחירה הנדסית זו נועדה לייצר הפרדת רשויות ברורה בין ממשק המשתמש, הלוגיקה העסקית ותשתית הנתונים, ובכך להבטיח את אבטחת המערכת, יציבותה וגמישותה לשינויים עתידיים.

להלן תרשים המתאר את מבנה הארכיטקטורה (שלוש השכבות) וזרימת המידע בין השכבות:



להלן תיאור מעמיק של כל שכבה כמתואר בתרשים:

1. שכבת התצוגה

שכבה זו מהווה את החזית הטכנולוגית של המערכת ורצה כולה בסביבת הדפדפן של המשתמש. תפקידה הוא לשמש כנקודת המפגש האינטראקטיבית בין האדם למכונה. היא אחראית על הרינדור הוויזואלי של רכיבי פלטפורמת Vaadin (כגון כרטיסי המוצר וטפסי השאלון), ניהול חווית המשתמש ואיסוף כלל הקלטים, החל מתהליכי הזדהות ועד להגדרת אילוצי "הסל החכם". מבחינת תקשורת, השכבה מתפקדת כ"לקוח", אורזת את המידע לאובייקטים בפורמט JSON, ושולחת בקשות רשת אסינכרוניות בפרוטוקול HTTP לעבר שכבת הלוגיקה. עם קבלת התשובה, היא מעדכנת את התצוגה למשתמש בזמן אמת.

2. שכבת הלוגיקה והשירותים

ליבת המערכת מתבצעת בצד השרת, בסביבת ריצה ייעודית בשם Apache Tomcat כחלק מ Spring Boot ומרכזת את כלל החוקיות העסקית. בתוך שכבה זו פועלים שני רכיבי תוכנה מרכזיים, הלב הוא רכיב השירות (Service), אשר קולט את בקשות ה-HTTP, מפענח אותן ומריץ את האלגוריתמים המורכבים של המערכת. כדי לבצע את משימתו, רכיב השירות נעזר ברכיב המאגר (Repository). תפקידו של רכיב המאגר הוא לשמש כמתווך טכני לנתונים ולתרגם את הבקשות הלוגיות לשאילתות מסד נתונים. במקביל, רכיב השירות מסוגל ליזום תקשורת ישירה החוצה לשירותי צד-שלישי (REST API) לצורך העשרת המידע, ללא תלות במסד הנתונים הפנימי.

3. שכבת הנתונים והאינטגרציה

תשתית המידע המרכזית שבה נשמרים נתוני האפליקציה בצורה קבועה, מאובטחת ומסודרת. שכבה זו מבוססת לחלוטין על מסד הנתונים MongoDB (היושב על שרת אחסון ענן ב - MongoDB Atlas) ומנהלת את אחסון ושליפת המידע מתוך האוספים השונים של המערכת כמו, מוצרים (בגדים), משתמשים, תגיות השאלון הדינמיות והיסטוריית ההזמנות. שכבה זו אחראית על שלמות המידע ואספקתו לשירותי ה - Backend, זרימת המידע מוגנת בצורה קפדנית, כך ששכבת התצוגה מופרדת ממנה לחלוטין ואינה מסוגלת לגשת למסד הנתונים בשום שלב, כל פעולת קריאה, כתיבה או עדכון של מידע מתבצעת אך ורק דרך תיווך ואימות של שכבת הלוגיקה העסקית.

9.2 מודל שרת לקוח

פעילות המערכת מושתתת על מודל שרת-לקוח המנהלת את חלוקת העבודה, ריכוז הסמכויות וזרימת המידע הדינמית בין רכיבי הרשת השונים:

- **צד הלקוח:** מיוצג על ידי דפדפן האינטרנט של משתמש הקצה (הן עבור ממשקי הלקוחות והן עבור ממשקי הניהול והבקרה של המערכת). תפקידו של צד הלקוח מתמקד בהצגה ויזואלית של נתונים, רינדור רכיבי הממשק, וניהול מצבי הקלט השונים. הוא אחראי לקלוט את אירועי המשתמש (UI Events) החל מטפסי הרשמה והזדהות, דרך הזנת פרמטרים ואילוצים במסכים השונים, ועד לביצוע פעולות רכישה ועדכון, ולתרגם אותם לבקשות מובנות (Requests) המועברות לשרת. מטעמי ביצועים, אבטחת מידע ושמירה על שלמות הנתונים, צד הלקוח אינו מבצע חישובים לוגיים מורכבים ואינו מקיים קשר ישיר עם שכבת האחסון.
- **צד השרת:** מיוצג על ידי שרת ה - Backend המבוסס על תשתית Spring Boot. השרת פועל כמאזין קבוע לפניות המגיעות מרשת האינטרנט, מנתח את הבקשות הנכנסות ומנתב אותן לרכיבי הלוגיקה המתאימים. השרת אחראי על הפעלת מנגנוני אימות ואבטחה, הרצת אלגוריתמים ומנועי חישוב, ניהול תהליכים אסינכרוניים, וביצוע מניפולציות (קריאה, כתיבה, עדכון ומחיקה) מול מסד הנתונים MongoDB. לאחר השלמת העיבוד הלוגי, השרת אורז את התוצאות ומחזיר תגובה מובנית, המאפשרת לצד הלקוח לעדכן את התצוגה בהתאם.

9.3 תהליכים של מע' ההפעלה

מימוש מודל שרת-לקוח והעברת הנתונים הדינמית באפליקציה נשענים באופן ישיר על מנגנוני הליבה של מערכת ההפעלה (בצד השרת ובמכשיר הקצה) ועל משאבי החומרה המנהלים את זרימת התקשורת ואת הרצת קוד המערכת:

- **תשתית סוקטים וחבוריות רשת:** מאחר והמערכת מבוססת על תקשורת דו-כיוונית דינמית, מערכת ההפעלה מייצרת צינור תקשורת לוגי המבוסס על סוקטים . הסוקט מהווה את נקודת הקצה המשלבת את כתובת ה - IP של המכשיר עליו רצה המערכת יחד עם פורט התקשורת. כאשר הלקוח מעוניין לבצע פעולה או לצרוך מידע, מערכת ההפעלה בדפדפן פותחת סוקט מקומי ומקשרת אותו לסוקט המאזין בצד השרת, ובכך נוצר ערוץ ישיר להזרמת נתונים הנדרש לקיומו של מודל שרת לקוח.
- **הקצאת פורטים :** מערכת ההפעלה של השרת מקצה פורט לוגי ייעודי (במקרה שלנו פורט 8080 המוקצה עבור שרת היישומים Tomcat המובנה ב-Spring Boot) לצורך האזנה קבועה לבקשות רשת נכנסות. כל פנייה של דפדפן, מכל סוג מסך או ממשק, מנותבת על ידי רכיבי הרשת של מערכת ההפעלה אל הפורט הספציפי הזה, וממנו היא מתורגמת ישירות לאירועים מנוהלים בתוך קוד ה - Backend.
- **רכיבי רשת פיזיים - כרטיס רשת:** התקשורת הלוגית מתורגמת לפעולת חומרה פיזית דרך כרטיס הרשת של השרת ושל מכשיר הקצה. מערכת ההפעלה מקבלת את חבילות המידע הדיגיטליות (בפורמט הנתונים שנבחר), ומנחה את כרטיס הרשת לתרגם אותן לאותות פיזיים המשודרים על גבי תשתית האינטרנט. בצד המקבל, כרטיס הרשת קולט את האותות, מפענח אותם ומעביר אותם בחזרה למערכת ההפעלה כרצף סיביות תוכנתי תקין.
- **ניהול תהליכים ומקביליות:** מערכת ההפעלה מנהלת את חלוקת המשאבים וזמן העיבוד של המעבד באמצעות תהליכים. שרת ה - Backend מנצל מנגנון זה כדי לטפל בבקשות של משתמשים מרובים במקביל, כך שכל פנייה מנותבת לתהליכון נפרד. בנוסף, האפליקציה משתמשת בתהליכים אסינכרוניים משניים לצורך הרצת מנועי חישוב, עיבודי נתונים כבדים או פניות לשירותים חיצוניים. פיצול זה מאפשר למערכת ההפעלה להמשיך לנהל את ממשק המשתמש העיקרי ולשמור עליו רציף, מבלי שפעולות רקע מורכבות יגרמו לעיכובים, השהיות או קפיאה של הדפדפן בצד הלקוח.

9.4 תיאור פרוטוקולי תקשורת

להלן פירוט פרוטוקולי התקשורת ופורמטי העברת הנתונים המשמשים להעברת המידע בין הלקוח לשרת, תוך הסבר על תפקידם והסיבות ההנדסיות לבחירתם:

- **פרוטוקול TCP:** פרוטוקול תעבורה מוכוון חיבור המבטיח העברת נתונים אמינה, מסודרת וללא שגיאות. הפרוטוקול מוודא שכל חבילת מידע שנשלחת מהדפדפן אכן הגיעה לשרת, ובמידה וחבילה הלכה לאיבוד היא נשלחת שנית. אמינות זו היא תנאי הכרחי למערכת מסחרית המנהלת הזמנות כספיות וחישובים אלגוריתמיים מדויקים.
- **פרוטוקול IP:** פרוטוקול האחראי על הכתובות והניתוב הפיזי של חבילות המידע ברשת הגלובלית. הוא משלים את פרוטוקול ה-TCP-תחת מודל (TCP/IP) ומבטיח שכל בקשת חישוב או הוספה לסל תדע למצוא את דרכה הפיזית בין כתובת ה-IP של המשתמש לכתובת ה-IP של שרת האפליקציה
- **פרוטוקול HTTP:** פרוטוקול שכבת האפליקציה המהווה את הבסיס להעברת נתונים, דפי רשת ומסמכים דינמיים במודל שרת-לקוח. הפרוטוקול מגדיר את המבנה של בקשות הרשת הנשלחות מצד הלקוח (HTTP Requests) הכוללות מתודות כגון GET, POST, PUT, DELETE) ואת מבנה התשובות החוזרות מצד השרת (הכוללות סטטוס קוד דוגמת 200 OK או 404 Not Found). בחירה זו מאפשרת סנכרון מובנה וסטנדרטי להעברת כלל ישויות המידע, פעולות המשתמשים ועדכוני הממשק השונים באפליקציה על גבי רשת האינטרנט
- **פורמט העברת נתונים JSON:** פורמט טקסטואלי קליל וסטנדרטי המשמש לייצוג והעברת אובייקטים מורכבים (כמו נתוני בגדים, תגיות ואילוצי תקציב) בתוך גוף בקשות ותגובות ה-HTTP. הפורמט נבחר בשל היותו קריא, חסכוני במשאבי רשת, ונתמך באופן טבעי הן על ידי הדפדפן והן על ידי שרת ה-Spring Boot.

9.5 שירותים חיצוניים

המערכת משלבת שירותים חיצוניים על מנת להעשיר את חוויית השימוש ולהעניק ערך מוסף ללקוח בדף הבית:

- **אינטגרציה מול OpenWeather API:** לצורך מימוש רכיב המלצות הלבוש החכם, השרת הלוגי פונה אל שירות צד-שלישי חיצוני המציע נתונים מטאורולוגיים בזמן אמת. התקשורת מול השירות מבוססת על ארכיטקטורת REST API באמצעות בקשות HTTP GET מאובטחות. השרת שולח את שם העיר שהזין המשתמש, ומקבל

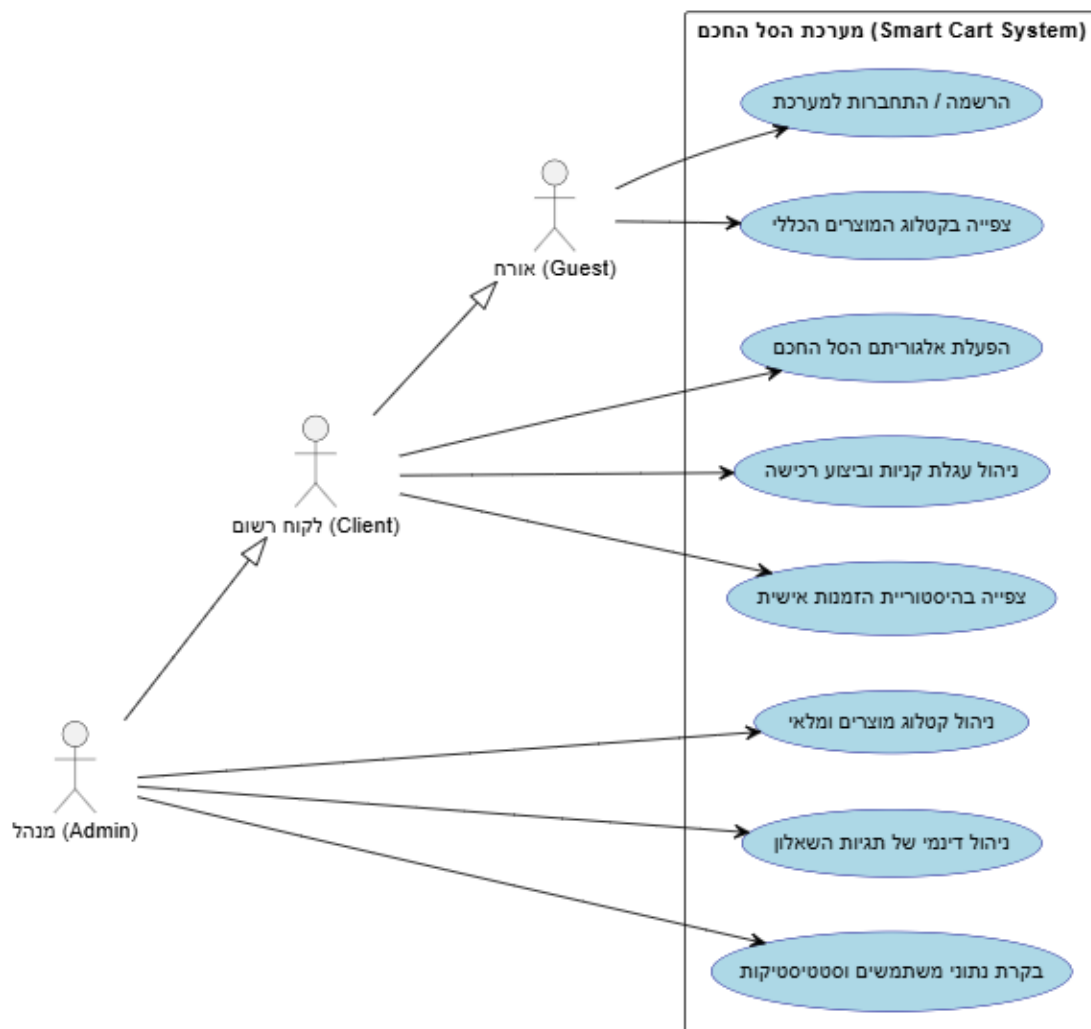
בחזרה קובץ נתונים מובנה בפורמט JSON המכיל את הטמפרטורה העדכנית, לפיה נקבע סוג הבגדים שיוצג למשתמש בתוך הוידג'ט בדף הבית.

10. ניתוח תרחישים וזרימת המידע במערכת המוצעת

פרק זה מציג את האינטראקציה של המשתמשים השונים מול המערכת ואת אופן זרימת הנתונים בין רכיבי התוכנה, תוך שימוש בתרשימי הנדסת תוכנה (UML).

10.1 תרשימי תרחיש Use case Diagram

להלן תרשימי Use-Case של המערכת אשר מציג פעולות מרכזיות אותם יכול המשתמש לבצע מול המערכת:



להלן פירוט משתמשי המערכת ומקרי השימוש:

משתמשי המערכת

- **אורח (Guest):** משתמש קצה ראשוני שאינו מזוהה במערכת. הרשאותיו מוגבלות לפעולות קריאה וצפייה בלבד, וכן לאינטראקציה ראשונית המיועדת להרשמה והתחברות לאתר. משתמש זה פועל בסביבה של קריאה-בלבד ללא שום יכולת השפעה על מסד הנתונים, ואינו מורשה לבצע אינטראקציה מסחרית או חישובית מכל סוג שהוא.
- **לקוח רשום (Client):** משתמש מחובר שעבר תהליך אימות. הוא יורש את זכות הצפייה של האורח, ומקבל הרשאה מורחבת לביצוע פעולות אינטראקטיביות, ניהול תהליכים אישיים וביצוע רכישות.
- **מנהל מערכת (Admin):** משתמש מורשה לניהול תשתית האתר. הוא יורש את כלל יכולות הלקוח הרשום (בגלישה, הפעלת אלגוריתם וקנייה), ובנוסף מחזיק בגישה בלעדית לכלי הבקרה והעריכה של האתר.

פירוט מקרי השימוש של משתמש אורח (Guest)

- **הרשמה והתחברות למערכת:** מאפשר למשתמש חדש ליצור חשבון אישי במערכת על ידי הזנת פרטים בסיסיים, או לאמת משתמש קיים באמצעות שם משתמש וסיסמה על מנת לפתוח סשן עבודה מאובטח ואישי.
- **צפייה בקטלוג המוצרים הכללי:** מאפשר למשתמש לצפות בכלל פריטי הלבוש הקיימים במלאי החנות, לדפדף בין הקטגוריות השונות ולקבל רושם ראשוני מהמגוון הקיים באתר ללא צורך בהזדהות מוקדמת.

פירוט מקרי השימוש של לקוח רשום (Client)

- **הפעלת אלגוריתם "הסל החכם":** תרחיש הליבה של הלקוח באתר. המשתמש מזין את אילוצי הקנייה שלו הכוללים הן את העדפות הסטייל והטעם האישי שלו מתוך שאלון ההעדפות, והן את מגבלות התקציב הכספי וטווחי כמויות הבגדים הרצויות. עם אישור הנתונים, המערכת מפעילה את מנוע החישוב ומציגה ללקוח את שילוב הפריטים האופטימלי והמותאם ביותר עבורו.
- **ניהול עגלת קניות וביצוע רכישה:** מאפשר למשתמש לבחון את הפריטים שנבחרו עבורו (או פריטים שבחר באופן ידני), לעדכן כמויות, ולבצע תהליך קנייה סופי המעביר את פרטי העסקה לרישום קבוע וסגור במערכת.
- **צפייה בהיסטוריית הזמנות אישית:** מאפשר ללקוח לגשת לאזור האישי שלו ולצפות בכלל הרכישות שביצע בעבר באתר, תוך קבלת פירוט מלא על מצב הפריטים והסכומים ששולמו בפועל בכל הזמנה.

פירוט מקרי השימוש של מנהל מערכת (Admin)

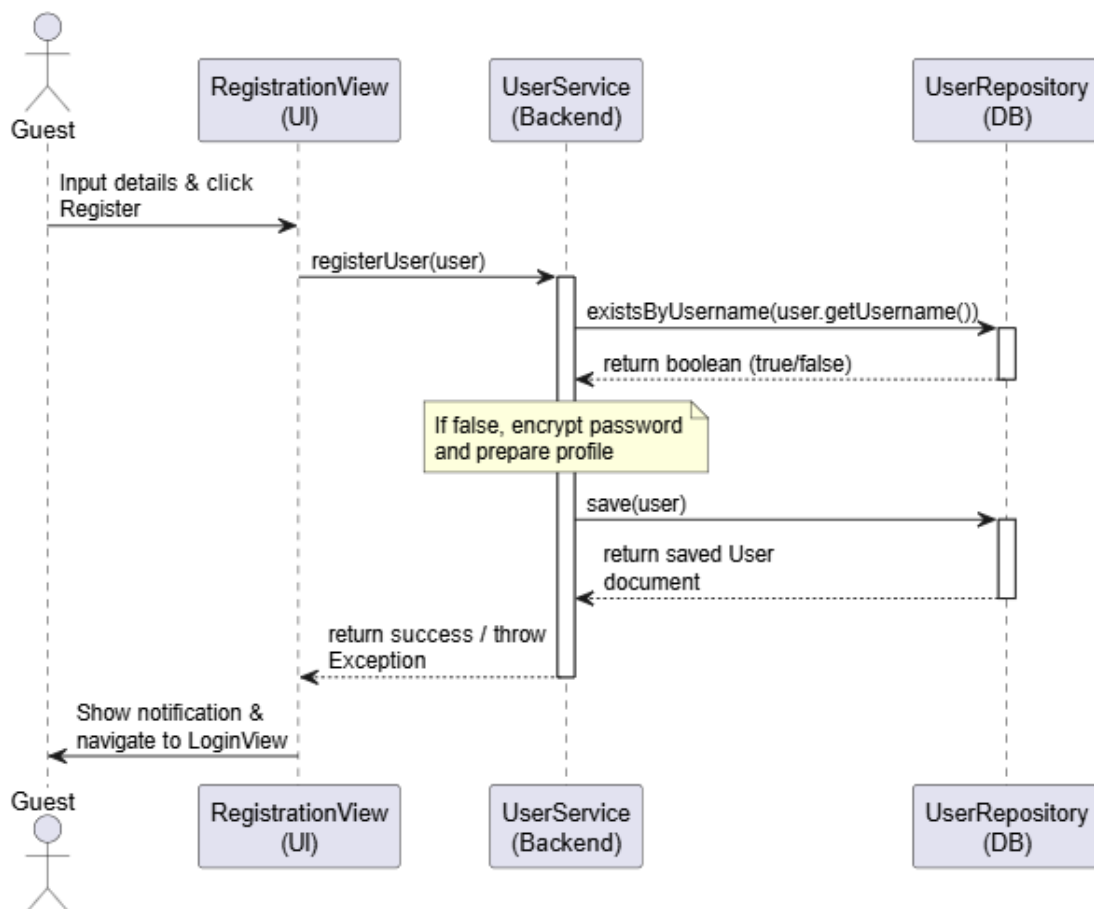
- **ניהול קטלוג מוצרים ומלאי:** מאפשר לאדמין לבצע פעולות הוספה, עריכה, עדכון ומחיקה של פריטי לבוש בקטלוג האתר (שם, מחיר, תמונה וקטגוריה) על מנת לשמור על מלאי עדכני ורלוונטי.
- **ניהול דינמי של תגיות השאלון:** מאפשר למנהל המערכת להוסיף או להסיר מאפיינים, קטגוריות ותגיות המשפיעים ישירות ובזמן אמת על השאלון שיוצג ללקוחות, ללא צורך בשינויי קוד קשיחים באפליקציה.
- **בקרת נתוני משתמשים וסטטיסטיקות:** מאפשר לאדמין לצפות ברשימת החשבונות הרשומים באתר, ולעקוב אחר נתונים סטטיסטיים המשקפים את היקף הפעילות והמכירות הכללי במערכת.

10.2 תרשימי רצף Sequence Diagram

תרשימי הרצף הבאים מתארים את סדר הפעולות הכרונולוגי ואופן מעבר המידע בין השכבות השונות במערכת על בסיס מימוש הפונקציות והמחלקות בקוד האפליקציה:

תרחיש הרשמה למערכת (Registration)

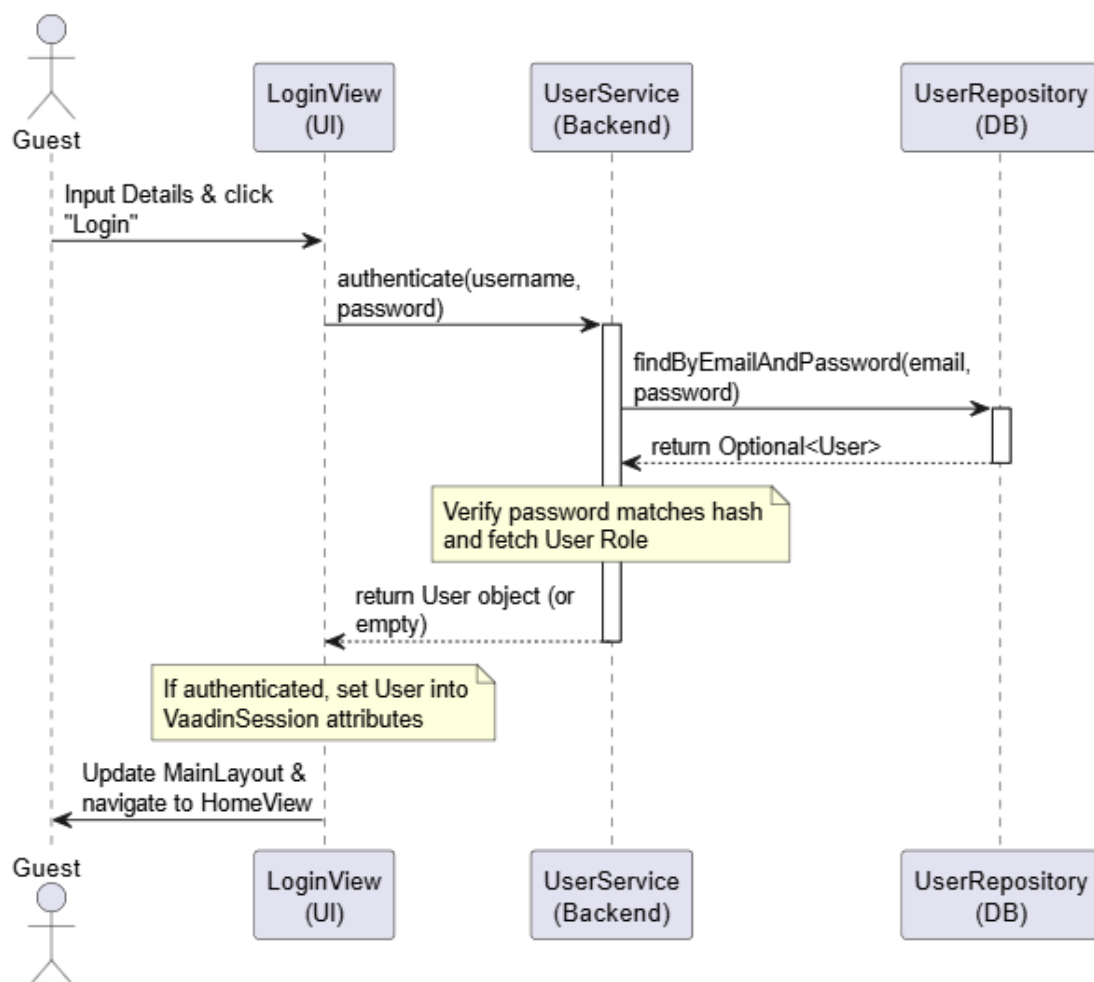
תרחיש זה מתאר את קליטת פרטי הרישום של אורח חדש דרך מסך הרישום, בדיקת יחודיות החשבון ושמירתו.



בתרחיש זה, המידע זורם כאשר המשתמש האורח מזין את נתוני הקלט שלו במסך הרישום (RegistrationView) הכוללים מייל, שם משתמש, סיסמה ועיר מגורים ולוחץ על כפתור האישור. נתונים אלו נארכזים ומועברים ישירות אל שכבת הלוגיקה בשרת (UserService), המפעילה שאילתת בדיקה מול מסד הנתונים (UserRepository) כדי לוודא שהמייל אינו קיים במערכת. במידה והבדיקה מחזירה ערך תקין, השרת מפעיל רכיב אבטחה המצפין ומערבל את הסיסמה לפורמט מוגן, ומנחה את שכבת הנתונים לבצע שמירה קבועה של מסמך המשתמש החדש ב- MongoDB, אשר מחזיר אישור הצלחה המאפשר ל- UI להציג למשתמש פלט של הודעת משוב חיובית ולהעביר אותו אוטומטית אל מסך ההתחברות.

התחברות למערכת (Login)

מתאר את תהליך אימות המשתמש (או האדמין) מול בסיס הנתונים ופתיחת סשן העבודה.

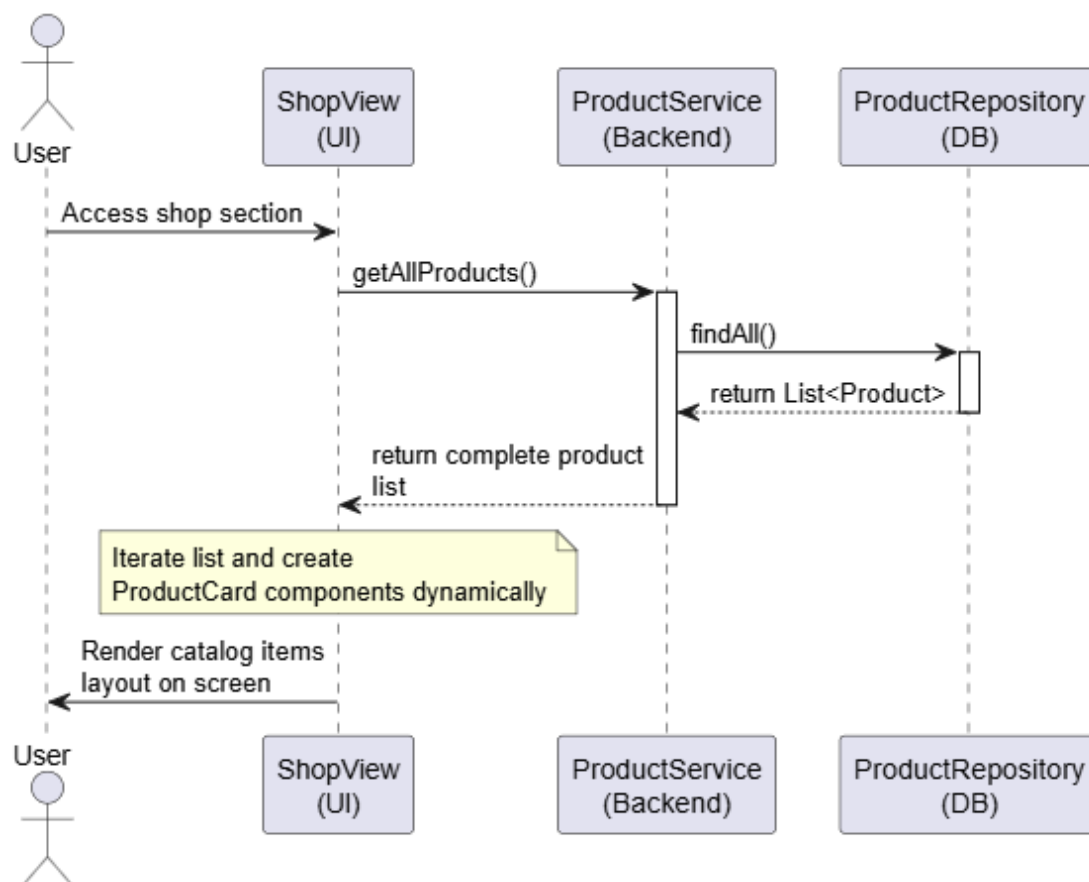


תהליך הזיהוי והאימות מתחיל כאשר המשתמש מזין את נתוני הקלט שלו, המורכבים משם המשתמש והסיסמה, בתוך טופס ה- LoginView ומאשר את הפעולה. נתוני קלט אלו מועברים מיד אל ה- Backend לשירות ה- UserService, אשר פונה לשכבת הנתונים

(UserRepository) ושולף את מסמך המשתמש המלא מה MongoDB הכולל את תפקידו ואת הסיסמה המוצפנת שלו. השרת מבצע חיפוש התאמה למייל ולקלט הסיסמה הנוכחיים בתוך מסמך המשתמשים, ובמידה וישנה התאמה מלאה, הוא מחזיר לממשק המשתמש נתוני פלט של אובייקט משתמש מאומת המכיל את רמת ההרשאה שלו, מה שמאפשר לטופס ה UI לשמור את זהותו בתוך מאפייני הסשן המאובטח של הדפדפן (VaadinSession), לרענן את סרגל הניווט הרחובי בהתאם לתפקידו ולהפנות אותו ישירות אל דף הבית.

צפייה בקטלוג המוצרים

מתאר את טעינת דף החנות ושליפת המוצרים לצורך הצגתם לכל סוגי המשתמשים.

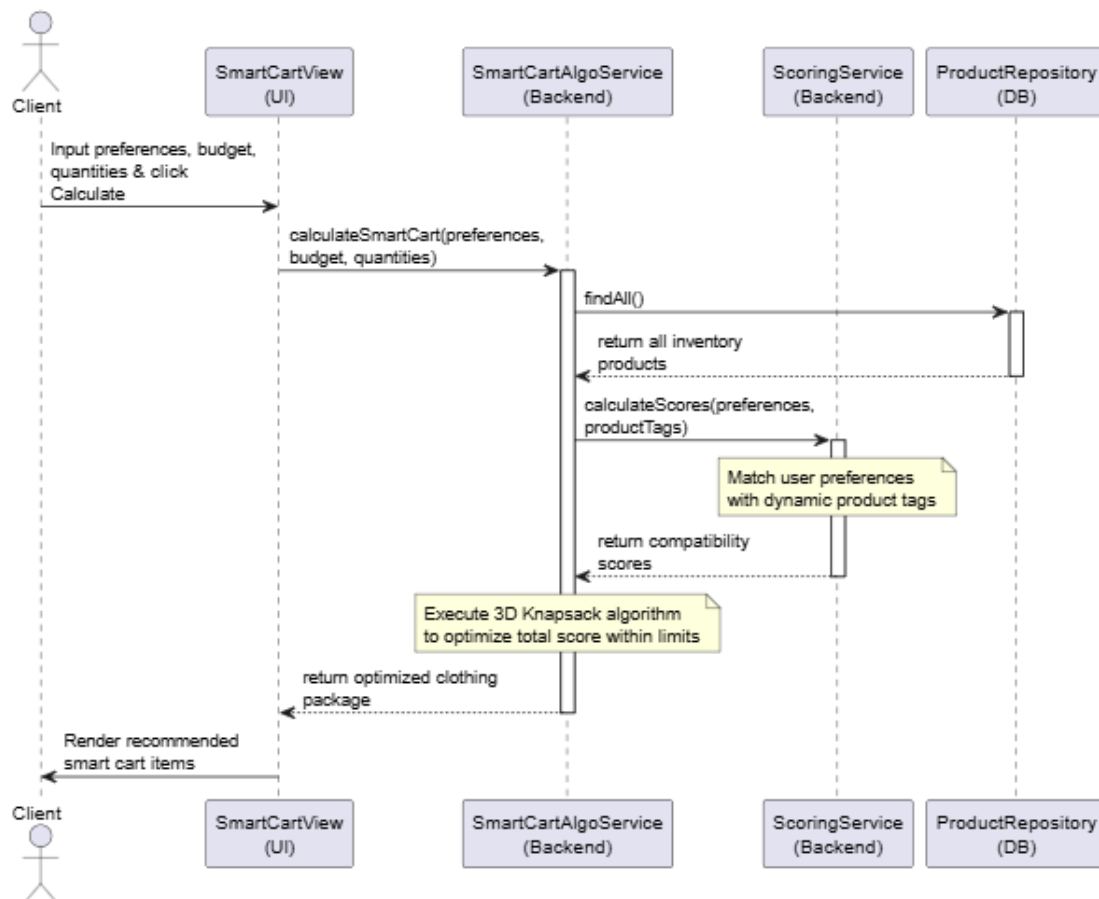


תרחיש זה מייצג פעולת גלישה וצפייה במערכת, שבה עם כניסת המשתמש אל לשונית החנות (ShopView), רכיב הממשק יוזם אוטומטית בקשת שליפה המועברת אל שכבת השירות של המוצרים (ProductService). השירות מפעיל פנייה אל ה ProductRepository ומבצע שאילתת שליפה רחבית של כלל פריטי הלבוש הקיימים במערכת, ומסד הנתונים MongoDB מחזיר כנתוני פלט רשימה מובנית של כל המוצרים על כלל מאפייניהם (שמות, מחירים, תמונות ותגיות סגנון), מה שמאפשר לשכבת הממשק לקלוט את האוסף, לעבד

אותו ולרנדר בצורה דינמית כרטיסי תצוגה ויזואליים (ProductCard) המוצגים למשתמש על גבי המסך ללא שינוי במצב מסד הנתונים.

הפעלת אלגוריתם "הסל החכם"

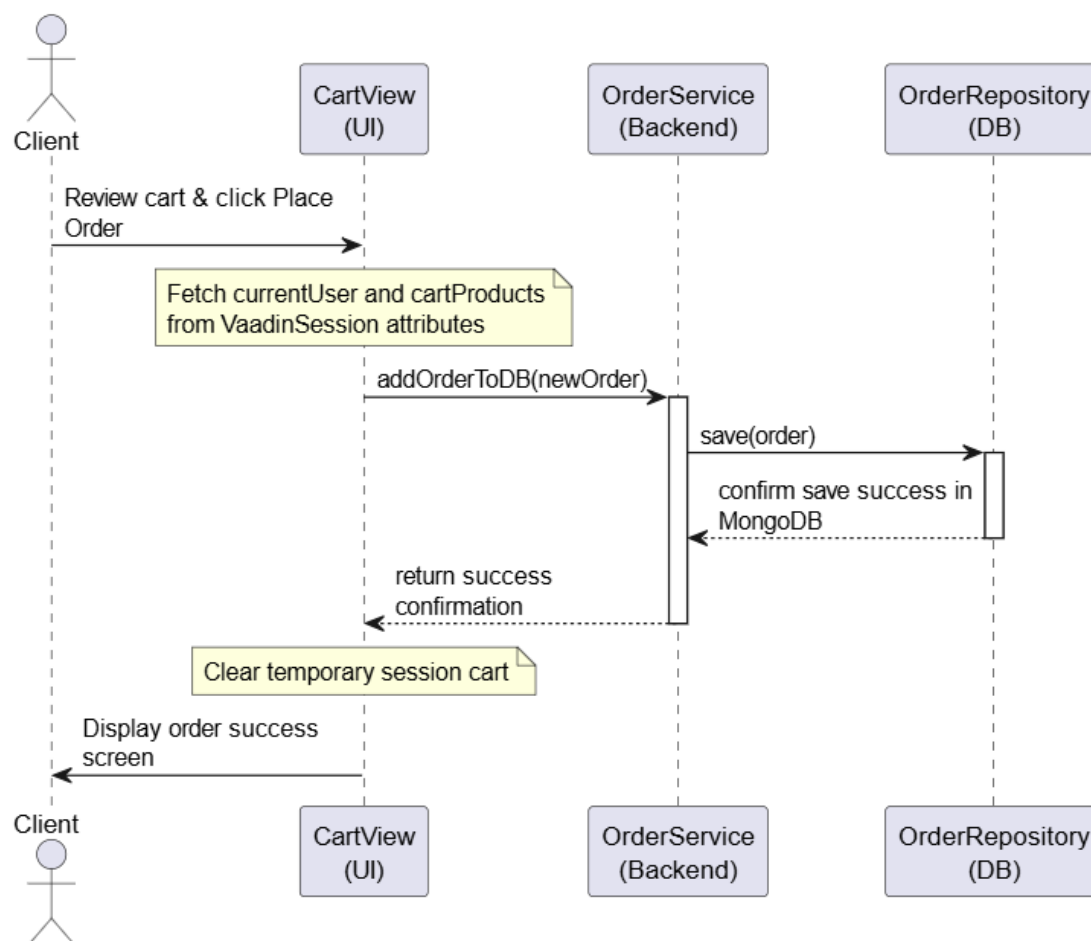
תרחיש הליבה הלוגי, המשלב איסוף פרמטרים, חישוב התאמה סטטיסטית דינמית והפעלת מנוע אופטימיזציה מתמטי.



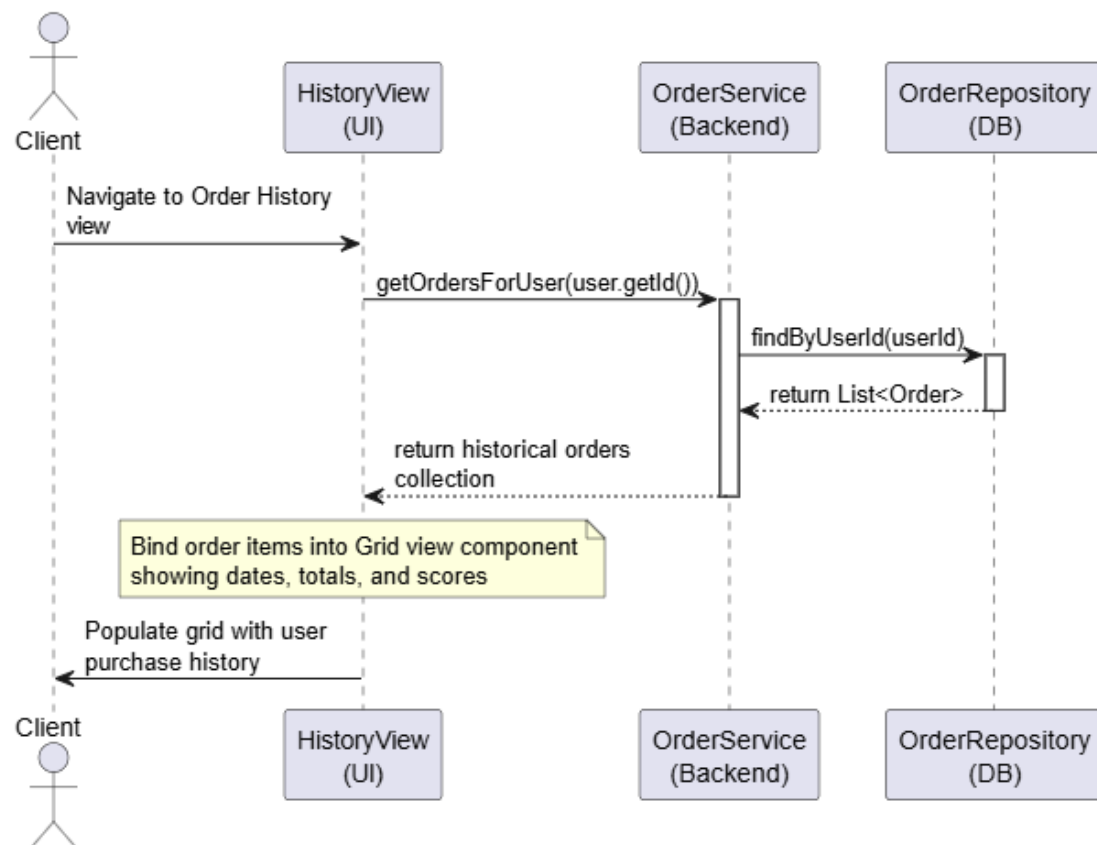
תהליך הליבה החכם מתחיל כאשר הלקוח מזין במסך ה SmartCartView אילוצי קלט מדויקים הכוללים תקציב כספי מקסימלי, טווח כמויות של מינימום ומקסימום פריטים רצויים, ותשובות מתוך שאלון הסגנון ומאשר את החישוב. נתוני קלט אלו מועברים לשירות האלגוריתם המרכזי (SmartCartAlgoService), אשר שולף את קטלוג הבגדים המלא ומעביר את הפריטים יחד עם העדפות הלקוח לשירות הניקוד (ScoringService) המבצע הצלבה ומחזיר ציון התאמה מספרי לכל בגד. בשלב זה, שירות האלגוריתם מריץ על בסיס הנתונים הללו מנוע אופטימיזציה מורכב (מטריצת תכנות דינמי) הבוחר את שילוב הפריטים שממקסם את ציון ההתאמה הכללי תחת מגבלות המחיר והכמות, ומחזיר לממשק המשתמש נתוני פלט של רשימת בגדים אופטימלית המוצגת ללקוח כחבילת סטייל מותאמת אישית.

ניהול עגלת קניות וביצוע רכישה

מתאר את שלב ה Checkout בו המערכת מבצעת "צילום מצב" של הפריטים ומחיריהם ויוצרת רשומת הזמנה קבועה.



תהליך הרכישה וסגירת העסקה מתחיל כאשר הלקוח המחובר מאשר את תכולת עגלת הקניות שלו במסך העגלה (CartView), פעולה המנחה את שכבת הממשק לאסוף את נתוני הקלט של המשתמש והפריטים הנוכחיים מתוך הסשן ולפנות אל שכבת הלוגיקה העסקית באמצעות הפונקציה addOrderToDB שבשירות ההזמנות (OrderService), שירות זה קולט את ישות ההזמנה המורכבת, מיישם את תבנית העיצוב להטמעת נתונים (Embedding) כדי לקבע "צילום מצב" היסטורי של מחירי הבגדים ופרטיהם, ומעביר את המידע המעובד הלאה אל שכבת הנתונים באמצעות הפקודה save במאגר ההזמנות (OrderRepository). המאגר מפעיל שאילתת כתיבה מול מסד הנתונים MongoDB שמבצע את השמירה הפיזית של העסקה ומחזיר אישור נתוני פלט על השלמת הטרנזקציה. עם קבלת האישור, שירות ה-Backend מחזיר אות הצלחה לממשק המשתמש, שמנחה את הדפדפן לנקות ולאפס את עגלת הקניות הזמנית של הלקוח ולהציג על המסך פלט ויזואלי סופית של הודעת סיכום המאשרת שהקנייה הושלמה בהצלחה וההזמנה נשמרה.

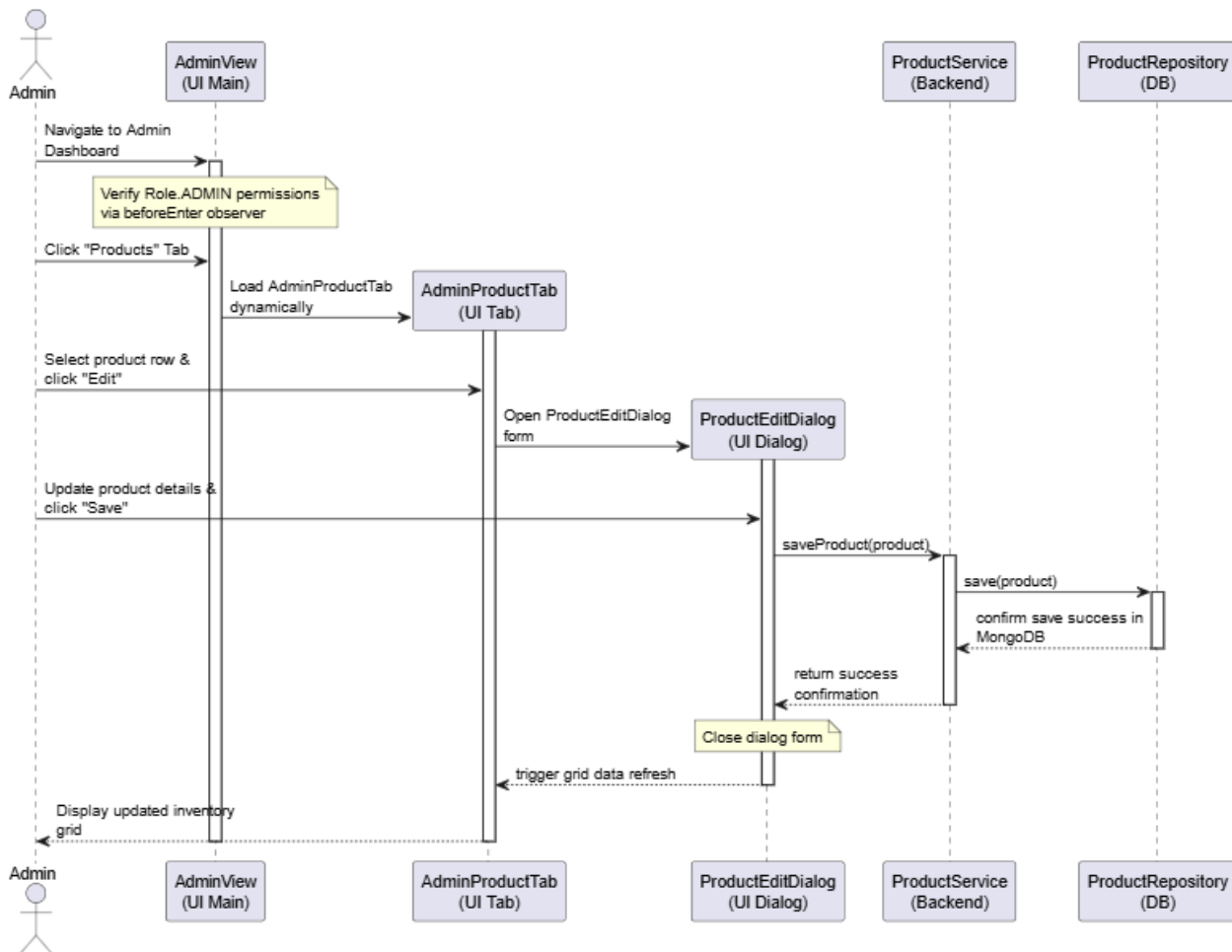
צפייה בהיסטוריית הזמנות אישית

מתאר שלילת הזמנות העבר המשיכות לחשבון המשתמש המחובר כעת.

גישת המשתמש לעסקאות העבר שלו מתחילה כאשר הוא מנווט אל ה HistoryView באזור האישי שלו, והממשק קולט אוטומטית מהסשן את נתוני הקלט של מזהה המשתמש הייחודי ומעביר אותם בבקשה לשירות ההזמנות (OrderService) השירות מפעיל שאילתת סינון ממוקדת מול ה OrderRepository השולפת אך ורק מסמכי הזמנה המכילים את מפתח המשתמש הספציפי, ומסד הנתונים מחזיר כנתוני פלט אוסף מובנה של כל עסקאות העבר שנמצאו. רשימה היסטורית זו מועברת בחזרה אל ממשק המשתמש, שמבצע קישור נתונים ומציג את פלט המידע בתוך טבלת מעקב מסודרת הכוללת תאריכים, פירוט בגדים, סכומים ששולמו בפועל וציוני התאמה.

ניהול קטלוג מוצרים ומלאי

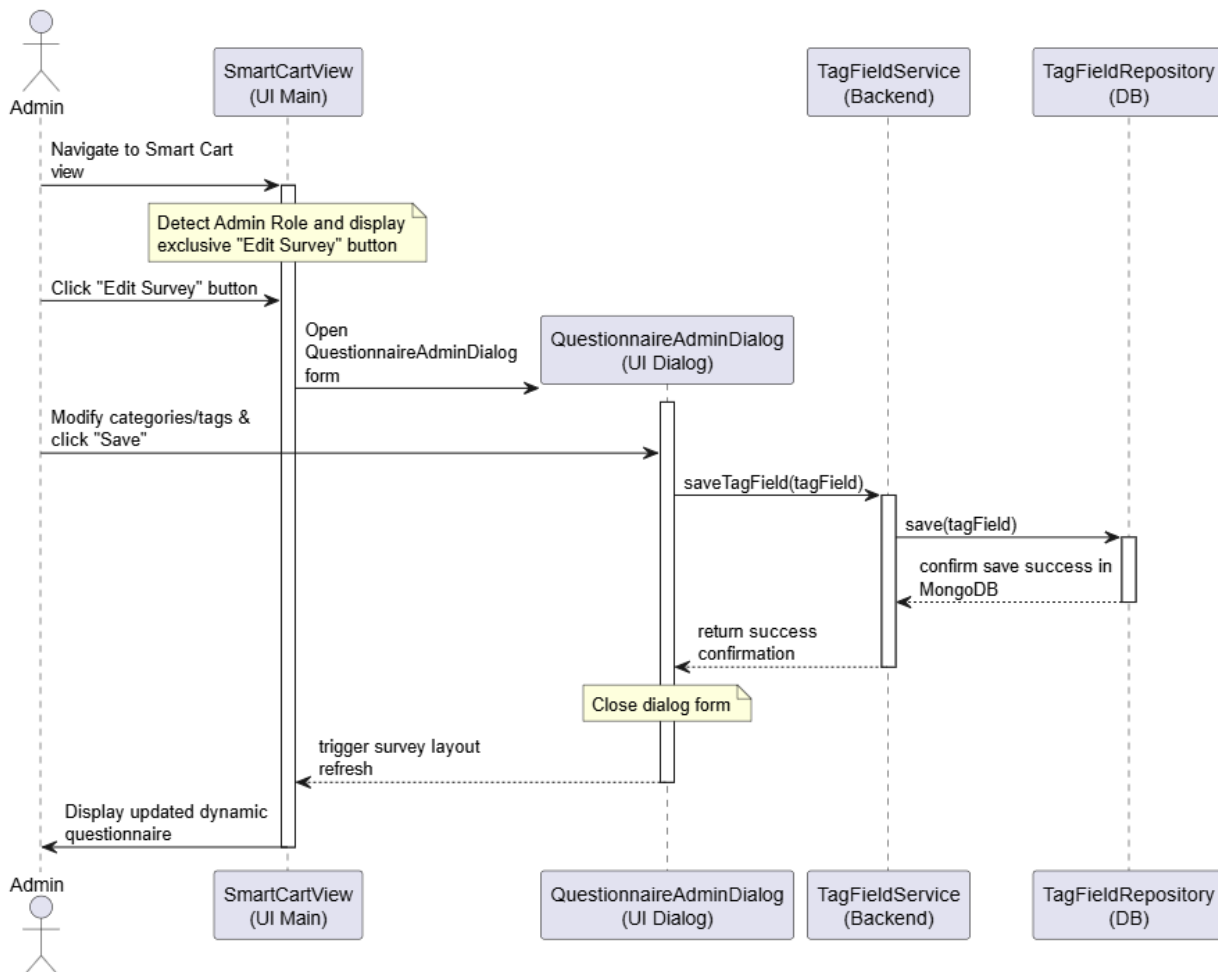
מתאר הוספה, עדכון או מחיקה של מוצרים על ידי האדמין דרך חלונית העריכה.



תהליך ניהול המלאי ועדכון הקטלוג מתחיל כאשר מנהל המערכת מנווט אל מסך השליטה הראשי (AdminView) אשר קולט את זהות המשתמש מהסמן ומבצע אימות הרשאות אבטחה בטרם טעינת הדף לאחר מכן, האדמין בוחר בכרטיסיית ניהול המוצרים, והממשק הראשי טוען באופן דינמי את תצוגת המשנה (AdminProductTab) המציגה את טבלת המלאי הנוכחית. כאשר המנהל בוחר לערוך פריט, נפתחת על גבי המסך חלונית עריכה ייעודית (ProductEditDialog) שלתוכה הוא מזין נתוני קלט חדשים (כגון מחיר, קטגוריה וקישור לתמונה) ולוחץ על כפתור השמירה. פעולה זו אורזת את ישות הפריט מהטופס ומעבירה אותה כקלט אל שירות המוצרים (ProductService) בצד השרת אשר מבצע וולידציה ומנחה את שכבת הנתונים (ProductRepository) לכתוב את המסמך המעודכן באוסף המלאי בתוך מסד הנתונים MongoDB. לאחר שמסד הנתונים מאשר את השמירה הפיזית ומוציא נתוני פלט של הצלחה, השרת מחזיר משוב חיובי לחלונית העריכה, הנסגרת אוטומטית ויוזמת רענון של טבלת הנתונים, כך שמסך האדמין מציג למשתמש באופן מיידי את תמונת המלאי המעודכנת.

ניהול דינמי של תגיות השאלון

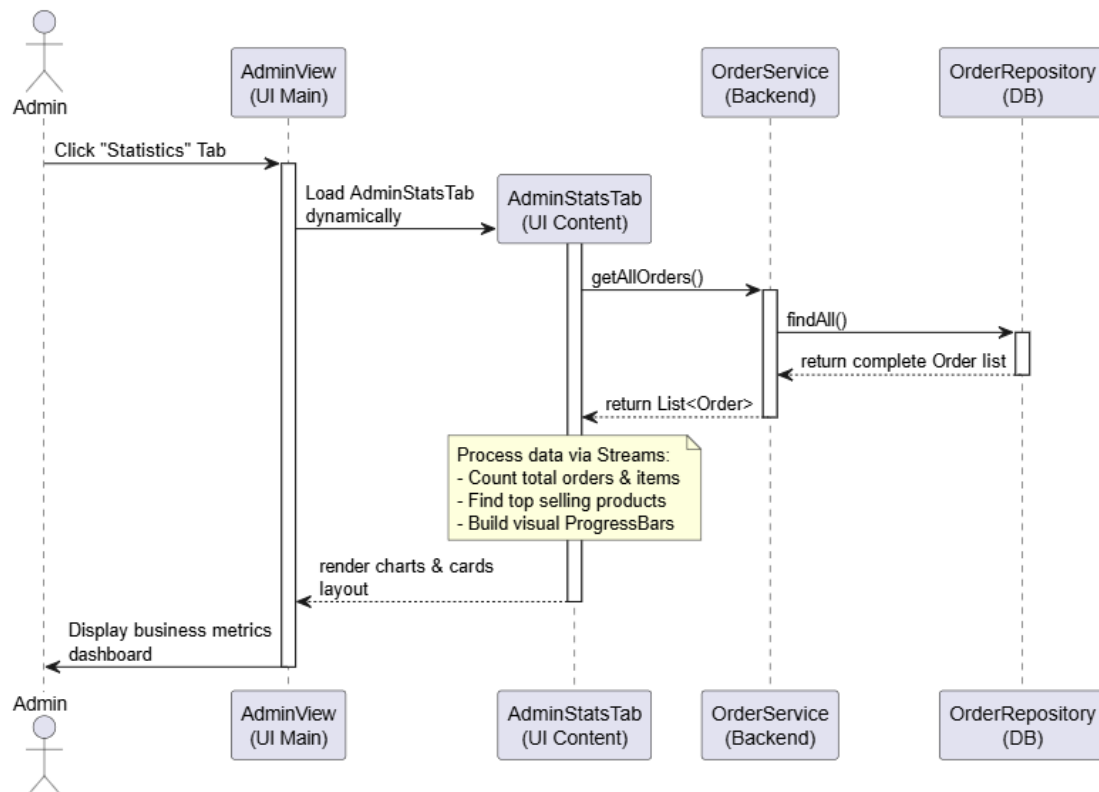
מתאר שינוי או עדכון של מבנה השאלון הדינמי על ידי האדמין.



תהליך ניהול ותחזוקת שאלון הסגנון מתחיל כאשר מנהל המערכת מנווט אל מסך הסל החכם (SmartCartView) אשר בודק את זהות המשתמש מהשכן, מזהה את הרשאותיו המורחבות ומציג עבורו כפתור עריכה ייעודי לשליטה בשדות הממשק. לחיצה על כפתור זה פותחת על גבי המסך חלונית ניהול צפה (QuestionnaireAdminDialog), שלתוכה האדמין מזין נתוני קלט של תצורת שאלון חדשה כגון הוספה או הסרה של שאלות, מאפיינים ותגיות סטיל ומאשר את השינויים. חלונית הניהול אורזת את ישות התוכן המעודכנת ומעבירה אותה כקלט אל שירות התגיות (TagFieldService) ב- Backend המעבד את המבנה ומנחה את שכבת הנתונים (TagFieldRepository) לכתוב ולשמור את מסמכי התצורה החדשים באוסף הייעודי ב MongoDB. השמירה מתבצעת בצורה גמישה המנצלת את היעדר הסכמה הקשיחה של מסד הנתונים, ולאחר שמסד הנתונים מאשר את השמירה הפיזית, השרת מחזיר נתוני פלט של הצלחה לחלונית העריכה, אשר נסגרת אוטומטית ויוזמת רענון מיידי של תצוגת המשתמש במסך הראשי, כך שמבנה השאלון המעודכן מוצג לאדמין מיד וללא צורך ברענון ידני של הדפדפן.

בקרת נתוני משתמשים וסטטיסטיקות

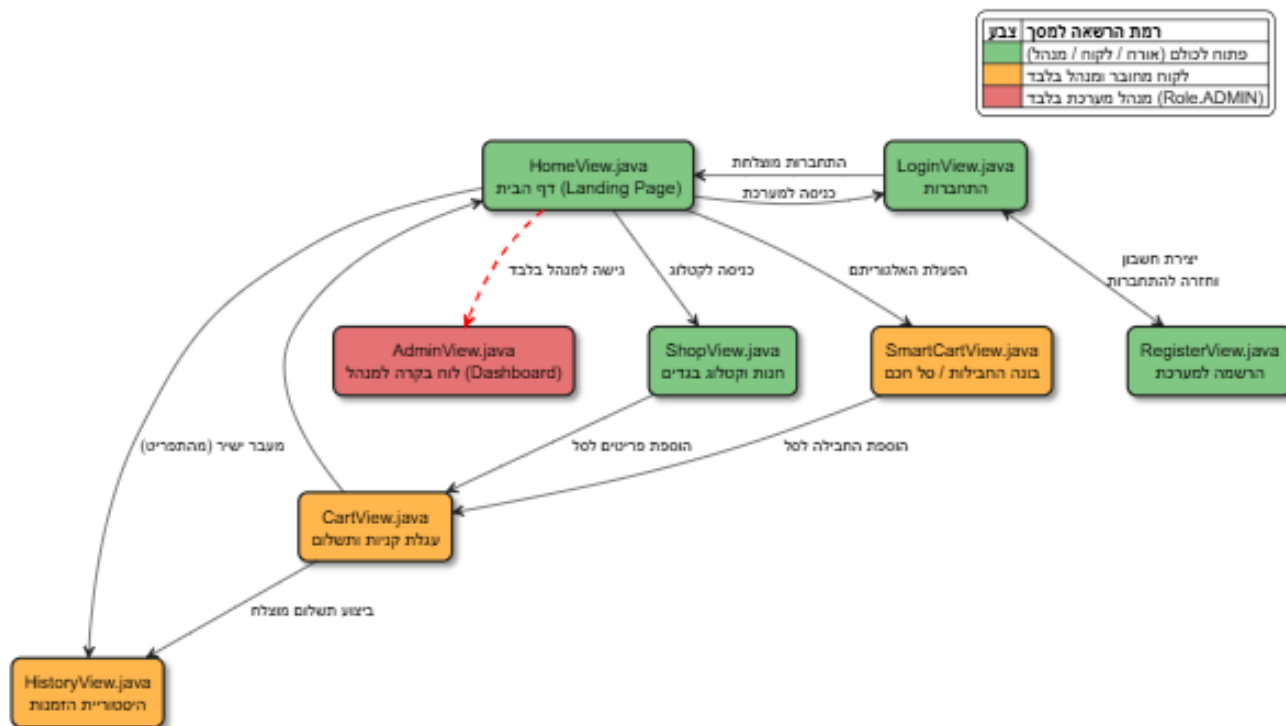
מתאר שליפה ועיבוד של נתוני הזמנות ומשתמשים רוחביים לצורך רינדור גרפים ודוחות בפאנל האדמין.



תהליך הפקת דוחות הסטטיסטיקה והמכירות מתחיל כאשר מנהל המערכת, הנמצא בפאנל השליטה הראשי (AdminView) לוחץ על כרטיסיית הסטטיסטיקה אירוע זה מנחה את הממשק לטעון באופן דינמי את תצוגת המשנה הייעודית לנתונים (AdminStatsTab) אשר יוזמת מיד בקשת שליפה רוחבית משירות ההזמנות (OrderService) ללא קלטים נוספים. השירות פונה אל שכבת הנתונים ומפעיל שאילתת שליפה מלאה מול מאגר ההזמנות (OrderRepository) לקבלת כלל מסמכי עסקאות העבר השמורות במסד הנתונים MongoDB. לאחר שמסד הנתונים מחזיר את אוסף הרשומות הגולמי כנתוני פלט לשירות וממנו ל UI-שכבת הממשק מריצה אלגוריתמים מקומיים המעבדים ומפליחים את הנתונים בזמן אמת לרבות חישוב סך כל ההזמנות, סכימת סך הפריטים הכולל שנמכרו, ואיתור ודירוג המוצרים הפופולריים ביותר. בתום העיבוד, הממשק מרנדר נתוני פלט ויזואליים המורכבים מכרטיסיות נתונים מסכמות, המעניקים לאדמין תמונת מצב עסקית ברורה ועדכנית ללא צורך ברענון הדף.

11.1 תרשים היררכיית המסכים

תרשים זרימת המסכים הבא מציג את דפי הליבה העיקריים של האפליקציה, את חצי הניווט הדו-כיווניים ביניהם, ואת חלוקת ההרשאות המובנית המונעת גישה ממבוקשת למשתמשים שאינם ברמת ההרשאה המתאימה.



ארכיטקטורת הממשק של המערכת בנויה משכבת מעטפת גלובלית ומשלוש רמות הרשאה המגדירות את זרימת המשתמש. המעטפת הראשי (MainLayout.java) מתפקדת כסרגל כלים עליון וקבוע, המאפשר למשתמש לנווט בחופשיות בין כלל המסכים שהוא מורשה לראות, ללא תלות במיקומו הנוכחי. גישה זו מונעת סרבול ויזואלי בתרשים על ידי יתור הצורך בחיצים מרובים מכל דף לכל דף, ומאפשרת לחיצי המעבר בתרשים לשקף את זרימת העבודה הלוגית והמומלצת של המערכת, ולא אילוץ טכני נוקשה.

רמת ההרשאה הראשונה פתוחה לכלל הגולשים וכוללת את דף הבית, (HomeView.java), אשר משמש כרכזת ניווט דינמית. עבור אורחים שאינם מחוברים, דף הבית מנגיש את מסך ההרשמה (RegistrationView.java) ואת מסך ההתחברות (LoginView.java), לצד אפשרות למעבר חופשי וצפייה פסיבית בקטלוג החנות (ShopView.java). ניתנת גם אפשרות לעבור מדף ההתחברות לדף ההרשמה וכן להפך. תהליך רישום מוצלח מסתיים בהפניה אוטומטית למסך ההתחברות, ולאחר אימות זהות תקין, המערכת מנתבת את

המשתמש בחזרה לדף הבית. בשלב זה, הדף משנה את תצורתו ומציג למשתמש המזוהה לוח בקרה אישי המכיל כרטיסיות גישה לפונקציות הליבה של האתר.

רמת ההרשאה השנייה היא רמה מוגנת הפתוחה ללקוחות מחוברים ולמנהלי מערכת בלבד, כאשר כל ניסיון גישה למסכים אלו מצד משתמש אורח נחסם בשלב הניתוב ומופנה חזרה אל דף הבית. רמה זו כוללת את מסך הפעלת "הסל החכם" (SmartCartView.java), בו הלקוח מזין את אילוצי התקציב ועונה על שאלון הסגנון. בניגוד למעבר מערכתי כפוי, לאחר שהמערכת מציגה את התוצאות האופטימליות, ניתן ללקוח החופש לאשר ולהוסיף את הפריטים לעגלה באופן ידני, והמעבר למסך עגלת הקניות (CartView.java) מתבצע לבחירתו דרך סרגל הניווט העליון. בתוך מסך העגלה מנוהל תהליך התשלום והסליקה, וגם כאן, עם השלמת הרכישה בהצלחה, המערכת אינה מאלצת מסלול יחיד, הלקוח מקבל חלונית סיכום המציעה לו אפשרות בחירה חופשית לחזור לדף הבית להמשך גלישה, או לנווט ישירות אל מסך היסטוריית ההזמנות (HistoryView.java) לצורך צפייה בפרטי העסקה שנסגרה.

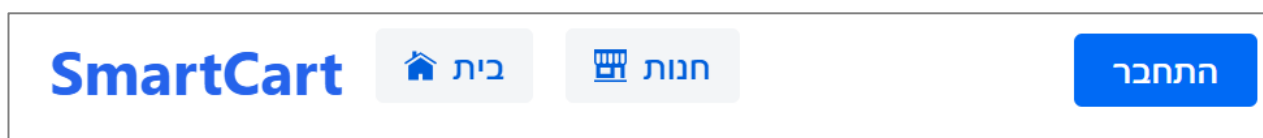
רמת ההרשאה השלישית והגבוהה ביותר שמורה למנהל המערכת בלבד ומורכבת מפאנל הניהול הראשי (AdminView.java). מסך זה חסום לחלוטין בפני אורחים ולקוחות רגילים, וכל ניסיון גישה ללא הרשאת אדמין מפורשת נדחה באופן אקטיבי. הגישה למסך הניהול והיציאה ממנו משולבות בצורה שקופה דרך סרגל הניווט העליון, מה שמעניק למנהל האתר שליטה מלאה, רציפה ומאובטחת על קטלוג המוצרים, נתוני המשתמשים והסטטיסטיקות של המערכת, מבלי לפגוע או להפריע לזרימת חוויית הקנייה השוטפת.

11.2 תיאור המסכים

סרגל ניווט גלובלי (MainLayout)

(הערה: רכיב זה מופיע באופן קבוע בכלל דפי המערכת ולכן מתואר כאן פעם אחת בלבד) לסרגל הניווט יש 3 מצבים שונים בהתאם לתפקיד המשתמש נציג את שלושתם ולאחר מכן נסביר ונפרט על הדברים השונים:

1. מצב אורח:



2. מצב משתמש מחובר:



3. מצב מנהל מערכת:



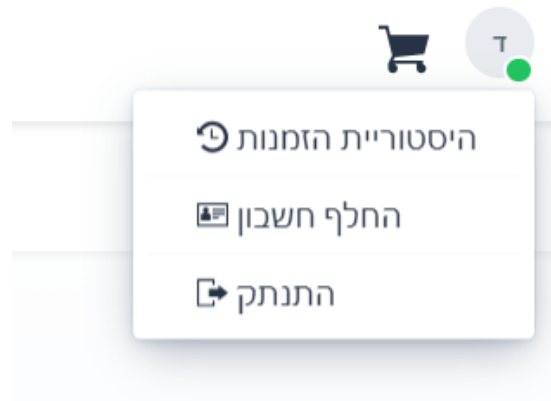
תיאור תפקידו: המחלקת המעטפת הראשית (MainLayout) מייצרת סרגל ניווט עליון המשותף לכל דפי האפליקציה. תפקידו המרכזי הוא לאפשר למשתמש להתמצא ולנווט בצורה ישירה ומהירה בין חלקי המערכת השונים מבלי לחזור אחורה בדפדפן. הסרגל הוא דינמי ומתעדכן בזמן אמת באמצעות מאזין (BeforeEnterObserver) בהתאם למצב ההתחברות ולרמת ההרשאות של המשתמש הוא מציג או מסתיר כפתורי פעולה על בסיס זיהוי הלקוח כאורח, כלקוח מחובר או כמנהל מערכת (Admin).

רכיבי ממשק משתמש (UI Components) ותפקידם:

- **כפתורי ניווט ראשיים (Button):** סדרת כפתורים הממוקמים בצד שמאל (אחד אחרי השני) של הסרגל. כל כפתור מורכב מטקסט ואייקון מתאים, ותפקידו לנתב את המשתמש לדף ספציפי:
 - **כפתור "בית" וכפתור "חנות":** מוצגים תמיד לכולם (כולל אורחים) ומאפשרים חזרה לדף הנחיתה או כניסה לקטלוג המוצרים.
 - **כפתור "סל חכם":** מוצג אך ורק למשתמשים מחוברים, ותפקידו להוביל למסך השאלון ואלגוריתם ההתאמה.
 - **כפתור "ניהול אתר":** מוצג בלעדית למשתמשים שזוהו כבעלי תפקיד ADMIN, ותפקידו להעביר אל פאנל השליטה הראשי של המנהל.
- **כפתור "התחבר":** כפתור בולט בצד שמאל המופיע אך ורק למשתמשים במצב "אורח". תפקידו להפנות את הגולש למסך ההרשמה והאימות (LoginView).
- **אייקון עגלת קניות אינטראקטיבי (Icon + Span):** מוצג רק למשתמשים מחוברים ולמנהל המערכת. מורכב מסמל של עגלה שעליו "מודבקת" תווית עגולה קטנה (Badge) בצבע אדום המציגה מספר. תפקיד התווית לספק חיווי בזמן אמת על כמות הפריטים שנוספו לסל הקניות (התווית מוסתרת לחלוטין אם הסל ריק). לחיצה על כל אזור האייקון מפנה את המשתמש למסך סיכום העגלה והתשלום (CartView).
- **תפריט פרופיל נפתח (MenuBar + Avatar):** אזור אישי המוצג רק למשתמשים מחוברים. הרכיב מורכב מתמונת דמות (Avatar) המכילה את השם המלא של הלקוח, ולצידה נקודת חיווי ירוקה קטנה (Div) המעידה על כך שהמשתמש מחובר (Online).
- **פריטי תפריט הפרופיל (MenuItem):** לחיצה על האוואטר פותחת רשימה נפתחת (Dropdown) המכילה שלוש פעולות:
 1. **"היסטוריית הזמנות":** הפניה למסך המעקב אחר רכישות עבר.
 2. **"החלף חשבון":** איפוס הסשן הנוכחי והעברה מהירה למסך ההתחברות.

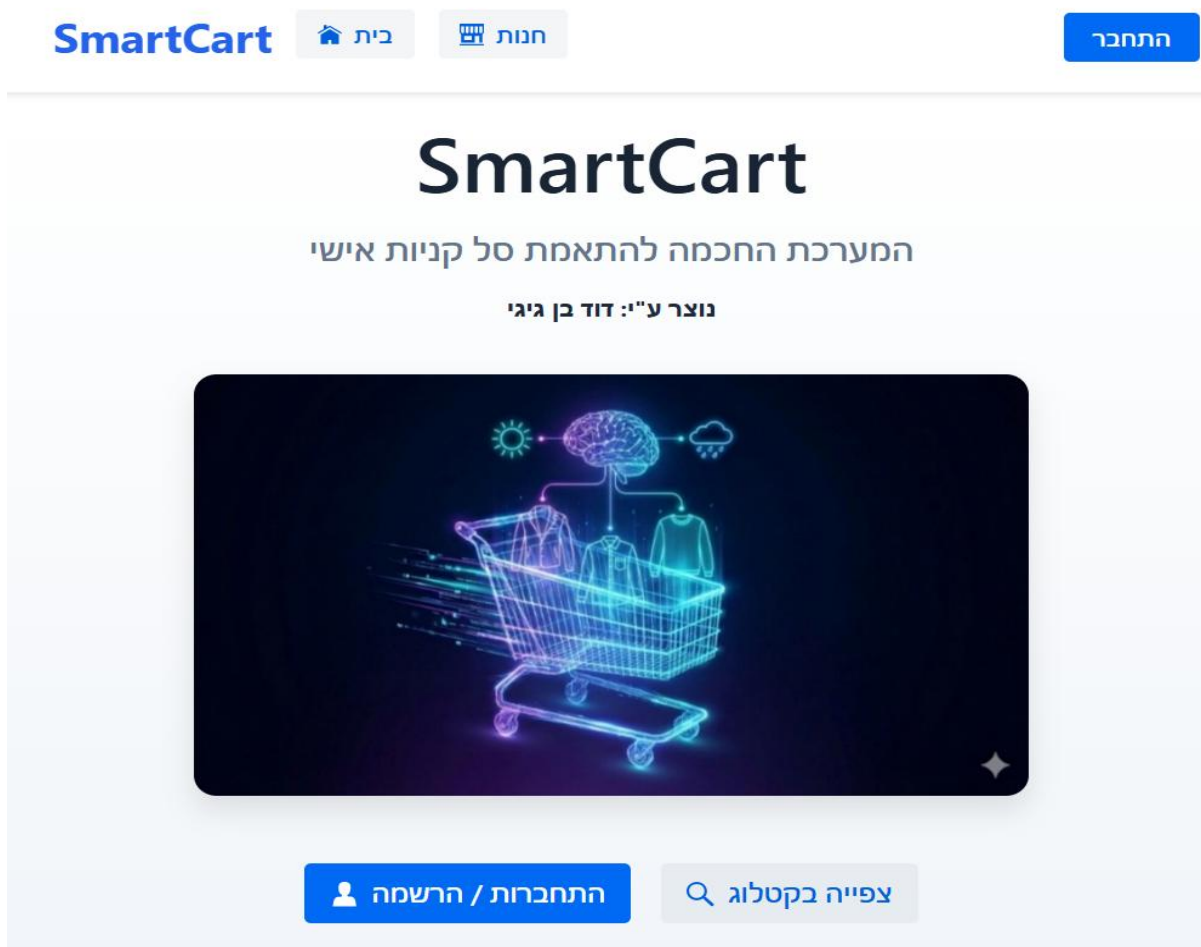
3. "התנתק": מחיקת פרטי המשתמש מהסשן והחזרת המערכת לתצוגת האורח של דף הבית.

להלן תמונה של תפריט הפרופיל:



הודעות ודיאלוגים קופצים (Dialogs & Notifications): רכיב זה משמש כמעטפת ניווט קבועה ואינו מכיל חלונות הודעה או דיאלוגים (Dialogs) קופצים משל עצמו. התפריט הצף היחיד הוא תפריט הפרופיל, אשר תואר תחת רכיבי הממשק.

דף הבית – תצוגת משתמש אורח

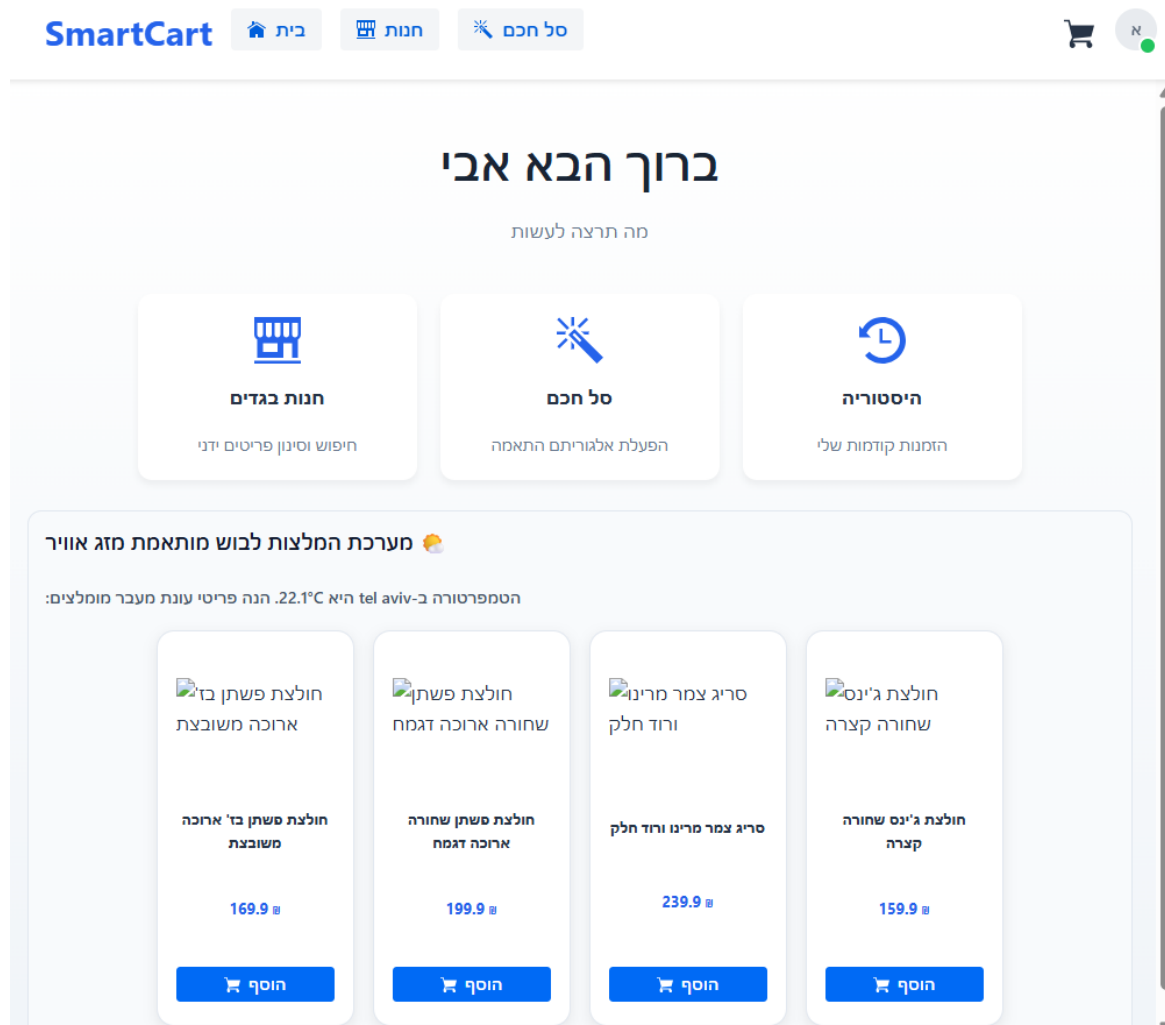


תיאור תפקידו: זהו דף הנחיתה הראשוני המוצג לגולשים שאינם מזוהים במערכת. תפקידו לספק היכרות ראשונית עם מטרת האפליקציה (התאמה חכמה של בגדים), לשדר אמינות, ולהציע למשתמש שתי פעולות עיקריות: להירשם/להתחבר למערכת כדי ליהנות מהתאמה אישית, או להמשיך לגלוש בקטלוג החנות בצורה חופשית ואנונימית (צפייה בלבד).

רכיבי ממשק משתמש ותפקידם:

- **כותרת ראשית ("SmartCart" - H1):** מציגה באופן בולט את שם המערכת כדי לבסס היכרות עם המותג מיד עם הכניסה לאתר.
- **כותרת משנית ("H3 - המערכת החכמה להתאמת סל קניות אישי"):** מגדירה בכמה מילים את הצעת הערך של האפליקציה ומה היא מסוגלת לעשות עבור הלקוח.
- **טקסט קרדיט (Span - "נוצר ע"י: דוד בן גיגי"):** טקסט אינפורמטיבי המציין את שם מפתח המערכת.
- **תמונת נחיתה / באנר (Image):** רכיב גרפי מרכזי (במקרה זה, איור של עגלת קניות דינמית ומוח) הממחיש ויזואלית את הקשר בין שופינג, בינה מלאכותית והתאמה אישית של בגדים. תפקידו לעורר עניין ולשפר את נראות הדף.
- **כפתור קריאה לפעולה "התחברות / הרשמה" (Button):** כפתור מודגש בצבע כחול עם אייקון משתמש, המיועד למשוך את עיקר תשומת הלב של הגולש. לחיצה עליו מנתבת את המשתמש לדף LoginView וממנו ניתן לעבור גם להרשמה.
- **כפתור - "צפייה בקטלוג" (Button):** כפתור בהיר יותר הכולל אייקון של זכוכית מגדלת. מיועד למשתמשים שרוצים לראות את הפריטים לפני פתיחת חשבון. מפנה ישירות ל - ShopView.

הודעות ודיאלוגים קופצים (Dialogs & Notifications): מסך זה הוא סטטי מבחינת המשתמש ואינו מקפיץ הודעות או חלונות (Dialogs) במצב אורח.

דף הבית – תצוגת משתמש מחובר

תיאור תפקידו: כאשר המערכת מזהה לקוח רשום מתוך הסשן, דף הבית משנה את תצורתו והופך ללוח בקרה (Dashboard) אישי. תפקידו לרכז עבור הלקוח את כל פונקציות הליבה של המערכת תחת גריד כרטיסיות אינטראקטיבי וברור, וכן לספק לו ערך מוסף מיידי בדמות המלצות לבוש דינמיות המבוססות על מזג האוויר בעיר מגוריו.

רכיבי ממשק משתמש ותפקידם:

- **ברכת שלום אישית (H1 - "ברוך הבא [שם המשתמש] ")**: כותרת דינמית השולפת את השם המלא של הלקוח מתוך מסד הנתונים כדי לייצר חוויה אישית.
- **רשת כרטיסי ניווט (FlexLayout + VerticalLayout)**: אזור מרכזי הכולל כרטיסיות מלבניות (Menu Cards) מעוצבות הכוללות אפקטים של ריחוף (hover). כל כרטיס מתפקד ככפתור כניסה גדול לאזור מסוים באתר. הרכיבים בתוך כל כרטיס:

- **אייקון (Icon):** ייצוג ויזואלי של יעד הניווט (חנות, שרביט קסם, שעון היסטוריה).
- **כותרת הכרטיס (Span):** טקסט מודגש כמו "חנות בגדים", "סל חכם" או "היסטוריה".
- **תיאור משני (Span):** טקסט קטן ואפור המסביר מה עושים באותו אזור (למשל: "הפעלת אלגוריתם התאמה").
- **רכיב המלצות מזג אוויר (WeatherRecommendationSection):** רכיב מורכב הממוקם מתחת לכרטיסי הניווט. הוא מכיל:
 - **כותרת (Span):** מציינת שההמלצות מותאמות אקלימית.
 - **טקסט חיווי טמפרטורה (Span):** מציג נתון בזמן אמת השאוב משירות מזג האוויר בהתאם לעיר המוגדרת בפרופיל המשתמש.
 - **קרוסלת/גרید כרטיסיות מוצר (HorizontalLayout):** מציג תמונות מוקטנות, שמות מוצרים ומחירים של פריטים ספציפיים שהמערכת סיננה (חורף או קיץ) מתוך קטלוג החנות, כולל כפתורי "הוסף" להכנסתם המהירה לסל.
- הודעות ודיאלוגים קופצים (Dialogs & Notifications):** במסך זה לא מוקפצות הודעות דיאלוג כברירת מחדל (הערה: במידה ולמשתמש חסר נתון עיר מגורים בפרופיל, רכיב מזג האוויר הפנימי יפעיל טופס הזנה, אך אין חלוניות רשמיות העולות מתוך ה HomeView עצמו).

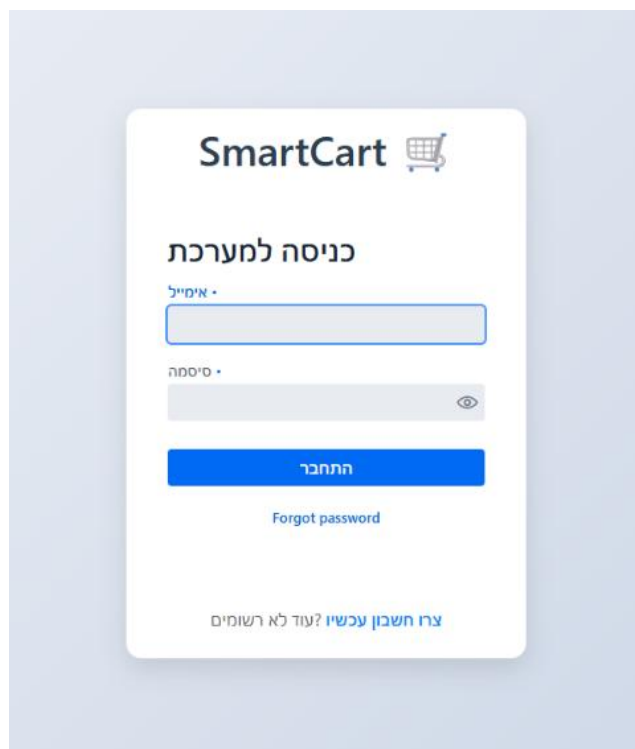
☀️ מערכת המלצות לבוש מותאמת מזג אוויר

לא הגדרת עיר מגורים בחשבונך. אנא עדכן עיר על מנת לקבל המלצות לבוש

עיר (באנגלית)

e.g. Jerusalem, Eilat

עדכן עיר והצג המלצות

מסך התחברות – אימות משתמשים קיימים (LoginView)**תיאור תפקידו:**

מסך ההתחברות מהווה את שער הכניסה המאובטח של המערכת עבור לקוחות שכבר יצרו חשבון בעבר (וכן עבור מנהלי המערכת). תפקידו לקלוט את פרטי ההזדהות של הגולש (אימייל וסיסמה), לשלוח אותם לאימות מול מסד הנתונים באמצעות שירות ה- UserService, ובמקרה של הצלחה – לייצר עותק של המשתמש בזיכרון המערכת (Session) ולהעניק לו גישה לאזורים המוגנים של האתר בהתאם להרשאותיו.

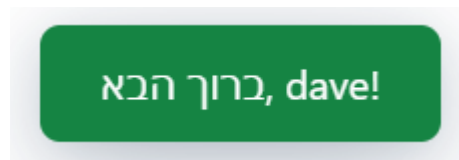
רכיבי ממשק משתמש ותפקידם:

- **מעטפת ועיצוב רקע (VerticalLayout):** המסך מורכב מרקע מדורג (Gradient) עדין, שעליו מונח כרטיס לבן (Card) צף עם פינות מעוגלות המרכז את כלל הפעילות במרכז המסך, תוך שמירה על עיצוב מודרני וקריא.
- **לוגו המערכת (H1):** כותרת בולטת "SmartCart" 🛒 בראש הכרטיס, המחזקת את זהות המותג.
- **רכיב טופס התחברות מובנה (LoginForm):** רכיב חכם של סביבת Vaadin הכולל תמיכה בתרגום לעברית (l18n). הרכיב מאגד בתוכו את השדות הבאים:
 - **שם משתמש / אימייל (TextField):** שדה טקסט להזנת כתובת הדואר האלקטרוני המשמשת כמזהה ייחודי.

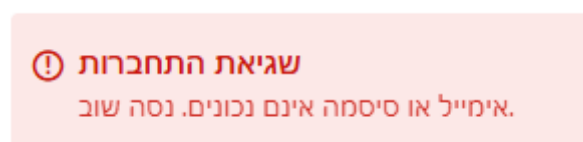
- **סיסמה (PasswordField):** שדה מאובטח להזנת הסיסמה. כולל אייקון של "עין" (Toggle Visibility) המאפשר למשתמש להציג או להסתיר את תווים הסיסמה.
- **כפתור "התחבר" (Submit Button):** כפתור הפעולה של הטופס. לחיצה עליו מפעילה את פונקציית האימות (LoginListener).
- **קישור לשחזור סיסמה ("Forgot password"):** כפתור טקסטואלי מובנה המאפשר גישה עתידית למנגנון איפוס סיסמאות.
- **אזור הפניה להרשמה (Button + Div):** שורת טקסט ("עוד לא רשומים?") המלווה בכפתור שקוף מסוג Tertiary ("צרו חשבון עכשיו"). תפקידו לספק נתיב מילוט לגולשים שהגיעו למסך בטעות ועדיין אין להם חשבון פעיל, ולנתב אותם למסך ה-RegistrationView.

הודעות ודיאלוגים קופצים (Notifications & Dialogs):

- **התראת התחברות מוצלחת (Success Notification):** במידה והאימות במסך הנתונים תקין, קופצת בראש המסך הודעה צפה ירוקה (LUMO_SUCCESS) למשך 3 שניות עם הטקסט: "ברוך הבא, [שם המשתמש]!". לאחר מכן המערכת מנווטת אוטומטית לדף הבית.



- **שגיאת אימות (Error Message):** במידה והאימייל או הסיסמה שגויים, רכיב הטופס עובר אקטיבית למצב שגיאה, ומציג הודעת אזהרה אדומה פנימית מעל השדות: "שגיאת התחברות - אימייל או סיסמה אינם נכונים. נסה שוב." המסך נשאר באותו מצב כדי לאפשר ניסיון חוזר.



מסך יצירת חשבון חדש (RegistrationView)

The image shows a UI mockup for a registration form titled "יצירת חשבון חדש" (Create New Account). The form is contained within a white rounded rectangle on a light blue background. It includes the following elements:

- שם מלא** (Full Name): A text input field.
- כתובת אימייל** (Email Address): A text input field.
- עיר מגורים** (City of Residence): A text input field.
- סיסמה** (Password): A text input field with an eye icon for toggling visibility.
- אימות סיסמה** (Confirm Password): A text input field with an eye icon for toggling visibility.
- הירשם עכשיו** (Register Now): A prominent blue button.
- כבר יש לכם חשבון? התחברו** (Already have an account? Log in): A link below the button.

תיאור תפקידו:

מסך ההרשמה מיועד לגולשים חדשים המעוניינים להצטרף למערכת. תפקיד המסך הוא לאסוף את נתוני החובה הנדרשים לפתיחת פרופיל (כולל עיר המגורים שתשמש בהמשך את מנוע המלצות מזג האוויר), לבצע בדיקות תקינות (וולידציה) על נתוני הקלט כגון וידוא ששדות הסיסמה תואמים ולשמור את ישות המשתמש החדשה (User) במסד הנתונים (MongoDB) תחת הרשאת ברירת המחדל של משתמש רשום (Role.REGISTERED_USER).

רכיבי ממשק משתמש ותפקידם:

- **כרטיס הרשמה מרכזי (VerticalLayout):** בדומה למסך ההתחברות, התוכן עטוף בכרטיס לבן על גבי רקע מדורג, אך כאן הכרטיס מוגדר לרוחב גדול יותר (עד 600px) כדי להכיל שדות מרובים.
- **כותרת הדף (H2):** "יצירת חשבון חדש" - מגדירה את מטרת הדף.
- **רשת שדות רספונסיבית (FormLayout):** רכיב פריסה חכם המסדר את השדות במסכים רחבים הוא מציג שני שדות בשורה, ובמסכים קטנים (סלולר) הוא קורס לשדה אחד בשורה למניעת שבירת תצוגה.
 - **שם מלא (TextField):** קליטת שם המשתמש לתצוגה במערכת (יופיע בבר העליון).

- **כתובת אימייל (EmailField):** שדה מיוחד הכולל וולידציה פנימית של מבנה כתובת דואר אלקטרוני. משמש כמזהה (ID) הראשי.
- **עיר מגורים (TextField):** נתון קריטי לפונקציונליות המערכת השואבת נתוני אקלים.
- **סיסמה (PasswordField):** להזנת קוד הגישה.
- **אימות סיסמה (PasswordField):** שדה זהה נוסף שמטרתו למנוע טעויות הקלדה.
- **כפתור ביצוע "הירשם עכשיו" (Button):** כפתור מודגש ברוחב מלא המפעיל את לוגיקת שמירת הנתונים בסרוויס ה-Backend.
- **כפתור חזרה "כבר יש לכם חשבון? התחברו" (Button):** כפתור טקסט משני המאפשר חזרה מהירה למסך ההתחברות במידה והגולש לחץ על כפתור ההרשמה בטעות.

הודעות ודיאלוגים קופצים (Notifications & Dialogs):

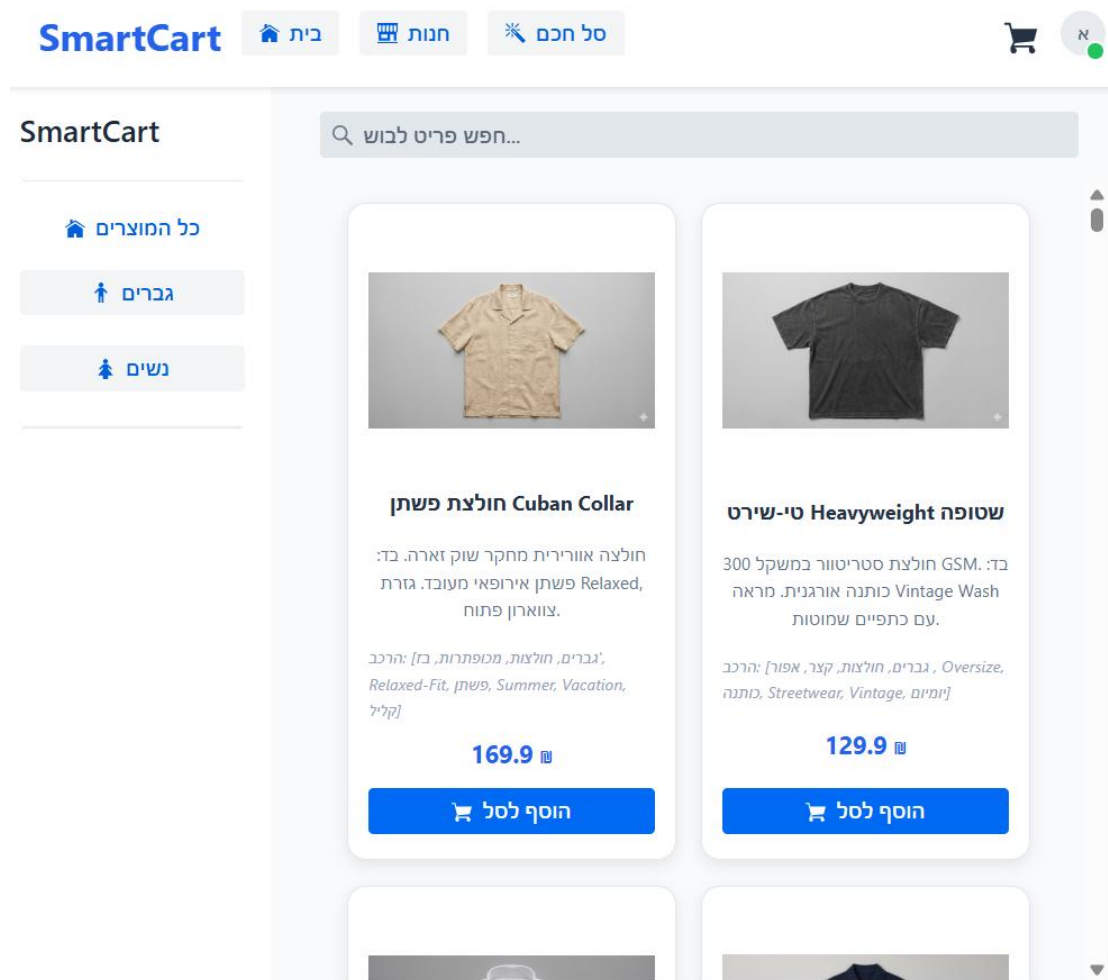
- **שגיאת אימות סיסמאות (Validation Error):** אם קיים חוסר התאמה בין שדה ה"סיסמה" ל"אימות סיסמה", תוקפץ למרכז המסך התראה אדומה (LUMO_ERROR) עם הכיתוב: "הסיסמאות אינן תואמות", ותהליך השמירה ייעצר.

הסיסמאות אינן תואמות

- **שגיאת משתמש כפול (DB Error):** אם שירות הנתונים מזהה שהאימייל כבר שמור במסד, תוקפץ התראה אדומה במרכז המסך: "האימייל כבר קיים במערכת".

האימייל כבר קיים במערכת

- **אישור הרשמה (Success Notification):** לאחר כתיבה מוצלחת למסד הנתונים, תוקפץ בראש המסך הודעה ירוקה (LUMO_SUCCESS): "החשבון נוצר בהצלחה!", והמערכת תעביר את הגולש באופן מיידי למסך ה-LoginView כדי שיוכל להתחבר עם פרטיו החדשים.

מסך קטלוג חנות הבגדים (ShopView)**תיאור תפקידו:**

מסך החנות משמש כקטלוג הווירטואלי המרכזי של המערכת. תפקידו להציג את כלל מלאי פריטי הלבוש הזמינים, ולאפשר למשתמשים לחפש ולסנן מוצרים בצורה יעילה לפי טקסט חופשי, מגדר או קטגוריות ספציפיות.

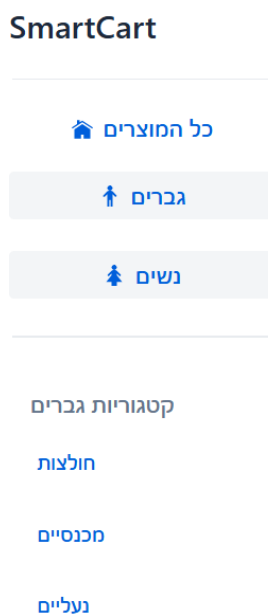
הערת הרשאות חשובה: המסך ברמת הצפייה (Read-Only) נגיש ופתוח לכלל הגולשים באתר (כולל אורחים). עם זאת, ביצוע אינטראקציה מסחרית, ניסיון להוסיף פריט לעגלת הקניות מוגבלת למשתמשים מחוברים בלבד. אורח שינסה להוסיף מוצר ינותב אוטומטית על ידי המערכת למסך ההתחברות.

רכיבי ממשק משתמש ותפקידם:

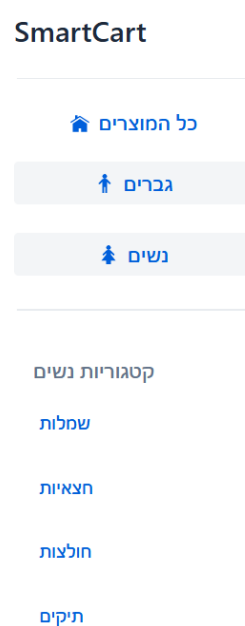
- **סרגל סינון צידי (Sidebar Layout):** מיכל אנכי קבוע הממוקם בצד המסך ומרכז את כל אפשרויות הסינון הראשיות. הרכיב כולל:
 - **כותרת (H2):** שם המותג "SmartCart" בראש הסרגל.
 - **כפתור "כל המוצרים" (Button):** מאפס את כל משתני הסינון (מגדר וקטגוריה) וטוען מחדש את כלל הקטלוג למצב ההתחלתי.

- **כפתורי סינון מגדר ("גברים" / "נשים"):** כפתורים ראשיים. לחיצה עליהם מסננת את הגריד המרכזי ובו-זמנית מפעילה לוגיקה דינמית המציגה תפריט משני של תת-קטגוריות רלוונטיות (כמו "חולצות", "ג'ינס", "שמלות").
- **כפתורי תת-קטגוריה דינמיים (Button):** כפתורים קטנים המתווספים לסרגל הצידי רק לאחר בחירת מגדר, ומאפשרים מיקוד נוסף של תוצאות החיפוש.

*כאשר נבחר לסנן בגדי גברים



*כאשר נבחר לסנן בגדי נשים



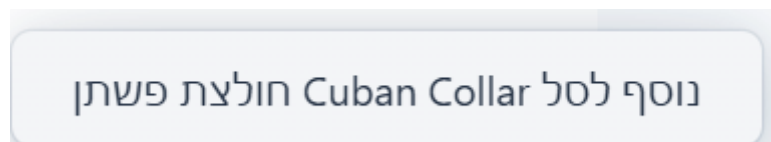
- **אזור התוכן המרכזי (Main Content Area):** המיכל המרכזי הפרוס על רוב שטח המסך ומגיב לפילטרים.

- **שורת חיפוש טקסטואלית (TextField):** שדה קלט המלווה באייקון זכוכית מגדלת. השדה מוגדר במצב מיוחד (ValueChangeMode.LAZY), כך שהוא מאזין להקלדת המשתמש ומעדכן את רשימת המוצרים בזמן אמת רגע לאחר סיום ההקלדה, ללא צורך בלחיצה על כפתור "חפש".
- **גריד מוצרים רספונסיבי וגלגלת אנכית (ScrollArea + FlexLayout):** מיכל תצוגה גמיש המסדר את כרטיסי המוצר בשורות ודוחף אותם לשורה הבאה אם אין מספיק מקום (Wrap). המיכל עטוף באזור גלילה המאפשר לגלול את המוצרים למטה מבלי להעלים את סרגל הסינון הצידי או את שורת החיפוש.

- **כרטיס מוצר סטנדרטי (ProductCard):** רכיב מותאם אישית המציג את ישות ה-Product. (במסך זה מוצג בתצורתו המלאה והמורחבת, בניגוד לתצורה הקומפקטית בדף הבית).
 - **תמונת המוצר (Image):** מוצגת בחלקו העליון של הכרטיס עם התאמת גודל אוטומטית.
 - **שם, תיאור והרכב בד (Spans):** רכיבי טקסט המציגים את פרטי הפריט (למשל: "שטופה טרי-שירט Heavyweight", מידע על סוג הכותנה וכו').
 - **מחיר (Span):** טקסט מודגש כחול המציג את המחיר לאחר חישוב (למשל 129.9 ₪).
 - **כפתור "הוסף לסל" (Button):** כפתור פעולה המפעיל בדיקת הרשאות. למשתמש מחובר: הכפתור מוסיף את אובייקט המוצר לרשימת העגלה בסשן וקורא לפונקציית עדכון של הבר העליון (העיגול האדום מתעדכן). למשתמש אורח: הכפתור מפעיל ניתוב לדף LoginView.

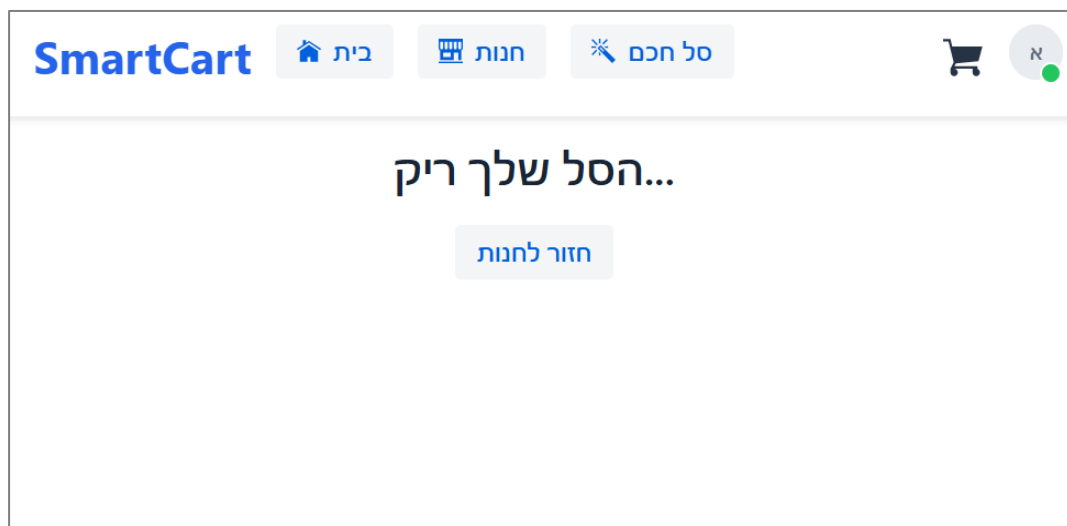
הודעות ודיאלוגים קופצים (Notifications & Dialogs):

1. **התראת הוספה לסל (Success Notification):** כאשר **משתמש מחובר** לוחץ על כפתור "הוסף לסל", המערכת מקפיצה בחלק התחתון של המסך (ברירת המחדל של הודעות מסוג זה בואדין) הודעת טקסט קצרה המאשרת את הפעולה (לדוגמה: "חולצת פולו סרוגה בטקסטורת ריב נוסף לסל").

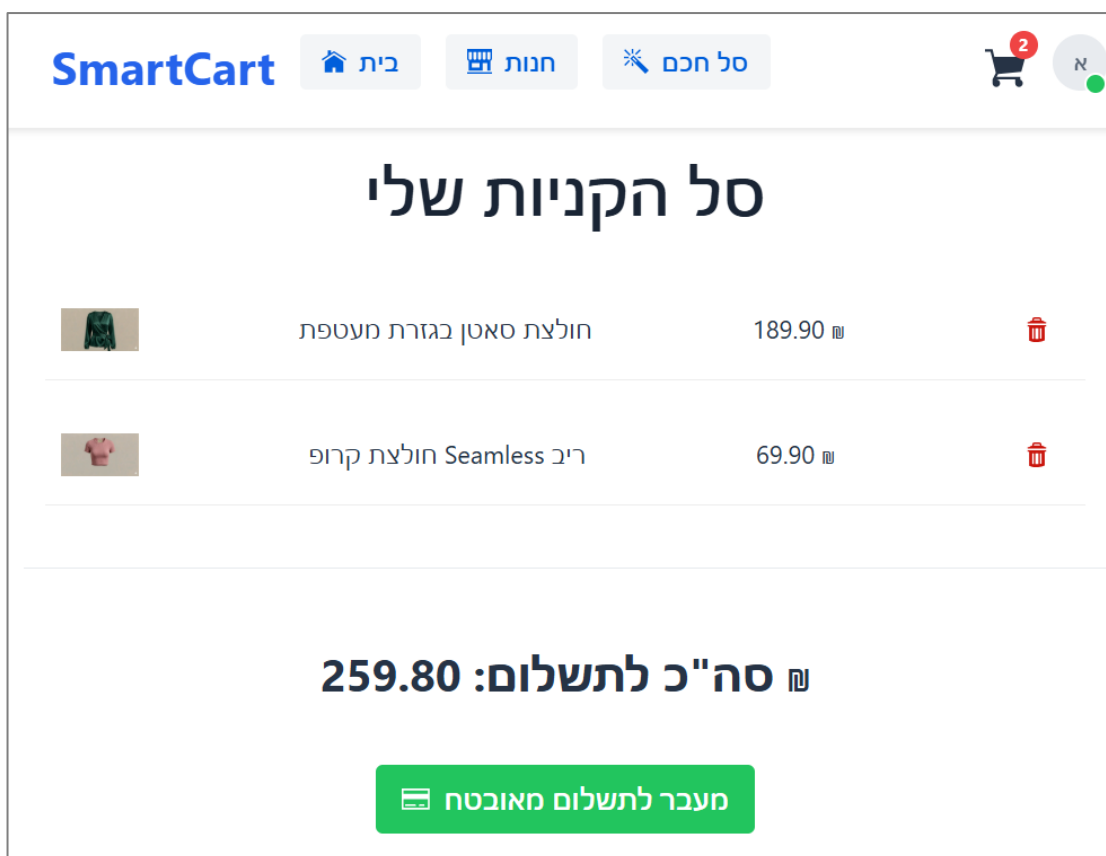


מסך עגלת הקניות וביצוע תשלום (CartView)

המסך כאשר העגלה ריקה:



המסך כאשר עגלה אם לפחות מוצר 1:



תיאור תפקידו:

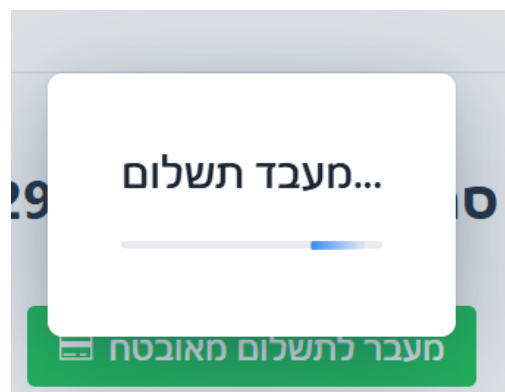
מסך זה מהווה את תחנת הסיום (Checkout) של תהליך הרכישה באתר. תפקידו למשוך את רשימת המוצרים שנאספו באופן זמני בתוך זיכרון הסשן (VaadinSession), להציג ללקוח סיכום ויזואלי וכספי של תכולת העגלה, לאפשר לו להסיר פריטים במידת הצורך, ולנהל את תהליך התשלום הסימולטני שבסופו ההזמנה נשמרת באופן קבוע במסד הנתונים תחת שם המשתמש שלו.

(הערת אבטחה: הגישה למסך זה מאובטחת. משתמש לא מחובר שינסה לגשת לכתובת ה-URL ינותב אוטומטית לדף הבית על ידי מאזין ה-beforeEnter).

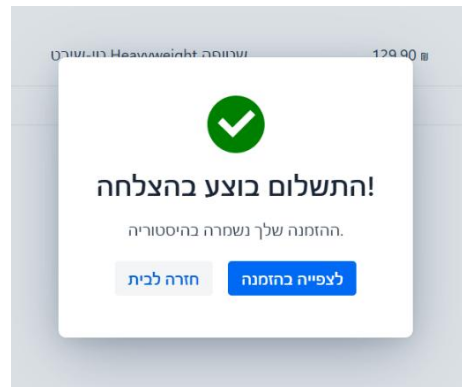
רכיבי ממשק משתמש ותפקידם:

- **כותרת המסך (H1):** "סל הקניות שלי".
- **תצוגת עגלה ריקה (H2 + Button):** במידה והסשן אינו מכיל פריטים, המערכת מסתירה את רוב רכיבי הדף ומציגה הודעה אינפורמטיבית ("הסל שלך ריק...") לצד כפתור המנתב את המשתמש חזרה לדף הקטלוג (ShopView).
- **רשימת הפריטים בסל (HorizontalLayout + VerticalLayout):** מיכל רספונסיבי הבנוי משורות. כל שורה מייצגת מוצר אחד ומכילה:

- **תמונת ממוזער (Image):** תצוגה מקדימה קטנה של הבגד להתמצאות מהירה.
 - **שם המוצר ומחירו (Spans):** פרטי הזיהוי והעלות של הפריט.
 - **כפתור הסרה/פח אשפה (Button):** כפתור מודגש באדום עם אייקון פח. לחיצה עליו מוחקת את המוצר מאובייקט ה-Cart שבסשן ומרעננת את הדף אוטומטית כדי לעדכן את הסכום הכולל.
 - **סיכום כספי (Span):** רכיב טקסטואלי מודגש בכתב גדול הממוקם מתחת לרשימת המוצרים ומציג את הסכום הסופי המשוקלל לתשלום בפורמט מטבע שקלי.
 - **כפתור ביצוע תשלום (Button):** כפתור ירוק גדול ומרכזי עם אייקון של כרטיס אשראי ("מעבר לתשלום מאובטח"). לחיצה עליו יוזמת את הלוגיקה העסקית שממירה את פריטי הסל הזמניים לישות הזמנה קבועה (Order מול OrderItem) ופותחת את דיאלוג הסליקה.
- הודעות ודיאלוגים קופצים (Notifications & Dialogs):**
- **חלונת המתנה ותהליך התשלום (ProgressBar & Dialog):** לאחר לחיצה על "תשלום", עולה על המסך חלונת (Dialog) קופצת שאינה ניתנת לסגירה בלחיצה בחוץ. בשלב הראשון החלונת מציגה מד התקדמות אסינכרוני אינסופי (ProgressBar - Indeterminate) עם הכיתוב "מעבד תשלום...". המדמה השהיית תקשורת לספק סליקה למשך 2 שניות (באמצעות Thread.sleep).



- **חלונית אישור עסקה:** עם סיום תהליך השמירה במסד הנתונים, תוכן הדיאלוג משתנה דינמית. הוא מציג אייקון וירוק גדול (VaadinIcon.CHECK_CIRCLE), הודעת "התשלום בוצע בהצלחה!", ושני כפתורי ניווט לסיום התהליך: "חזרה לבית" ו-"לצפייה בהזמנה". הלחיצה על אחד הכפתורים מאפסת לחלוטין את עגלת הקניות בסשן, סוגרת את הדיאלוג ומנתבת את המשתמש ליעדו.



מסך היסטוריית הזמנות (HistoryView)

בית
חנות
סל חכם

היסטוריית הזמנות

תאריך	מחיר	מוצרים שנרכשו
26/05/2026 20:35	259.80 ₪	

תיאור תפקידו:

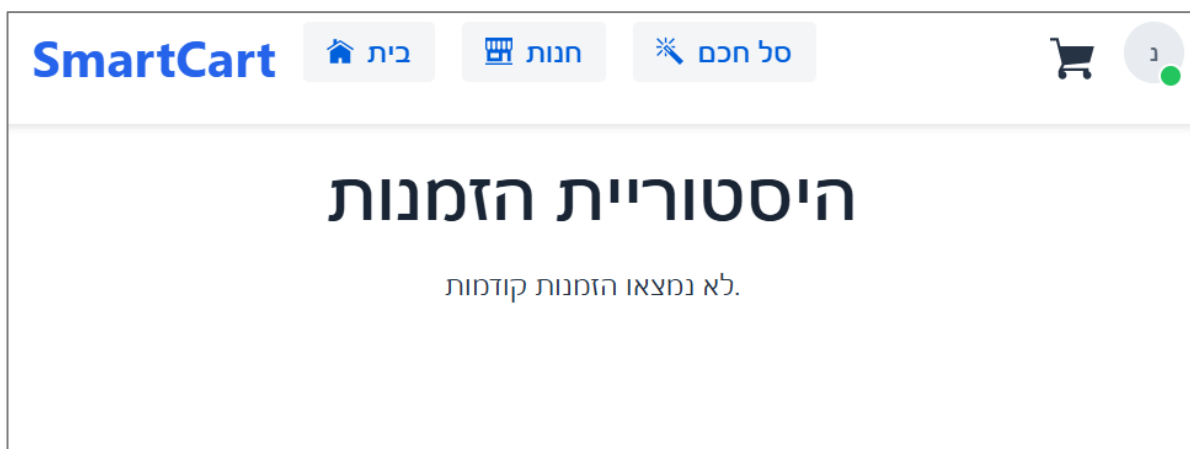
מסך היסטוריית ההזמנות מהווה אזור אישי המאפשר ללקוח מעקב אחר הרכישות הקודמות שלו. תפקיד המסך הוא לפנות ל-OrderService, לשלוף ממסד הנתונים את כל מסמכי ההזמנה המשויכים ל-ID של הלקוח המחובר הנוכחי, ולהציג אותם בצורה טבלאית, מאורגנת וויזואלית, המאפשרת זיהוי מהיר של התאריכים, סכומי העסקה והפריטים שנרכשו בה. המסך מאובטח ברמת הניתוב (BeforeEnterObserver) וחוסם גישה אחרים.

רכיבי ממשק משתמש ותפקידם:

- **כותרת הדף (H1):** מציגה בבירור את שם המסך ("היסטוריית הזמנות").
- **רכיב טבלה מתקדם (<Order>Grid):** ליבת הממשק של דף זה. טבלה דינמית המציגה את האובייקטים שנשלפו מהמסד, כאשר כל שורה בטבלה מייצגת הזמנה שלמה אחת (Order). הטבלה מוגדרת להציג את כל השורות בבת אחת (setVisibleAllRows) ללא גלילה פנימית, וכוללת את העמודות הבאות:
 - **עמודת תאריך:** מציגה את חותמת הזמן (תאריך ושעה) של מועד סגירת העסקה בפורמט קריא.
 - **עמודת מחיר:** מציגה את סך כל התשלום הכולל לאותה הזמנה בפורמט מטבע.
 - **עמודת "מוצרים שנרכשו" (ComponentRenderer):** עמודה ויזואלית מיוחדת. במקום להציג שמות בטקסט רגיל, עמודה זו מייצרת מיכל אופקי (HorizontalLayout) שבתוכו תמונות ממוזערות (Images) של כל הבגדים שנכללו בהזמנה. רכיב זה משדרג משמעותית את חוויית המשתמש ומאפשר זיהוי מידי של הקנייה ללא צורך בקריאת טקסטים ארוכים. התמונות ערוכות עם רדיוס מעוגל קל למראה נקי.
- **רכיב חיווי כלי / טולטיפ (Tooltip):** על כל תמונה ממוזערת של פריט בעמודת המוצרים מופעלת תכונת setTitle. תפקיד רכיב פסיבי זה הוא להציג למשתמש את שמו המלא של הפריט רק כאשר הוא מרחף מעל התמונה הקטנה עם סמן העכבר.

הודעות ודיאלוגים קופצים (Notifications & Dialogs):

במידה וללקוח המחובר טרם ביצע רכישות באתר (השליפה מהמסד מחזירה רשימה ריקה), הטבלה אינה מוצגת ובמקומה מוצגת הודעת טקסט אינפורמטיבית סטטית בגוף הדף (Span - "לא נמצאו הזמנות קודמות"). המסך אינו מקפיץ חלונות (Dialogs) צפות.



מסך הפעלת אלגוריתם "הסל החכם" (SmartCartView)

המסך לפני הרצת אלגוריתם ההתאמה:

SmartCart

בית

חנות

סל חכם

נ

בונה החבילות החכם - שאלון סטייל

עבור מי?

מה מחפשים?

אירוע

סגנון

סוג בד

צבע

עונה

דוגמה

תקציב (₪)

למשל 500

מינימום פריטים

1

מקסימום פריטים

3

חשב סל אופטימלי

המסך אחרי הרצה ומציאת סל מתאים העומד בכול האילוצים:

חשב סל אופטימלי

החבילה המוצעת עבורך: 3 פריטים | עלות החבילה: 259.70 ₪ | התאמה ממוצעת: 44.4%



אתלטיק Racerback גופיית

חופש תנועה מלא בכתפיים. בד: מיקרופייבר קל משקל. בד מנדף מהיר יבוש.

Athletic-Fit, גברים, חולצות, גופיות, ניאון: הרכב [ספורט, Gym, Running, מיקרופייבר, Fit]

79.9 ₪

הוסף לסל



שחורה Muscle Fit טי-שירט

בד אלסטי במיוחד המדגיש את מבנה הגוף. בד: 95% כותנה, 5% לייקרה. שרוולים קצרים וצמודים.

Muscle-Fit, גברים, חולצות, קצר, שחור: הרכב [זמיום, Gym, Basic, כותנה-לייקרה, Fit]

89.9 ₪

הוסף לסל



Performance גופיית אימון

לביצועים מקסימליים. בד: 100% גורת גב. פוליאסטר מנדף זיעה שחיין לתנועה חופשית.

Active, גברים, חולצות, גופיות, שחור: הרכב [חלק, Gym, Sport, פוליאסטר]

89.9 ₪

הוסף לסל

עמוד 54 מתוך 111

תיאור תפקידו:

מסך "הסל החכם" מהווה את מוח המערכת ואת נקודת המפגש בין חוויית המשתמש לאלגוריתמיקה. תפקידו לאפשר ללקוח המחובר להגדיר בצורה מדויקת את העדפות האופנה שלו באמצעות שאלון דינמי, להציב אילוצי תקציב וכמות (מינימום/מקסימום פריטים), ולהפעיל את אלגוריתם האופטימיזציה. בסיום החישוב, המסך מציג ללקוח את החבילה האופטימלית ביותר שנמצאה עבורו, תוך מתן אפשרות להוסיף את הפריטים לעגלת הקניות. המסך מאובטח לחלוטין וחסום בפני משתמשים אורחים.

רכיבי ממשק משתמש ותפקידם:

- **כותרת הדף (H2):** "בונה החבילות החכם - שאלון סטייל" מגדירה בבירור את מטרת המסך.
- **כפתור עריכת שאלון לאדמין (Button):** רכיב הרשאות דינמי. כפתור זה ("ניהול שדות השאלון") מופיע בראש המסך אך ורק אם המשתמש המחובר הוא מנהל מערכת (Role.ADMIN). תפקידו לפתוח חלונית ייעודית לעריכת תצורת השאלון (הוספה/הסרה של שדות ותגיות).

- **מיכל השאלון הדינמי (VerticalLayout):** אזור טופס חכם הנבנה בזמן אמת מתוך מסד הנתונים (דרך ה-TagFieldService).
- **שדות בחירה דינמיים (<String>Select):** תפריטים נפתחים (Combo-boxes) המציגים את קטגוריות החיפוש (למשל: סגנון, אירוע, צבע, עבור מי). הקוד מסדר אותם בצורה רספונסיבית ואסתטית של עד 4 שדות בשורה (HorizontalLayout).
- **שורת אילוצי החישוב (Constraints Row):** מאגדת את גבולות הגזרה עבור האלגוריתם:
 - **תקציב (NumberField):** שדה מספרי להזנת סכום המקסימום שהלקוח מוכן להוציא (ב-₪).
 - **מינימום / מקסימום פריטים (IntegerField):** שדות מספריים טבעיים להגדרת גבולות הכמות של החבילה, על מנת למנוע מצב של חבילה ריקה או חריגה מדרישות הלקוח. מוגדרים עם ערכי ברירת מחדל של 1 עד 3.
- **כפתור הפעלה "חשב סל אופטימלי" (Button):** כפתור קריאה לפעולה ירוק ובולט עם אייקון של שרביט קסמים. לחיצה עליו אוספת את כלל נתוני הטופס ומשגרת את תהליך החישוב.

• אזור תצוגת תוצאות לאחר חישוב:

- **תווית סיכום (Span):** רכיב טקסט המציג שורת סיכום עם נתוני העסקה (כמות הפריטים שנבחרו, העלות הכוללת של החבילה, וציון ההתאמה הממוצע באחוזים).
- **גריד תוצאות (FlexLayout):** מיכל רספונסיבי המאכלס את כרטיסי המוצר (ProductCard) שהאלגוריתם בחר.
- **כרטיס מוצר קומפקטי (ProductCard ב-Compact Mode):** שימוש חוזר (Reusability) במחלקת כרטיס המוצר של החנות, אך העברת פרמטר `compact = true` המקטין את התמונות והגופנים ומסתיר את תיאור הבד. הדבר נעשה אדריכלית כדי לחסוך שטח מסך ולהציג חבילה שלמה בצורה מרוכזת וידידותית. בעל כפתור הוספה מינימליסטי.

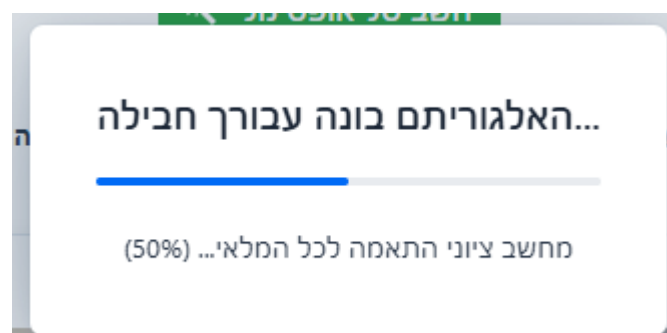
הודעות ודיאלוגים קופצים (Notifications & Dialogs):

1. **שגיאת וולידציה (Notification):** במידה והמשתמש לוחץ על חישוב מבלי להזין תקציב (או מזין ערך שלילי), תוקפץ הודעת אזהרה: "נא להזין תקציב תקין", והחישוב ייעצר.



2. **חלונית התקדמות החישוב (Progress Dialog):** ליבת הפעולה של המסך. דיאלוג מודאלי החסום לסגירה, המכיל מד התקדמות (ProgressBar) פנימי. מכיוון שחישוב האלגוריתם (ציוני התאמה + תכנון דינמי) דורש משאבים, הפעולה מתבצעת בתהליכון רקע נפרד (Thread). החלונית מתעדכנת בזמן אמת מהתהליכון לשכבת ה-UI (ui.access) ומשקפת את שלבי העבודה באחוזים:

- 20%: "מנתח שאלון סטייל..."
- 50%: "מחשב ציוני התאמה לכל המלאי..."
- 80%: "מבצע אופטימיזציה לתקציב..."
- 100%: "החבילה מוכנה!" (ולאחר מכן החלונית נסגרת מעצמה והתוצאות מוצגות).



3. **הודעת חוסר תוצאות (Span):** במידה והאלגוריתם מחזיר רשימה ריקה (לא ניתן לעמוד באילוצי התקציב/הכמות), לא מוקפצת חלונית, אלא תווית הסיכום מתעדכנת לטקסט אינפורמטיבי המנחה את המשתמש: "לא נמצא שילוב אופטימלי תחת האילוצים. נסה להגדיל את התקציב."

חשב סל אופטימלי ✖

לא נמצא שילוב אופטימלי תחת האילוצים. נסה להגדיל את התקציב.

4. **התראת הוספה לסל (Notification):** לחיצה על "הוסף" באחד מכרטיסי התוצאה תקפיץ חיווי קצר במסך (למשל: "חולצה קצרה נוסף לסל") ותעדכן את הבר העליון במקביל.

חולצת אוקספורד יוקרתית נוסף לסל

דיאלוג ניהול שדות ואופציות השאלון (QuestionnaireAdminDialog)

ניהול שדות השאלון (אדמין) ⚙

ניהול שדות ואופציות השאלון (אדמין) ⚙

צור שדה תגית חדש:

למשל: סוג בד, מותג + הוסף שדה

השדות הקיימים והערכים שלהם:

עבור מי? 🗑

גברים ✕ נשים ✕

הוסף אופציה... +

מה מחפשים? 🗑

הכל ✕ חולצות ✕ מכנסיים ✕ נעליים ✕

תיקים ✕ חליפות ✕ מעילים ✕ שמלות ✕

חצאיות ✕

סגור ושומר שינויים

תיאור תפקיד הדיאלוג:

זהו ממשק צף (Modal Dialog) המופיע בלעדית למנהל המערכת (Admin) לאחר לחיצה על כפתור התצורה בראש דף הסל החכם. תפקידו להעניק לאדמין יכולת שליטה מלאה (CRUD) על תצורת השאלון הדינמי – להוסיף, להסיר ולערוך קטגוריות שאלות ותגיות מבלי צורך בעדכון קוד האפליקציה. הדיאלוג עובד בצמוד ל-TagFieldService, מעדכן את הרשומות ב-MongoDB, וכאשר הוא נסגר, הוא מפעיל פונקציית Callback (באמצעות האובייקט Runnable onChangeCallback) המנחה את דף הסל החכם שמתחתיו לרענן את עצמו באופן מיידי ולהציג למשתמש את השאלון המעודכן.

רכיבי ממשק משתמש בתוך הדיאלוג ותפקידם:

- **אזור הוספת שדה ראשי (קטגוריה חדשה):** ממוקם בראש החלונית.
 - **שדה טקסט (TextField):** לקליטת שם הקטגוריה החדשה (לדוגמה: "סוג אירוע").
 - **כפתור הוספה (Button):** שומר את הקטגוריה החדשה במסד הנתונים ומרענן את תצוגת הדיאלוג.
- **גריד שדות ואופציות קיימים (HorizontalLayout / VerticalLayout):** מיכל דינמי המייצר שורת ממשק עבור כל קטגוריה הקיימת במערכת (למשל עבור השדה "מגדר" או "צבע").
 - **כותרת שדה ומחיקה:** כותרת קטנה המציגה את שם השדה, ולצידה כפתור אדום מסוג Tertiary עם אייקון פח (VaadinIcon.TRASH). לחיצה עליו מוחקת את השדה כולו יחד עם כלל אפשרויותיו מהמערכת.
 - **תצוגת אופציות כ"תגיות מחיקות":** כל תגית/אופציה קיימת בתוך השדה מוצגת ככפתור קטן מסוג Error עם סימון "X". לחיצה על תגית מסירה אותה ספציפית מהשדה ללא מחיקת השדה עצמו.
 - **אזור הוספת אופציה:** שדה קלט טקסטואלי מקומי וכפתור הוספה (אייקון "+") הממוקמים בתחתית כל כרטיסיית-שדה. משמשים להוספת אופציה חדשה ספציפית רק לאותו השדה (למשל, הוספת הצבע "תכלת" לשדה "צבע").

- **כפתור סגירה ושמירה (Button):** כפתור ממוקם בתחתית החלונית. תפקידו לסגור את הדיאלוג ובמקביל לעורר את פונקציית הרענון של ממשק האב (השאלון הראשי).

פאנל ניהול ראשי למנהל מערכת (AdminView)

תיאור תפקיד המסך והגדרות אבטחה:

זהו לוח הבקרה (Dashboard) המרכזי של האפליקציה, המיועד למנהלי מערכת בלבד. הגישה למסך זה מאובטחת באופן אקטיבי ברמת הניתוב (באמצעות BeforeEnterObserver); המערכת בודקת את אובייקט המשתמש השמור בסשן, ואם התפקיד (Role) שלו אינו ADMIN או אם מדובר באורח הבקשה נדחית והגולש מוחזר אוטומטית לדף הבית.

ארכיטקטורת המסך מבוססת על רכיב ניווט פנימי של כרטיסיות (Tabs). לחיצה על כל כרטיסייה מרוקנת את אזור התוכן התחתון (מיכל Div מרכזי) וטוענת לתוכו באופן דינמי את רכיב התצוגה (Tab Layout) הרלוונטי, ללא צורך בטעינה מחדש של הדף המלא.

להלן ארבעת כרטיסיות הניהול:

כרטיסייה 1: ניהול מוצרים (AdminProductTab)

SmartCart

בית חנות סל חכם ניהול אתר

ד

לוח בקרה למנהל

סטטיסטיקה הזמנות ורווחים ניהול משתמשים ניהול מוצרים

ניהול קטלוג מוצרים

הוסף מוצר חדש +

שם המוצר	מחיר	תגיות	פעולות
Cuba... חולצת פשתן	169.90 ₪	...צות, מכופתרות, בז	
Heavywei... טי-שירט	129.90 ₪	... חולצות, קצר, אפור	
... אוקספורד יוקרתית	199.90 ₪	...צות, מכופתרות, לבן	
... וגה בטקסטורת ריב	159.90 ₪	..., חולצות, קצר, כחול	
W... חולצת ג'ינס דנים	189.90 ₪	...ות, ג'ינס, כחול דנים	
... צת הנלי ריב ארוכה	119.90 ₪	...ות, ארוך, אפור מלנז	

- **תיאור תפקידו:** ממשק הניהול של קטלוג החנות. מאפשר לאדמין לבצע פעולות CRUD מלאות (יצירה, קריאה, עדכון, מחיקה) על ישויות ה-Product במסד הנתונים.
- **רכיבי ממשק משתמש:**
 - **כפתור "הוסף מוצר חדש" (Button):** כפתור מודגש בראש המסך. לחיצה עליו פותחת את דיאלוג העריכה במצב ריק להזנת נתונים חדשים.
 - **טבלת נתונים דינמית (<Product>Grid):** טבלה המפרטת את כלל המוצרים. כוללת עמודות עבור שם המוצר, מחירו, רשימת התגיות המשויכות אליו (המשמשות את האלגוריתם), ועמודת "פעולות".
 - **כפתורי פעולה בטבלה (Buttons):** בכל שורת מוצר קיימים שני כפתורים: כפתור עריכה (אייקון עיפרון) לפתיחת נתוני המוצר לעדכון, וכפתור מחיקה אדום (אייקון פח) להסרת המוצר לחלוטין מהמערכת.
- **הודעות ודיאלוגים קופצים:**
 - **דיאלוג הוספה/עריכה (ProductEditDialog):** חלונית קופצת המכילה טופס מלא להזנת פרטי המוצר (שם, מחיר, תמונות, תגיות).
 - **התראת מחיקה:** בעת לחיצה על פח האשפה, המוצר נמחק ומוקפצת הודעה ירוקה בראש המסך: "המוצר נמחק בהצלחה", והטבלה מתרעננת מיד.

דיאלוג העריכה של מוצר חדש:

הוספת מוצר חדש

שם מוצר

מחיר (₪)

0

כתובת תמונה (URL)

תיאור הבגד

...כאן כנס על הגזרה, הבד והסטייל

תגיות (הפרד בפסיק)

למשל: גברים, חולצות, חורף

שמור שינויים

ביטול

דיאלוג יצירה של מוצר קיים:

Cuban Collar עריכת מוצר: חולצת פשתן

שם מוצר

Cuban Collar חולצת פשתן

מחיר (₪)

169.9

כתובת תמונה (URL)

images/linen-cuban.jpg

תיאור הבגד

חולצה אוורירית מחקר שוק זארה. בד: 100% פשתן צווארון פתוח, Relaxed אירופאי מעובד. גזרת

תגיות (הפרד בפסיק)

פשתן 100%, Relaxed-Fit, גברים, חולצות, מכופתרות, בו קליל, Summer, Vacation

שמור שינויים

ביטול

כרטיסייה 2: ניהול משתמשים (AdminUserTab)

SmartCart

בית

חנות

סל חכם

ניהול אתר



לוח בקרה למנהל

סטטיסטיקה הזמנות ורווחים ניהול משתמשים ניהול מוצרים

ניהול משתמשים במערכת

שם משפחה + ...	אימייל	הרשאה / תפקיד	פעולות
dave	davidbg1415@gmail.com	ADMIN	
ilan	ilan@gmail.com	REGISTERED_U...	
דויד	david@gmail.com	REGISTERED_U...	
שוהם	shohamsharvit68@gmail.com	REGISTERED_U...	
אבי	avi@gmail.com	REGISTERED_U...	
ישראל	isr@gmail.com	REGISTERED_U...	
נעומי	neomi@gmail.com	REGISTERED_U...	

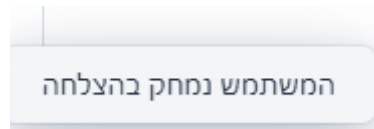
- **תיאור תפקידו:** ממשק שליטה על קהילת הלקוחות והמשתמשים הרשומים באתר, המאפשר צפייה בפרטיהם ומחיקת חשבונות סוררים או לא פעילים.

- **רכיבי ממשק משתמש:**

- **טבלת משתמשים (<User>Grid):** מציגה את השם המלא, כתובת האימייל וההרשאה (Role) של כל משתמש.
- **כפתור מחיקת משתמש (Button):** בכל שורה ישנו אייקון פח אשפה. הערה לוגית חשובה: הקוד מבצע בדיקת הגנה אל מול הסשן הנוכחי. אם מזהה המשתמש בשורה זהה לזה של האדמין המחובר, כפתור המחיקה מנוטרל (Disabled) כדי למנוע מצב בו מנהל המערכת מוחק את עצמו בטעות ונועל את המערכת.

• הודעות ודיאלוגים קופצים:

- לאחר לחיצה על מחיקה וביצוע תקין מול שירות הנתונים, מוקפצת התראה: "המשתמש [שם המשתמש] נמחק בהצלחה". במקרה של כשל במחיקה, מוקפצת הודעת שגיאה המכילה את פרטי הבעיה.



כרטיסייה 3: הזמנות ורווחים (AdminOrdersTab)

חודש / שנה	סך הכל רווח	מספר הזמנה
05/2026	5227.00 ₪	6a045a4ec1dbdd6c42d385c5
		6a045d40c1dbdd6c42d385c7
		6a04ae4bb59733f50ef13ab6
		6a04b371b59733f50ef13ac0
		6a04b50bb59733f50ef13ac1
		6a05c7fd4f0d5973f5bdeabc
		6a05c9f64f0d5973f5bdeabd
		6a09ca84c285075fb70c4081
		6a09ccd2c285075fb70c4082

- **תיאור תפקידו:** לוח בקרה פיננסי המנתח את נתוני הרכישות ההיסטוריים, מקבץ רווחים לפי חודשים ומציג את העסקאות הבודדות שבוצעו במערכת.

• רכיבי ממשק משתמש:

- **כרטיסיית הכנסות כוללות (VerticalLayout):** קופסה ירוקה בולטת בראש הדף המציגה את סך כל ההכנסות ברוטו (₪) שנוצרו אי פעם במערכת, מחושב בזמן אמת.

- **טבלת פילוח חודשי** (`<MonthRevenueRow>Grid`): טבלה הממוקמת בצד ימין. מופעל עליה אלגוריתם (מבוסס `Java Streams`) המקבץ את כלל ההזמנות לפי חודש ושנה, ומציג שורה מסכמת של רווח לכל חודש (למשל: 05/2026 -> 1250.00 ₪).

- **טבלת היסטוריית הזמנות מערכתית** (`<Order>Grid`): טבלה רחבה המפרטת כל הזמנה שבוצעה במערכת (מספר הזמנה, תאריך מדויק, סכום העסקה וכמות הפריטים שנקנו בה).

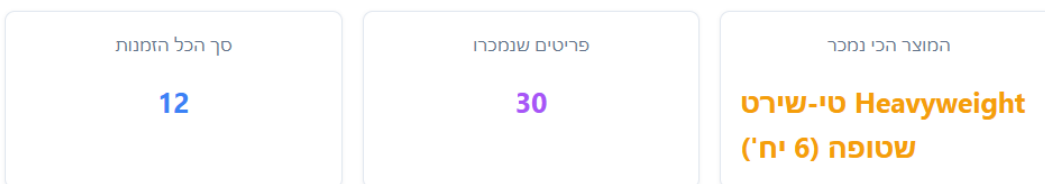
- **הודעות ודיאלוגים קופצים:** אין חלוניות קופצות בכרטיסייה זו.

כרטיסייה 4: סטטיסטיקה (`AdminStatsTab`)



לוח בקרה למנהל

ניהול מוצרים ניהול משתמשים הזמנות ורווחים סטטיסטיקה

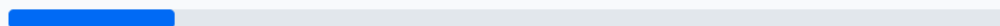


פילוח ויזואלי של פופולריות מוצרים (באחוזים)

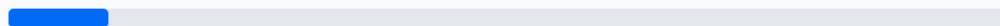
שטופה - 6 יחידות (20.0%) Heavyweight טי-שירט



חולצת אוקספורד יוקרתית - 5 יחידות (16.7%)



יחידות 3 - Heavy חולצת פלאנל משובצת (10.0%)



- **תיאור תפקידו:** תצוגת נתונים אנליטית מבוססת אלגוריתמיקה המנתחת את ביצועי החנות. המסך מציג נתונים יבשים ומייצר פילוח ויזואלי של פופולריות פריטים מתוך סך כל הפריטים שנמכרו.

- **רכיבי ממשק משתמש:**

- **כרטיסיות נתונים אופקיות** (`Stat Cards`): שלוש קופסאות מידע המציגות נתונים מכריעים: סך ההזמנות הכולל, כמות הפריטים הפיזיים שנמכרו, והמוצר הנמכר ביותר במערכת. (מחושב באמצעות `Collectors.groupingBy` לחילוף הפריט הנפוץ ביותר).

- **גרף ברים ויזואלי (ProgressBar + VerticalLayout):** תצוגה גרפית של שלושת המוצרים הפופולריים ביותר. עבור כל מוצר מוצגת תווית (Span) עם שמו והאחוז היחסי שלו מסך המכירות הכולל. מתחת לתווית מוצג מד (ProgressBar) המשמש באופן יצירתי כעמודת גרף למילוי נפח האחוזים ויזואלית.
- **הודעות ודיאלוגים קופצים:** אין חלונות קופצות. במידה ואין מספיק נתוני רכישה בחנות כדי לייצר גרף, מוצגת הודעת טקסט חלופית המודיעה זאת בתוך חלון הגרף.

12. תיאור התוכנה

12.1 סביבת הפיתוח של התוכנה

12.1.1 מערכת הפעלה

- **הבחירה:** Windows 10/11 (בסביבת הפיתוח), תמיכה חוצת-פלטפורמות בסביבת הריצה.
- **נימוק:** מכיוון שהמערכת מבוססת על שפת Java הרצה על גבי מכונה וירטואלית (JVM), היא אינה תלויה במערכת הפעלה ספציפית. פיתוח המערכת התבצע בסביבת Windows מטעמי נוחות וזמינות כלים, אך שרת ה-Tomcat ומסד הנתונים בענן יכולים לרוץ על שרתי Linux באופן חלק ויעיל. בצד הלקוח, המערכת נגישה מכל מערכת הפעלה (Windows, macOS, Linux, Android, iOS) דרך דפדפן רשת סטנדרטי וללא צורך בהתקנה מקומית.

12.1.2 סביבות הפיתוח (IDEs)

- **הבחירה:** Visual Studio Code (גרסה 1.108.0).
- **נימוק:** סביבה זו נבחרה בזכות היותה קלת משקל, מודרנית וניתנת להרחבה. היא משמשת לכתיבה, ניפוי שגיאות (Debugging) וניהול הפרויקט באמצעות מנגנון Maven. בנוסף, סביבה זו מאפשרת שילוב הרחבות (Extensions) קריטיות לייעול העבודה, כגון "MongoDB Extension" המאפשרת גישה מהירה לניהול הנתונים בענן ישירות מתוך הקוד ללא מעבר לכלים חיצוניים, ו-"PlantUML" המאפשרת יצירת תרשימי הנדסת תוכנה (כגון Use Case ו-Sequence) מבוססי קוד תוך קבלת תצוגה מקדימה בזמן אמת.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

- **הבחירה בצד שרת (Backend):** Spring Boot (גרסה 3.5.9).
- **נימוק:** תשתית מובילה בעולם ה-Java המאפשרת הקמה מהירה של רכיבי שרת. המסגרת כוללת בתוכה שרת HTTP פנימי מסוג Apache Tomcat, מנהלת בצורה חכמה את שכבת ה-Services להרצת הלוגיקה העסקית, מספקת ממשקי Repositories לביצוע פעולות מול מסד הנתונים, ומכילה REST Controllers לניהול התעבורה ברשת (בקשות ותגובות HTTP).

- **הבחירה בצד לקוח (Frontend):** Vaadin (גרסה 24.9.5).
- **נימוק:** פלטפורמה ייחודית המאפשרת פיתוח ממשקי משתמש (UI) עשירים ורספונסיביים לדפדפן באמצעות כתיבת קוד Java בלבד, תוך ייתור הצורך בשימוש ישיר בשפות צד-לקוח מסורתיות כמו HTML, CSS או JavaScript. שינויים ויזואליים המבוצעים בצד השרת מסונכרנים לדפדפן בזמן אמת באופן חלק באמצעות טכנולוגיית Web-Sockets ומנגנון PUSH מאובטח.

12.1.4 שפות תכנות

- **הבחירה:** Java (גרסה 21).
- **נימוק:** שפת עילית מונחית עצמים (OOP) אשר נבחרה בזכות יכולתה המוכחת והיציבה לספק מענה מלא לפיתוח מערכת בסדר גודל של Full-Stack, הן בצד השרת (לוגיקה וחישובים אלגוריתמיים) והן בצד הלקוח (ממשקי תצוגה). הגרסה העדכנית (Java 21) נבחרה כיוון שהיא מבטיחה תמיכה ארוכת טווח (LTS), מספקת יציבות מערכתית, ומכילה שיפורי ביצועים משמעותיים בניהול זיכרון, המאפשרים כתיבת קוד מודרני, בטוח ויעיל לחישובי האלגוריתם.

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

- על מנת להשלים את מעטפת הפיתוח ולייעל תהליכים, נעשה שימוש בספריות ותשתיות העזר הבאות:
- **Spring Web:** תשתית ליצירת ה-RESTful API וניהול בקשות השרת. ספרייה זו מספקת אנוטציות מפתח למיפוי ניתובים (Routing) ומטפלת אוטומטית בהמרת נתוני אובייקטים לפורמט JSON מאחורי הקלעים.
- **Spring Data MongoDB:** שכבת מתאם הנתונים (ODM) המקשרת בין האובייקטים ב-Java לבין מסמכי הנתונים ב-MongoDB. היא חוסכת כתיבת קוד שאילתות ידני מורכב על ידי שימוש בממשק MongoDBRepository, המייצר שאילתות דינמיות אוטומטית על בסיס שמות הפונקציות המוגדרות.
- **Spring Boot DevTools:** כלי עזר חיוני להאצת תהליך הפיתוח. הכלי מאפשר טעינה מחדש אוטומטית של שינויי קוד, מבטל מנגנוני מטמון (Caching) בזמן הפיתוח, ומבצע אתחול מהיר כדי לוודא שכל שינוי משתקף באופן מיידי ללא צורך בהפעלה מלאה וארוכה של השרת.
- **MongoDB Atlas:** שירות ענן מנוהל (DBaaS) המשמש לאירוח מסד הנתונים של המערכת ללא צורך בתחזוקת שרת פיזי. השירות מספק מעטפת אבטחה מקיפה הכוללת ניהול הרשאות גישה וכן מניעת גישה עוינת ברמת הרשת על ידי הגדרת רשימת כתובות IP מורשות בלבד (Whitelist).

12.2 תיאור מבנה הפרויקט והמחלקות

12.2.1 עץ המודולים (חבילות) של הפרויקט

```

C:.\
└── smartcart
    ├── AppMain.java
    ├── datamodels
    │   ├── Order.java
    │   ├── OrderItem.java
    │   ├── Product.java
    │   ├── Role.java
    │   ├── TagField.java
    │   └── User.java
    ├── repositories
    │   ├── OrderRepository.java
    │   ├── ProductRepository.java
    │   ├── TagFieldRepository.java
    │   └── UserRepository.java
    ├── services
    │   ├── DataBaseInitializerService.java
    │   ├── OrderService.java
    │   ├── ProductService.java
    │   ├── ScoringService.java
    │   ├── SmartCartAlgoService.java
    │   ├── TagFieldService.java
    │   ├── UserService.java
    │   └── WeatherService.java
    └── ui
        ├── AdminOrdersTab.java
        ├── AdminProductTab.java
        ├── AdminStatsTab.java
        ├── AdminUserTab.java
        ├── AdminView.java
        ├── CartView.java
        ├── HistoryView.java
        ├── HomeView.java
        ├── LoginView.java
        ├── MainLayout.java
        ├── ProductCard.java
        ├── ProductEditDialog.java
        ├── QuestionnaireAdminDialog.java
        ├── RegistrationView.java
        ├── ShopView.java
        ├── SmartCartView.java
        └── WeatherRecommendationSection.java
  
```

תיאור מבנה הפרויקט

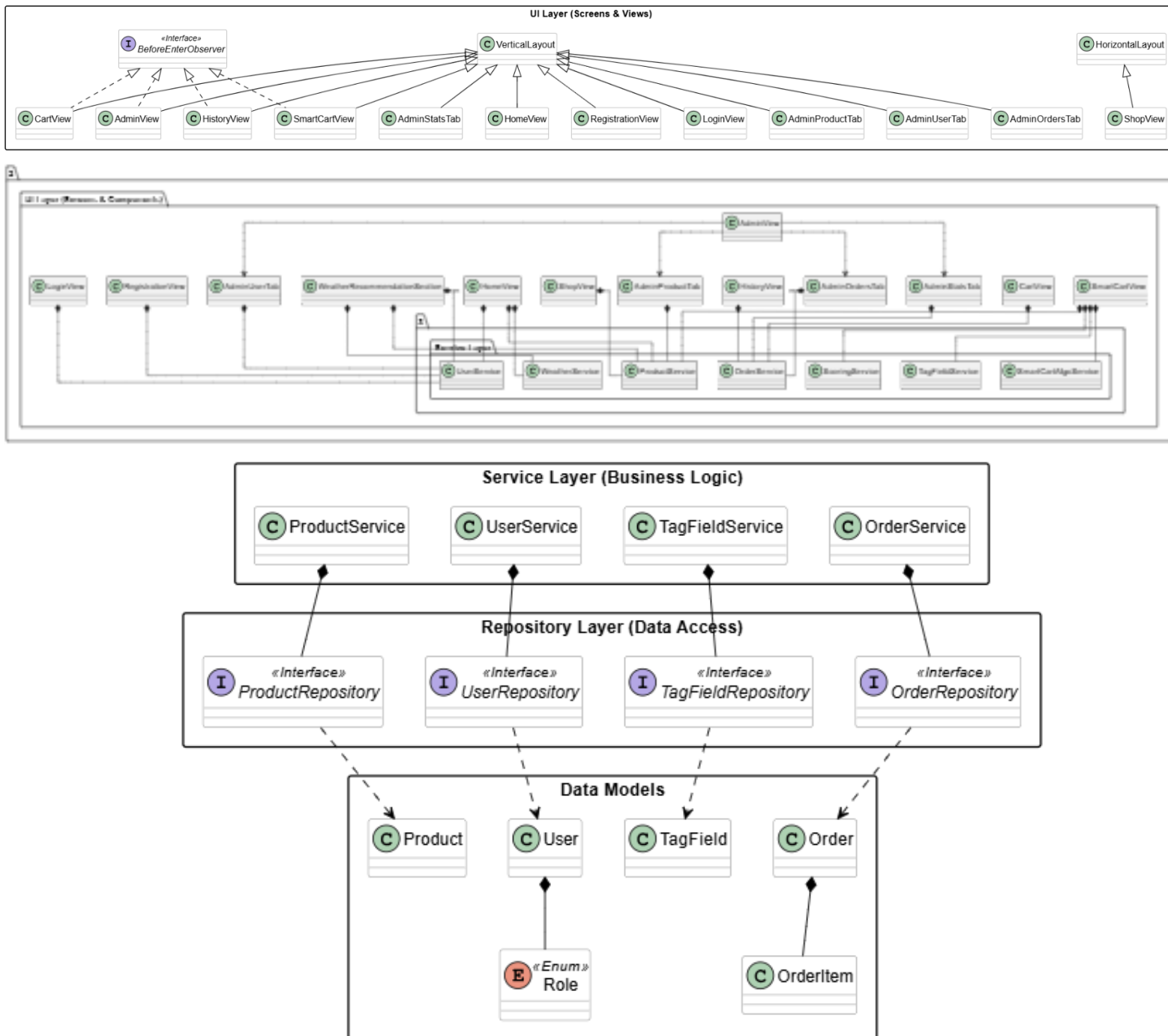
המערכת פותחה בהתאם לסטנדרטים של **ארכיטקטורת שכבות**, המבטיחה הפרדה ברורה בין ניהול הנתונים, הלוגיקה העסקית והממשק הגרפי. מבנה זה מאפשר תחזוקה גבוהה, פיתוח מודולרי ויכולת הרחבה עתידית.

פירוט השכבות:

- **datamodels (שכבת ה-Entities):** שכבה זו מגדירה את המבנה של אובייקטי הנתונים במערכת. היא משתמשת במיפוי מול מסד הנתונים MongoDB (באמצעות Spring Data), ומייצגת את הישויות המרכזיות כמו משתמשים (User), מוצרים (Product) והזמנות (Order).
- **repositories (שכבת ה-Data Access):** שכבה זו אחראית על התקשורת מול מסד הנתונים. היא מספקת ממשקים לביצוע פעולות CRUD (יצירה, קריאה, עדכון ומחיקה) בצורה בטוחה ומהירה, ללא צורך בכתיבת שאילתות מורכבות.
- **services (שכבת הלוגיקה העסקית):** ה-"מוח" של המערכת. כאן מרוכזים כל האלגוריתמים והחישובים המורכבים, כגון מנוע ה-Knapsack (SmartCartAlgoService), ניתוח העדפות (ScoringService), ותקשורת עם שירותים חיצוניים (WeatherService). שכבה זו מנתקת את הלוגיקה מה-UI, ומאפשרת שימוש חוזר בקוד.
- **ui (שכבת התצוגה):** שכבת הממשק מבוססת על Vaadin Framework. היא מחולקת למסכים (Views) ורכיבים (Components). השכבה צורכת את ה-Services כדי להציג נתונים למשתמש, תוך הקפדה על הפרדת התצוגה מהלוגיקה העסקית (שמירה על "Thin UI").
- **AppMain:** נקודת הכניסה של האפליקציה, האחראית על איתחול הסביבה והפעלת השרת.

12.2.2 תרשים המחלקות UML Class Diagram

להלן תרשים המחלקות המפורט:

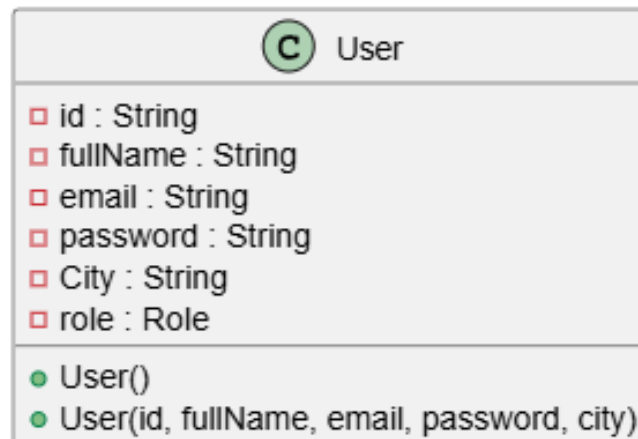


12.2.3 תיאור מחלקות מפורט

להלן תיאור מחלקות מפורט, עבור כל מחלקה נוסף תרשים מקוצר, פעולות ותכונות מרכזיות, והסברים מפורטים.

תיקית – DataModels

User

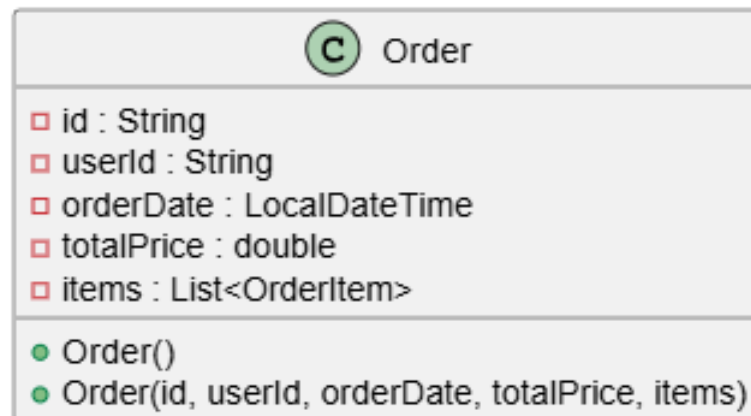


תיאור ותפקיד:

מחלקה זו מהווה ישות בסיסית במערכת, המייצגת משתמש רשום. המחלקה ממופת ישירות לאוסף בשם Users בתוך מסד הנתונים MongoDB באמצעות אנוטציית @Document.

דגשים:

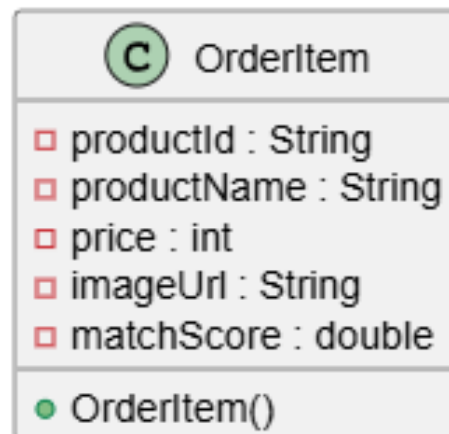
- **אחריות:** המחלקה מרכזת את כל פרטי הזהות של המשתמש, כולל נתוני התחברות והרשאות (Role).
- **ניהול הרשאות:** השימוש בטיפוס Role מאפשר הפרדה לוגית בין משתמש רגיל למנהל מערכת (ADMIN), דבר המהווה את הבסיס למנגנון בקרת הגישה באפליקציה.

Order**תיאור ותפקיד:**

מחלקה זו מייצגת את ישות ההזמנה במערכת. היא משמשת לתיעוד היסטוריית הרכישות ומבצעת קישור לוגי בין המשתמש (userId) לבין אוסף הפריטים שנרכשו בפועל. המחלקה ממופת לאוסף Orders ב-MongoDB.

דגשים טכנולוגיים:

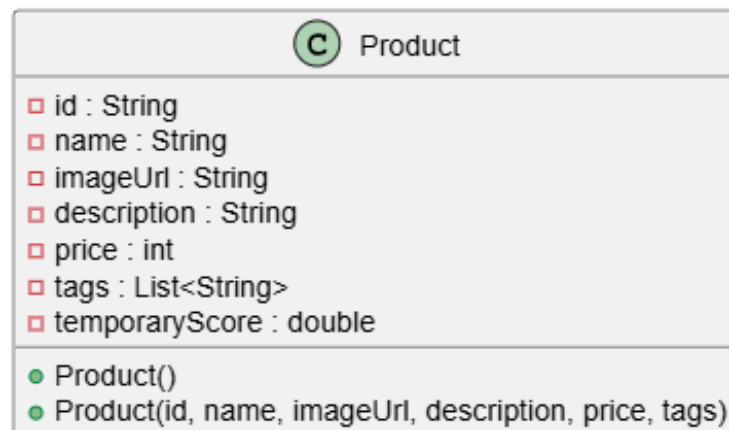
- **אחריות:** ניהול כלל נתוני העסקה - החל משיוך המשתמש, דרך תזמון הרכישה (LocalDateTime), ועד חישוב העלות הכוללת.
- **קומפוזיציה:** המחלקה מכילה רשימה (List<OrderItem>), המייצגת קשר של "הכלה". המשמעות היא שפריטי ההזמנה הם ישויות המוגדרות כחלק מההזמנה עצמה ואינם קיימים באופן עצמאי מחוץ להקשר של עסקת המכירה.
- **ניהול נתונים:** השימוש ב-List מאפשר למערכת לתמוך בסלי קניות המכילים מספר פריטים משתנה, מה שמעניק גמישות רבה למודל העסקי.

OrderItem**תיאור ותפקיד:**

מחלקה זו מייצגת פריט בודד בתוך הזמנה. תפקידה המרכזי הוא לשמש כ-Snapshot (צילום מצב) של המוצר כפי שהיה ברגע הרכישה. בניגוד למוצר ב"קטלוג", שעלול להשתנות (עדכון מחיר, שינוי שם), ה-OrderItem שומר את נתוני המוצר המדויקים כפי שהיו בעת ביצוע העסקה.

דגשים טכנולוגיים:

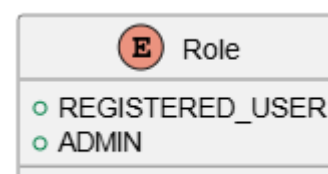
- **שימור נתוני אלגוריתם:** השדה matchScore הוא אחד השדות החשובים בפרויקט. הוא מתעד את הציון שהאלגוריתם החכם העניק לפריט עבור המשתמש, דבר המאפשר ניתוח עתידי של אפקטיביות האלגוריתם.
- **הפרדת אחריות:** המחלקה לא מייצגת "מוצר" (Product) אלא "תיעוד רכישה". לכן היא כוללת רק את המידע הנחוץ להצגת היסטוריית הזמנות (שם, תמונה ומחיר).
- **אופטימיזציה:** שמירת הנתונים בתוך אובייקט ה-OrderItem (במקום רק קישור ל-ID) חוסכת פניות נוספות למסד הנתונים בעת הצגת היסטוריית הזמנות למשתמש.

Product**תיאור ותפקיד:**

מחלקה זו מייצגת פריט בודד בקטלוג הבגדים. היא משמשת כמודל המרכזי לתצוגת מוצרים בממשק המשתמש ולסינון מוצרים באמצעות אלגוריתמים. המחלקה ממופת לאוסף Products ב-MongoDB.

דגשים:

- **אחריות:** ניהול המידע הסטטי של המוצר (שם, תיאור, מחיר, תגיות) לצד מידע דינמי המחושב בזמן ריצה.
- **טיפול בשדות זמניים (@Transient):** השימוש באנוטציה `@Transient` עבור השדה `temporaryScore` הוא ביטוי לאופטימיזציה: הציון מחושב על ידי האלגוריתם בכל פעם שהמשתמש מבקש התאמה אישית, ואין לו צורך להיכתב למסד הנתונים. זה חוסך פעולות כתיבה מיותרות למסד ומבטיח שהמידע נשאר רלוונטי לזמן ריצה בלבד.
- **גמישות סיווג:** השדה `tags <String>List` מאפשר סיווג רב-ערכי של מוצרים (למשל: "חורף", "אלגנט", "מבצע"), מה שמקל על שאילתות סינון וחיפוש מורכבות במערכת.

Role**תיאור ותפקיד:**

מחלקה זו מוגדרת כ-Enum (טיפוס ערכים קבוע) המשמש להגדרת רמת ההרשאות של משתמש במערכת. היא מהווה את "חוזת ההרשאות" של האפליקציה ומבטיחה שרק ערכים מוגדרים מראש יוכלו להיכנס לשדה ה-`role` ביישום ה-`User`.

דגשים:

- **אחריות:** הפרדה חד-משמעית בין משתמש רגיל (REGISTERED_USER) לבין מנהל מערכת (ADMIN).
- **בקרת גישה:** הערכים הללו משמשים בתוך ה-beforeEnter (כפי שראינו ב- AdminView) כדי לבצע אכיפת הרשאות ברמת ה-UI - משתמש שאינו בעל תפקיד ADMIN לא יוכל לצפות בתוכן הניהולי.
- **טיפוס בטוח:** השימוש ב-Enum במקום ב-String פשוט מונע שגיאות הקלדה בזמן פיתוח ומבטיח שהמערכת עובדת רק עם תפקידים תקינים.

TagField

C TagField
id : String fieldName : String options : List<String>
TagField() TagField(fieldName, options)

תיאור ותפקיד:

מחלקה זו מגדירה את מבנה נתוני הסינון של המערכת. היא משמשת לניהול דינמי של תגיות (למשל: "עונה", "צבע", "סגנון"), ומאפשרת למנהל המערכת להוסיף קטגוריות סינון חדשות מבלי לשנות את הקוד. המידע נשמר באוסף tag_fields ב-MongoDB.

דגשים:

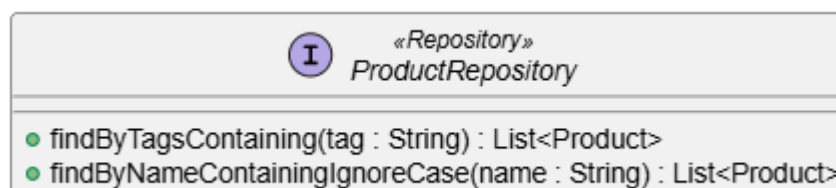
- **אחריות:** הפרדה בין "שם הקטגוריה" (למשל: "צבע") לבין "ערכי הקטגוריה" (למשל: "כחול", "אדום", "ירוק").
- **גמישות:** השימוש במחלקה זו מאפשר לממשק המשתמש לבנות רכיבי ComboBox בצורה דינמית. במקום שהקוד יהיה "נעול" על ערכים ספציפיים, הוא שואב אותם ממסד הנתונים, מה שמקנה גמישות מקסימלית למערכת.
- **טיפול באוספים:** השדה options <String>List מבטיח שכל קטגוריה תוכל להכיל כמות בלתי מוגבלת של אפשרויות סינון, מה שמקל על הרחבת הקטלוג בעתיד.

תיקיית - Repository.**OrderRepository****תיאור ותפקיד:**

ממשק זה מהווה את שכבת הגישה לנתונים עבור אוסף ההזמנות (Orders) ב-MongoDB. הוא יורש מהממשק MongoRepository, מה שמעניק לו אוטומטית את כל פעולות ה-CRUD הבסיסיות (Create, Read, Update, Delete) ללא צורך בכתיבת קוד SQL או שאלות MongoDB מורכבות.

דגשים:

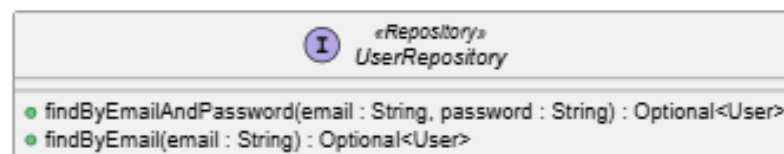
- **אחריות:** תיווך בין הלוגיקה העסקית (OrderService) לבין מסד הנתונים. הממשק מאפשר לשמור ולשלוף אובייקטים מסוג Order.
- **שאלות דינמיות:** הפעולה findById. המערכת מפרשת את שם הפעולה באופן אוטומטי ומייצרת את השאלתה המתאימה למסד הנתונים כדי לשלוף את כל ההזמנות המשויכות למשתמש מסוים, מה שמקצר משמעותית את זמן הפיתוח.
- **סטנדרטיזציה:** השימוש ב-@Repository מגדיר את הממשק כ-Bean בתוך ה-Context של Spring, מה שמאפשר הזרקה (Injection) פשוטה שלו לתוך שירותים אחרים.

ProductRepository**תיאור ותפקיד:**

ממשק זה משמש כשכבת הגישה לנתונים עבור אוסף המוצרים (Products) ב-MongoDB. הוא מאפשר לבצע פעולות CRUD סטנדרטיות לצד שאלות סינון מורכבות, החיוניות לתפקוד האלגוריתם של המערכת.

דגשים:

- **אחריות:** הנגשת נתוני הקטלוג ללוגיקה העסקית. הממשק מרחיב את `MongoRepository`, מה שמבטיח עבודה מול בסיס הנתונים בצורה `Type-safe` (בטוחה מבחינת טיפוסים).
- **שאליות מתקדמות:** * `findByTagsContaining`: משתמשת ביכולת של `Spring Data` לבצע שאלית על שדות שהם מערך (`List/Array`), מה שמאפשר שליפה יעילה של מוצרים לפי תגיות.
 - `findByTagNameContainingIgnoreCase`: מדגימה שימוש ב-`Query` `Keywords`. המערכת מפרשת את המילה `Containing` כחיפוש "מכיל" (חלק ממחרוזת) ואת `IgnoreCase` כביטול רגישות לרישיות (`Case-insensitive`), מה שמבטיח חוויית חיפוש נוחה וידידותית למשתמש.

UserRepository**תיאור ותפקיד:**

ממשק זה מהווה את שכבת הגישה לנתונים עבור אוסף המשתמשים (`Users`) ב-`MongoDB`. הוא מאפשר ביצוע פעולות אימות ורישום של משתמשים בצורה בטוחה ויעילה, תוך שימוש במנגנוני השאליות של `Spring Data`.

דגשים:

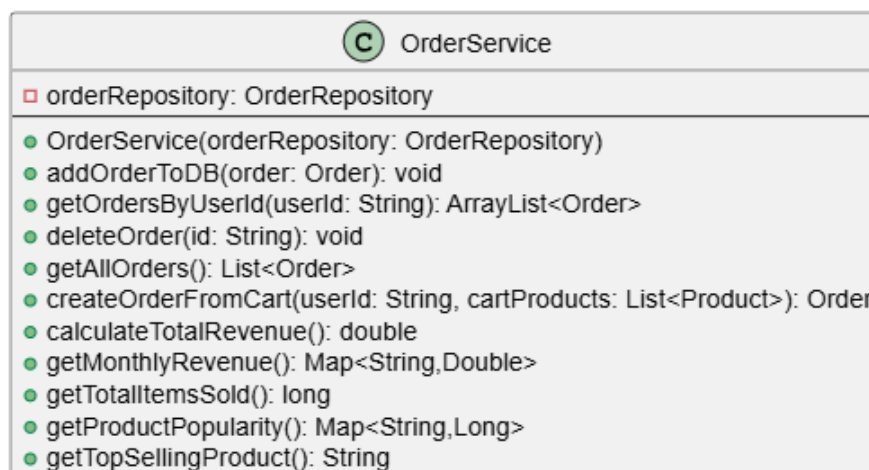
- **אחריות:** ניהול אובייקטי המשתמש, עם דגש על אימות זהות (`Login`) ובדיקת תקינות נתונים בזמן הרשמה (`Registration`).
- **שימוש ב-Optional:** השימוש בטיפוס `Optional<User>` הוא פרקטיקה מתקדמת בתכנות מונחה עצמים. הוא מחייב את המפתח שקורא למתודה להתייחס במפורש למקרה שבו המשתמש לא נמצא, ובכך מונע באופן אקטיבי שגיאות מסוג `NullPointerException` שהן גורם שכיח לבאגים במערכות.
- **אופטימיזציה של שאליות:** `findByEmail` מאפשרת למערכת לבצע בדיקה מהירה של קיום משתמש במערכת בזמן הרשמה, מה שמונע כפילות של כתובות אימייל במסד הנתונים.

TagFieldRepository**תיאור ותפקיד:**

ממשק זה משמש כשכבת הגישה לנתונים עבור אוסף ה-tag_fields ב-MongoDB. מטרתו היא לאפשר שליפה וניהול של הגדרות הסינון הדינמיות של הקטלוג.

דגשים:

- **אחריות:** הנגשת קטגוריות הסינון למערכת. בזכות הירושה מ-MongoRepository, הממשק מקבל "מהקופסה" את כל הפעולות הדרושות למנהל המערכת כדי להוסיף, לעדכן או למחוק קטגוריות סינון (כמו "צבע" או "מידה") מבלי לשנות את הקוד.
- הממשק לא מממש אף מתודה באופן ידני, שכן הפונקציונליות הבסיסית שמציע ה-Framework מספיקה כדי לנהל את המידע הדינמי. זה מפגין הבנה של שימוש יעיל ב-Spring Data כדי לחסוך זמן פיתוח ומקום בקוד.
- **אבסטרקציה:** השכבות העסקיות פונות לממשק זה מבלי להכיר את המורכבות של ה-Database, מה שמאפשר להחליף בעתיד את מסד הנתונים או את מבנה הטבלה מבלי לשבור את שאר חלקי המערכת.

תיקית Services.**OrderService****תיאור ותפקיד:**

מחלקה זו מהווה את "המוח" העסקי והפיננסי של ניהול ההזמנות במערכת. היא משמשת כשכבת תיווך מרכזית (Service Layer) המפרידה לחלוטין בין ממשק המשתמש (UI) לבין מסד הנתונים. מעבר לפעולות CRUD בסיסיות, המחלקה מרכזת את כל הלוגיקה העסקית המורכבת של החנות: המרת עגלת קניות וירטואלית להזמנה ממשית, עיבוד נתונים

סטטיסטיים, וחישוב רווחים פיננסיים (Data Aggregation). עיצוב זה שומר על עקרון "הפרדת הרשויות" (Separation of Concerns) ומבטיח ששכבת התצוגה תישאר נקייה מחישובים.

דגשים:

- **הפרדה ולוגיקה עסקית טהורה:** השירות אחראי על תהליכי ליבה כמו המרת רשימת מוצרים (Product) מהסל לאובייקטי OrderItem היסטוריים, וחישוב אוטומטי של סך התשלום. ה-UI רק מעביר אליו את העגלה, והשירות דואג לכל השאר.
- **עיבוד נתונים וסטטיסטיקה:** המחלקה מכילה אלגוריתמים מתקדמים מבוססי Java Stream API לצורך ניתוח אנליטי של היסטוריית הרכישות. היא מבצעת קיבוץ הכנסות לפי חודשים (getMonthlyRevenue), ספירת פריטים שנמכרו, ואיתור המוצר הפופולרי ביותר, תוך שימוש במפות גיבוב לייעול זמן הריצה.
- **הזרקת תלויות (Dependency Injection):** ה-OrderRepository מוזרק דרך הבנאי. המערכת (Spring) מנהלת את יצירת המופע של ה-Repository ומזריקה אותו לשירות, דבר המבטיח גמישות וקלות בבדיקות יחידה.
- **ניתוק תלות בקטלוג:** בעת יצירת ההזמנה, השירות דואג לשמור את הנתונים הספציפיים של המוצר (שם ומחיר ביום הרכישה) בתוך ההזמנה עצמה. כך נשמרת היסטוריה אמינה גם אם המוצר נמחק או מחירו השתנה בעתיד.

ProductService

«Service» ProductService
productRepository : ProductRepository addProductToDB(product : Product) : boolean getAllProducts() : ArrayList<Product> getProductsByTag(tag : String) : ArrayList<Product> findFiltered(search : String, gender : String, category : String) : List<Product> deleteProduct(id : String)

תיאור ותפקיד:

מחלקה זו מהווה את ה-Service Layer המרכזי לניהול קטלוג המוצרים. תפקידה להנגיש את נתוני המוצרים ל-UI תוך ביצוע מניפולציות לוגיות (סינון וחיפוש) על הנתונים.

דגשים:

- **סינון מתקדם (findFiltered):** זוהי הפעולה המורכבת ביותר במחלקה. היא משתמשת ב-Java Streams API כדי לבצע סינון רב-שלבי על רשימת המוצרים. הפונקציה מאפשרת סינון דינמי לפי שם, מגדר וקטגוריה, תוך התחשבות במצבים

בהם חלק מהקריטריונים ריקים (null). זה מראה על תכנון חכם שמאפשר חוויית משתמש גמישה בחיפוש במוצרים.

- **טיפול ב-Case Sensitivity:** במתודה `findFiltered`, ביצעתי המרה של שם המוצר ושל מחרוזת החיפוש ל-`toLowerCase()` לפני ביצוע ה-`contains`. פעולה זו מבטיחה שהמשתמש יקבל תוצאות מדויקות גם אם הוא לא הקפיד על אותיות גדולות או קטנות בחיפוש.
- **הפרדה בין Repository ל-Service:** בעוד ה-`Repository` מטפל בשאליות בסיסיות מול `MongoDB`, ה-`Service` לוקח את הנתונים ו"מעבד" אותם. גישה זו שומרת על קוד נקי ותומכת באפשרות להרחיב את המערכת בהמשך (למשל, הוספת לוגיקה שמחשבת הנחות או מבצעים לפני שהמוצרים מוצגים למשתמש).

TagFieldService

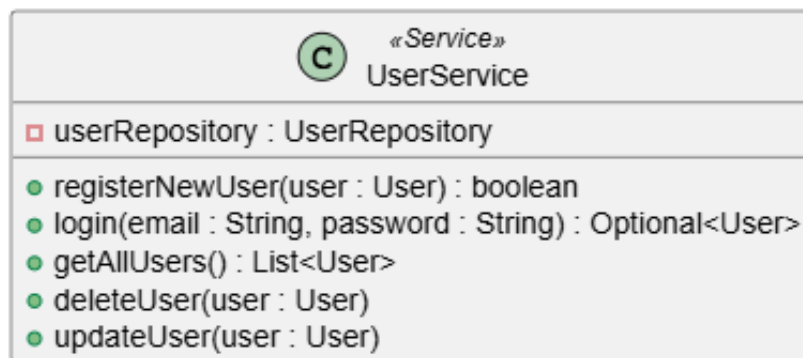


תיאור ותפקיד:

שכבת השירות לניהול שדות התיוג הדינמיים במערכת. שירות זה מהווה את התשתית המאפשרת למנהל האתר לעדכן את אפשרויות הסינון של הקטלוג בזמן אמת, מבלי לשנות את קוד המקור של המערכת.

דגשים טכנולוגיים:

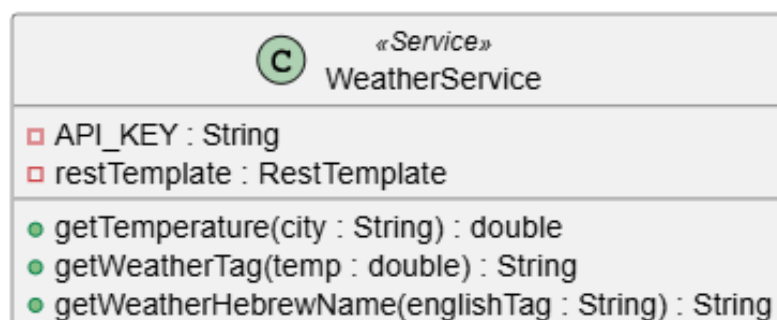
- **אחריות:** ניהול מלא של הגדרות ה-`TagField`. זהו `Service` המעניק גישה ישירה לנתוני התצורה של המערכת, מה שמבטיח עקביות בין מה שהמנהל מגדיר לבין מה שהמשתמש רואה ב-UI.

UserService**תיאור ותפקיד:**

זהו השירות המרכזי לניהול זהויות המשתמשים במערכת. הוא אחראי על אכיפת חוקי הרישום, אימות נתוני התחברות וניהול מחזור החיים של חשבונות המשתמשים.

דגשים:

- **בקרת רישום (registerNewUser):** השירות מבצע בדיקת אימות כפולה - ראשית, הוא מוודא שאין כפילות של כתובת אימייל (שכן היא משמשת כמזהה ייחודי), ושנית, הוא מבצע **הזרקת ערך ברירת מחדל** לתפקיד המשתמש (REGISTERED_USER), מה שמבטיח שכל משתמש שנרשם מקבל הרשאות מתאימות באופן אוטומטי ללא התערבות ידנית.
- **אימות מאובטח (login):** השימוש ב-Optional באימות המשתמש מונע טיפול לא תקין בערכי null ומחייב את שכבת ה-UI לטפל במקרה שבו פרטי ההתחברות אינם נכונים.
- **ניהול הרשאות:** המחלקה משלבת את ה-Role (Enum) בתהליך היצירה, מה שמבטיח שהמערכת עובדת תחת מודל הרשאות סדור (Role-based access).

WeatherService**תיאור ותפקיד:**

מחלקה זו מהווה שירות אינטגרציה חיצוני. היא אחראית על שליפת נתוני מזג אוויר בזמן אמת מ-API חיצוני (OpenWeatherMap) ותרגומם לתגיות לוגיות שהמערכת יודעת לעבוד איתן ("Winter", "Summer" וכו').

דגשים (לוגיקה עסקית):

- **צריכת API ו-Parsing:** השירות משתמש ב-`RestTemplate` לביצוע קריאות HTTP וב-`ObjectMapper` (Jackson) כדי לבצע Parsing (פענוח) למבנה ה-JSON המורכב שמגיע מהשרת. זהו מימוש של דפוס עבודה קלאסי בתקשורת Client-Server.
- **טיפול בחריגים:** במתודה `getTemperature`, הוספתי מנגנון try-catch שמחזיר ערך ברירת מחדל במקרה של תקלה (למשל, אין אינטרנט או ה-API לא זמין). זה מבטיח שהאפליקציה לא תקרוס אם השירות החיצוני נופל.
- **תרגום ערכים (Mapping):** השירות לא רק מביא את הנתון הגולמי, אלא מבצע לוגיקה של החלטה (`getWeatherTag`) שקובעת איזו תגית של "עונה" מתאימה לטמפרטורה שנמדדה. בנוסף, קיימת הפרדה בין התגית הלוגית לבין התצוגה בעברית (`getWeatherHebrewName`), מה שמאפשר גמישות בשפות הממשק מבלי לפגוע בלוגיקה של מסד הנתונים.

ScoringService

C ScoringService	
●	<code>calculateScoresBySurvey(products : List<Product>, selectedTags : List<String>)</code>
■	<code>hasTag(tags : List<String>, value : String) : boolean</code>

תפקיד המחלקה:

מחלקה זו מתפקדת כמנוע ההתאמה של המערכת. היא אחראית על חישוב דינמי של "ציון התאמה" לכל מוצר בקטלוג, בהתבסס על ההעדפות שהמשתמש בחר בשאלון. הציון משקף באחוזים עד כמה הפריט תואם לדרישות הספציפיות של המשתמש.

פעולות משמעותיות:

- `calculateScoresBySurvey`: הפעולה המרכזית במחלקה. מקבלת רשימת מוצרים ורשימת תגיות מבוקשות. היא מבצעת חישוב יחסי (באחוזים) של כמות התגיות החופפות, תוך התעלמות מבחירות ריקות או בחירה ב"הכל". בסוף החישוב, הפעולה מעגלת את הציון ושומרת אותו באופן זמני באובייקט המוצר.
- `hasTag`: פעולת עזר פנימית שמבצעת חיפוש בטוח של תגית בתוך רשימה. היא מוודאת שההשוואה תעבוד גם אם יש הבדלים באותיות גדולות/קטנות (Case-Insensitive) או רווחים מיותרים, ובכך מונעת שגיאות סינון.

SmartCartAlgoService

 SmartCartAlgoService
• <code>getOptimalBundle(items : List<Product>, budgetUnits : int, minItems : int, maxItems : int) : List<Product></code>

תפקיד המחלקה:

מחלקה זו מהווה את הליבה האלגוריתמית והמתמטית של המערכת ("הסל החכם"). תפקידה הוא לחשב ולהרכיב את סל הקניות האופטימלי ביותר עבור המשתמש, תוך ניהול מאזן עדין בין מספר אילוצים מרכזיים המתרחשים במקביל כגון - עמידה במסגרת תקציב מקסימלית שהמשתמש הגדיר, שמירה על דרישות כמות הפריטים (רף מינימום ורף מקסימום), ומיקסום כולל של ציון ההתאמה (Score) השלם של הסל. מכיוון שבעיה כזו אינה ניתנת לפתרון מדויק על ידי מיון רגיל או גישה "חמדנית" (Greedy algorithm) "שכן בחירת פריט אחד משפיעה ישירות על התקציב והמקום הנותרים לפריטים אחרים – המחלקה מספקת פתרון הנדסי ויעיל לבעיית אופטימיזציה מורכבת, (NP-Hard) כדי להעניק ללקוח את התמורה הגבוהה והמדויקת ביותר לכספו.

פעולות משמעותיות:

- **getOptimalBundle**: זוהי הפעולה המרכזית במחלקה, המממשת אלגוריתם תכנון דינמי (Dynamic Programming) מתקדם המבוסס על וריאציה תלת-ממדית של בעיית "תרמיל הגב" (0/1 Knapsack Problem) "הפעולה מבוצעת במספר שלבי עומק:

ראשית, היא מאתחלת מערך תלת-ממדי (dp) שבו כל תא מייצג מצב לוגי אפשרי ומוגדר מראש. שלושת הממדים של המערך מייצגים את: אינדקס הפריט הנוכחי מתוך רשימת המוצרים, כמות הפריטים המדויקת שנבחרה עד לאותו שלב, וסך התקציב שנוצל בפועל. מצבים שאינם הגיוניים מתמטית מאותחלים בערך -1 כדי לסננם.

שנית, הפעולה מריצה לולאות מקוננות הממלאות את הטבלה. עבור כל מוצר, הפעולה בוחנת את מצב התקציב והכמות הנתונים, ומקבלת החלטה לוגית בין שתי חלופות: "לדלג" על הפריט (ולשמר את הציון המקסימלי שהושג בלעדיו עד כה), או "לקחת" אותו. ההחלטה לקחת אותו תתבצע רק אם התקציב מאפשר זאת ואם התוספת של ציון המוצר לציון של תת-הבעיה הנותרת מייצרת ערך גבוה יותר.

שלישית, עם סיום חישוב כל תתי-הבעיות ומילוי הטבלה, האלגוריתם סורק את מרחב התוצאות הסופיות (בשורה האחרונה של המערך) כדי לאתר את הציון המקסימלי האבסולוטי (bestScore) שעדיין עומד בטווח הכמויות המורשה (בין minItems ל-maxItems).

בסופו של תהליך, מבוצע "שחזור לאחור" (Traceback) "האלגוריתם מתחיל מנקודת השיא שנמצאה וצועד אחורה בתוך המערך התלת-ממדי כדי להתחקות אחר מסלול ההחלטות. כל פריט שעבורו הוכח כי הבחירה "לקחת" העלתה את הערך הכולל – נדגם, מתווסף לרשימת התוצאות, ומוחזר לבסוף כסל הקניות האופטימלי והסופי של המשתמש.

תיקיית UI.

HomeView

C HomeView
weatherService : WeatherService productService : ProductService userService : UserService
HomeView(weatherService : WeatherService, productService : ProductService, userService : UserService) createHeader(user : User) buildGuestView(layout : VerticalLayout) buildAuthenticatedView(layout : VerticalLayout, user : User) createMenuCard(title : String, desc : String, vlcon : VaadinIcon, route : String) : VerticalLayout

תפקיד המחלקה:

מחלקה זו מהווה את דף הנחיתה הראשי והמרכזי של המערכת, המשמש כצומת ניתוב עבור המשתמשים. היא אחראית על תצוגה דינמית המשתנה בזמן אמת, המערכת בודקת את ה VaadinSession ומחליטה האם להציג חלון ראוה לאורחים או לוח בקרה אישי למשתמשים מחוברים. השימוש ברכיבי הפריסה של vaddin (VerticalLayout, FlexLayout) מאפשר יצירת ממשק מודרני ורספונסיבי ישירות מקוד ה - Java, ללא צורך בקובצי HTML חיצוניים.

פעולות משמעותיות:

- **buildGuestView**: פעולה הבונה את ממשק המשתמש עבור אורחים שטרם הזדהו. היא כוללת אלמנטים שיווקיים כמו כותרות מעוצבות, תמונת אווירה, וכפתורי הנעה לפעולה המנתבים את המשתמש במהירות לעמוד ההרשמה או לצפייה חופשית בקטלוג.
- **buildAuthenticatedView**: פעולה המרכיבה את לוח הבקרה האישי למשתמשים רשומים. מעבר ליצירת כרטיסי הניווט הראשיים (חנות, סל חכם, היסטוריה), הפעולה מדגימה הרכבת רכיבים על ידי שילוב ווידג'ט מזג האוויר (WeatherRecommendationSection) המופעל אוטומטית על בסיס עיר המגורים השמורה של המשתמש.

LoginView

C LoginView
userService : UserService
LoginView(userService : UserService)

תפקיד המחלקה:

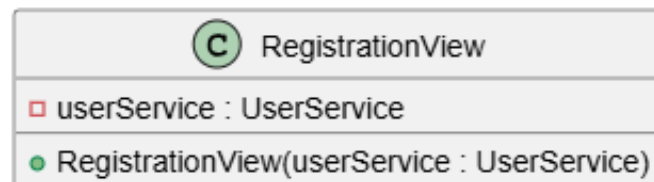
מחלקה זו אחראית על ניהול שער הכניסה לאפליקציה (Authentication). היא מספקת ממשק משתמש מאובטח לאימות נתוני הגישה מול מסד הנתונים באמצעות שירות ה-

UserService. בנוסף לאימות, המחלקה מנהלת את ה State של המשתמש באפליקציה על ידי יצירת ה Session ההתחלתי שילווה אותו לאורך כל חוויית הקנייה, דבר קריטי לשמירת נתונים זמניים כמו עגלת הקניות.

פעולות משמעותיות:

- **בנאי (Constructor) ולוגיקת אימות:** הבנאי מגדיר את רכיב ה - LoginForm המובנה של Vaadin ומתרגם אותו לעברית באמצעות LoginListener. החלק המרכזי הוא מאזין ההתחברות (Login Listener). במקרה של זיהוי מוצלח, המערכת שומרת את אובייקט ה User ב - VaadinSession, מאתחלת עבורו סל קניות ריק, ומציגה הודעת ברכה. במקרה של שגיאה, המערכת מפעילה את מנגנון השגיאה הפנימי של הטופס ללא קריסת המערכת, תוך מתן חיווי ברור למשתמש.

RegistrationView



תפקיד המחלקה:

מחלקה זו מנהלת את תהליך קליטת המשתמשים החדשים במערכת. היא בונה טופס אינטראקטיבי ורספונסיבי לאיסוף פרטי היסוד של הלקוח, ומשמשת כ"שומר סף" המוודא את תקינות הנתונים לפני שליחתם לשכבת השרת. השימוש ב - FormLayout מבטיח שהטופס יתאים את עצמו אוטומטית למסכי מחשב או מכשירים ניידים, ומעניק חוויית שימוש חלקה.

פעולות משמעותיות:

- **בנאי (Constructor) ואימות נתונים:** הבנאי מאגד את כל שדות הקלט (כגון PasswordField ו EmailField). בלוק הלוגיקה של כפתור ההרשמה מבצע אימות צד-לקוח חיוני (כמו השוואת סיסמאות) לפני הפנייה ל DB. לאחר מכן, הוא יוצר אובייקט משתמש חדש, מקצה לו הרשאות ברירת מחדל (REGISTERED_USER), ומטפל בתשובת השרת. במקרה של הצלחה המשתמש מנותב למסך ההתחברות עם הודעה ירוקה, ובמקרה של כפילות נתונים (כמו אימייל תפוס) מוצגת הודעת שגיאה נקודתית המאפשרת לו לתקן את ההזנה.

MainLayout

C MainLayout
headerContainer : HorizontalLayout cartBadge : Span
● MainLayout() ● beforeEnter(event : BeforeEnterEvent) ■ createHeader() ■ createProfileMenu(user : User) : Div ■ createCartIcon() : HorizontalLayout ● refreshCartCount() ■ createNavButton(text : String, icon : VaadinIcon, route : String) : Button

תפקיד המחלקה:

המחלקה מהווה את תבנית הבסיס (Layout) שעוטפת את כל דפי האפליקציה. היא אחראית על יצירת וניהול סרגל הניווט העליון המשותף לכל המסכים במערכת. מעבר לתצוגה הסטטית, המחלקה מממשת את הממשק BeforeEnterObserver, מה שמאפשר לה להאזין לאירועי ניווט ולרענן את הסרגל באופן אוטומטי. כך היא מבטיחה שתמיד יוצג התוכן הרלוונטי ביותר למצב המשתמש בזמן אמת (למשל, הסתרת כפתור "התחבר" והצגת תפריט פרופיל אישי מיד לאחר זיהוי מוצלח של המשתמש מול השרת או הצגת כפתור "ניהול אתר" במידה ומזוהה כאדמין).

פעולות משמעותיות:

- **beforeEnter , createHeader**: פעולות אלו עובדות בסנכרון כדי לנקות ולבנות מחדש את סרגל הניווט בכל מעבר דף. הן שולפות את אובייקט המשתמש מה - VaadinSession ומתאימות את התפריט להרשאותיו (כך לדוגמה, נוצר כפתור "ניהול אתר" שמוצג אך ורק למשתמשים בעלי תפקיד אדמין).
- **refreshCartCount**: פונקציה ציבורית קריטית המאפשרת לרכיבים חיצוניים במערכת (כמו כרטיסי המוצר השונים) לעדכן בזמן אמת את המספר המופיע על סמל עגלת הקניות. עדכון זה מתבצע ברמת ה DOM בלבד ללא צורך ברענון העמוד כולו, מה שתורם משמעותית לחוויית משתמש חלקה ומהירה.

.WeatherRecommendationSection

C WeatherRecommendationSection
weatherService : WeatherService productService : ProductService userService : UserService productsLayout : HorizontalLayout missingCityLayout : VerticalLayout infoLabel : Span
WeatherRecommendationSection(weatherService : WeatherService, productService : ProductService, userService : UserService) setupMissingCityForm() loadRecommendations(city : String)

תפקיד המחלקה:

זהו רכיב UI מותאם אישית (Widget) חכם המשמש להצגת המלצות לבוש דינמיות על בסיס נתוני אמת. המחלקה משלבת מידע משלושה שירותי Backend שונים (מזג אוויר, קטלוג, משתמשים) כדי לייצר חוויית קנייה אישית וייחודית ללקוח. היא מבררת את הטמפרטורה בעיר המגורים של המשתמש, מתרגמת אותה לעונה רלוונטית (כמו קיץ או חורף), ומציגה פריטים תואמים מהחנות. במקרה שבו המשתמש טרם הזין עיר מגורים בהגדרותיו, הרכיב יודע לשנות את פניו לחלוטין ולהציג טופס עדכון פרטים במקום לקרוס או להציג שגיאה.

פעולות משמעותיות:

- **loadRecommendations**: הפעולה המרכזית של הווידג'ט. היא מקבלת את שם העיר, פונה ל-API החיצוני לקבלת טמפרטורה, ונעזרת ב- Streams כדי לסנן את קטלוג המוצרים המלא בהתאם למזג אוויר בעיר המתמש. לבסוף היא חותכת את הרשימה ל-4 פריטים בלבד ומייצרת מהם כרטיסי מוצר מכווצים שמשתלבים יפה בשטח המסך המוגבל.
- **setupMissingCityForm**: הפעולה שאחראית על יצירת הממשק החלופי. היא מאפשרת למשתמש להקליד את שם העיר שלו ישירות מתוך הווידג'ט. בלחיצת כפתור, היא מעדכנת את הנתון ב- Database, רושמת אותו בסשן המקומי, ומיד קוראת מחדש לטעינת ההמלצות - הכל בצורה חלקה וללא טעינת דף.

.ProductCard

C ProductCard
ProductCard(product : Product) ProductCard(product : Product, compact : boolean) handleAddToCart(product : Product)

תפקיד המחלקה:

מחלקה זו היא רכיב תצוגה עצמאי המייצג אובייקט "מוצר" באופן ויזואלי במערכת. החוזקה המרכזית שלה היא בתמיכה בשני מצבי תצוגה: מצב "רגיל" המיועד לעמוד החנות המרכזית (וכלל תיאור מלא והרכב בד), ומצב "קומפקטי" שנועד לחסוך גובה ומקום (עבור הווידג'טים השונים).

גמישות אדריכלית זו מונעת כפילות קוד במערכת ומאפשרת לשמור על אחדות עיצובית של המוצרים בכל רחבי האפליקציה, תוך התאמה אוטומטית למיקום שלהם במסך.

פעולות משמעותיות:

- **בנאי משולב:** המחלקה כוללת שני בנאים, כאשר המרכזי שבהם מקבל בנוסף למוצר גם דגל בוליאני (compact). בהתאם לדגל זה, הבנאי משנה בצורה דינמית את מידות הכרטיס (רוחב וגובה), את גודל הגופנים, ואת נראות רכיבי הטקסט המשניים, כדי לדחוס את התוכן בצורה אסתטית.
- **handleAddToCart:** מאגדת את הלוגיקה המופעלת בעת לחיצה על כפתור הרכישה. היא מוודאת תחילה קיום של משתמש מחובר (ואם אין, מנתבת להתחברות). במידה ויש משתמש, היא מצרפת את המוצר לרשימת עגלת הקניות שב Session, שולחת התראת הצלחה ירוקה, ומאתרת את מופע ה MainLayout הפעיל כדי להורות לו לרענן את ה Badge של הסל בפינת המסך.

.ShopView

C ShopView
- productService : ProductService - productGrid : FlexLayout - searchField : TextField - subCategoryContainer : VerticalLayout - selectedGender : String - selectedCategory : String
- ShopView(productService : ProductService) - createSidebar() : VerticalLayout - showSubCategories(gender : String) - createSubBtn(categoryName : String) - createMainContent() : VerticalLayout - updateList()

תפקיד המחלקה:

מחלקה זו מהווה את ממשק חנות הבגדים המרכזית של המערכת, בה הלקוח יכול לשוטט ולחפש פריטים מתוך הקטלוג המלא. היא מעוצבת בתבנית של מסך מפוצל הכולל סרגל כלים צידי (Sidebar) לסינון קטגוריאלי, ואזור תוכן מרכזי המציג את כרטיסי המוצרים בפריסה גמישה (Grid). בזכות שימוש נכון ברכיבי התצוגה של Vaadin, הדף מתעדכן באופן דינמי וללא רענון, מה שמעניק חוויית דפדוף רציפה, מהירה ומודרנית התואמת לאתרי סחר אלקטרוני מתקדמים ומשאירה את הלקוח ממוקד בקנייה.

פעולות משמעותיות:

- **createMainContent , updateList:** פעולות העובדות יחד לניהול אזור התצוגה המרכזי. מוגדר שדה חיפוש טקסטואלי חכם הפועל במצב LAZY, כלומר הוא ממתין שהמשתמש יסיים להקליד את המילה לפני שהוא פונה לשרת, במטרה לחסוך קריאות עומס מיותרות ל Database בכל הקלדת אות. פעולת ה updateList

אחראית לשלוף בזמן אמת את המוצרים התואמים לשילוב המדויק של טקסט החיפוש, המגדר והקטגוריה, ולרנדר מחדש את רשת כרטיסי המוצר בהתאמה מושלמת.

- **createSidebar , showSubCategories**: פעולות הבונות את תפריט הסינון הצידי והאינטראקטיבי. הלוגיקה כאן מאפשרת חשיפה הדרגתית של אפשרויות. לחיצה על כפתור מגדר ("גברים" או "נשים") מנקה מיד את הבחירות הקודמות ומייצרת בזמן אמת כפתורי תת-קטגוריה ספציפיים ורלוונטיים לאותו מגדר (כמו "שמלות" או "חולצות"). גישה עיצובית זו שומרת על ממשק משתמש נקי, מונעת עומס ויזואלי ומנחה את הלקוח בצורה אינטואיטיבית אל הפריט המבוקש.

.CartView

C CartView
□ orderService : OrderService
● CartView(orderService : OrderService)
■ createItemRow(p : Product, cart : List<Product>) : HorizontalLayout
■ processPayment()
● beforeEnter(event : BeforeEnterEvent)

תפקיד המחלקה:

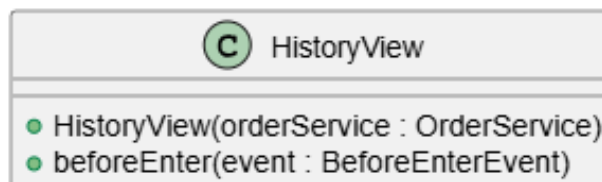
מחלקה זו מנהלת את עגלת הקניות של הלקוח ואת תהליך הרכישה (Checkout). היא קוראת את רשימת הפריטים השמורה זמנית ב - VaadinSession ומציגה אותם בצורה חזותית ונוחה, תוך ביצוע חישובים מתמטיים אוטומטיים של הסכום הכולל לתשלום. המחלקה מהווה את תחנת הסיום וההמרה של תהליך הקנייה, שכן היא אחראית על סימולציית הסליקה המאובטחת, יצירת אובייקט ההזמנה ההיסטורי (Order), ושמידתו באופן קבוע במסד הנתונים כדי להפוך עגלה וירטואלית זמנית לעסקה ממשית ומתועדת.

פעולות משמעותיות:

- **processPayment**: הפעולה המורכבת ביותר בדף המדמה תהליך תשלום אסינכרוני. היא פותחת חלון קופץ (Dialog) עם מד התקדמות מונפש (ProgressBar) ומריצה תהליך רקע (Thread) כדי למנוע את קפיאת ממשק המשתמש בזמן ה"סליקה". בסיום ההשהיה היזומה, היא אורזת ביעילות את כל פריטי הסל לרשימת OrderItem, שומרת את ההזמנה דרך ה - OrderService בבסיס הנתונים, מרוקנת את הסשן, ומציגה הודעת אישור ברורה המאפשרת למשתמש לנווט להיסטוריה או חזרה לחנות.
- **createItemRow**: פעולה הבונה שורה גרפית עבור כל פריט בודד בסל, הכוללת תמונה ממוזער, שם, מחיר מעוצב, וכפתור מחיקה פונקציונלי. הלוגיקה של כפתור המחיקה (אייקון פח האשפה) מבצעת מניפולציה ישירה ומיידית על אוסף הנתונים

שבסשן, מעדכנת את הזיכרון ומרעננת את המסך. כך הלקוח זוכה לשליטה מלאה, בטוחה ומהירה על תוכן העגלה שלו רגע לפני ביצוע התשלום.

.HistoryView



תפקיד המחלקה:

מחלקה זו מספקת ללקוח אזור אישי ובטוח לצפייה בהיסטוריית הרכישות המלאה שלו. היא שולפת ממסד הנתונים את כל ההזמנות שבוצעו בעבר על ידי המשתמש המחובר, ומציגה אותן ברכיב טבלה מתקדם ורספונסיבי (Grid) הכולל תאריכים מפורטים, סכומים סופיים ופירוט ויזואלי של הפריטים. רכיב זה חיוני ביותר לבניית אמון וחוויית שירות לקוחות עצמאי (Self-Service) ברמה גבוהה, שכן הוא מאפשר למשתמש לעקוב בשקיפות אחר ההוצאות שלו ולשחזר אילו פריטים רכש, מבלי לפנות לתמיכה טכנית חיצונית.

פעולות משמעותיות:

- **עיבוד נתונים דינמי בתוך ה - Grid:** הבנאי של המחלקה אינו מסתפק ביצירת טבלה פשוטה, אלא משתמש ביכולות עיבוד מורכבות של Vaadin. ההישג המרכזי הוא השימוש ב - ComponentRenderer עבור עמודת "מוצרים שנרכשו". כלי זה מאפשר לרוץ בלולאה פנימית על פריטי ההזמנה (OrderItem) ולייצר בזמן אמת שורת תמונות ממוזערות של הבגדים, הכוללת מנגנון Tooltip חכם המציג את שם המוצר המלא בעת ריחוף עכבר, פתרון אלגנטי וקומפקטי שמייתר לחלוטין את הצורך בטבלאות מקוננות ומסורבלות למראה.
- **beforeEnter:** פעולת הגנה אבטחתית קריטית המממשת את BeforeEnterObserver. מכיוון שדף זה מכיל מידע אישי, כלכלי ורגיש, הפעולה מוודאת הרשאות לפני טעינת הדף. במידה ואורח לא מזוהה ינסה להקליד את כתובת ה - "history" ישירות בדפדפן, המערכת תזהה שהששן שלו ריק מנתוני זיהוי, תחסום את הגישה לנתונים ותנתב אותו באופן אוטומטי ובטוח חזרה לעמוד הבית הציבורי.

SmartCartView

SmartCartView
dpService : SmartCartAlgoService productService : ProductService scoringService : ScoringService tagFieldService : TagFieldService
SmartCartView(dpService : SmartCartAlgoService, productService : ProductService, scoringService : ScoringService, tagFieldService : TagFieldService) buildDynamicQuestionnaire() runSmartAlgoWithProgress() updateStatus(dialog : Dialog, bar : ProgressBar, label : Span, val : double, text : String, percent : int) displayResults() beforeEnter(event : BeforeEnterEvent)

תפקיד המחלקה:

מחלקה זו היא "המוח" האינטראקטיבי של המערכת וגולת הכותרת של הפרויקט. היא מספקת למשתמש ממשק ויזואלי להגדרת אילוצים אישיים (תקציב מקסימלי, כמות פריטים רצויה) ולמילוי שאלון העדפות סטייל דינמי. לאחר איסוף הנתונים, המחלקה אחראית על תהליך מורכב המשלב בין שירותי חישוב הציונים (ScoringService) לאלגוריתם הליבה לאופטימיזציה (SmartCartAlgoService), במטרה להרכיב באופן אוטומטי סל קניות מותאם אישית. בנוסף, המחלקה מזהה אם המשתמש המחובר הוא מנהל (Admin), ומציגה עבורו כפתור ייעודי לעריכת השאלון ישירות ממסך זה, מה שמעיד על עיצוב מודולרי וחכם.

פעולות משמעותיות:

- **buildDynamicQuestionnaire**: פעולה גמישה וחכמה המייצרת את ממשק השאלון "על הלוח. במקום "לקודד קשה" את שדות הסינון, היא שולפת את השדות והאפשרויות ממסד הנתונים באמצעות ה- TagFieldService, ומייצרת רכיבי Select (תיבות גלילה) בהתאמה. גישה זו הופכת את המערכת לסקיילבילית וקלה לתחזוקה עתידית ללא צורך בעדכון קוד.
- **runSmartAlgoWithProgress**: פעולה קריטית לחוויית המשתמש. מכיוון שאלגוריתם ה- Knapsack התלת-ממדי עשוי לדרוש זמן חישוב ממושך, הפעולה מריצה אותו בתוך תהליכון נפרד (Thread) כדי למנוע את קפיאת הדפדפן. במקביל, היא פותחת דיאלוג עם מד התקדמות (ProgressBar) המדווח למשתמש על שלבי העיבוד (איסוף נתונים, מתן ציונים, הרצת DP), ובסיום מחזירה את התוצאות ל UI Thread בבטחה באמצעות ui.access.

.QuestionnaireAdminDialog

C QuestionnaireAdminDialog
tagFieldService : TagFieldService fieldsContainer : VerticalLayout onChangeCallback : Runnable
QuestionnaireAdminDialog(tagFieldService : TagFieldService, onChangeCallback : Runnable) refreshDialogFields()

תפקיד המחלקה:

מחלקה זו מספקת חלונית ניהול מרחפת (Dialog) ומתקדמת המיועדת למנהלי המערכת בלבד. תפקידה לאפשר שליטה מלאה (CRUD) על השדות הדינמיים המרכיבים את אלגוריתם ההתאמה ושאלון הסטייל ("צבע", "סגנון", "בד" וכו'). היא מאפשרת למנהל לייצר קטגוריות סינון חדשות מאפס, להוסיף או להסיר אופציות ספציפיות בתוך כל קטגוריה, או למחוק קטגוריות שלמות. המחלקה מעוצבת בתצוגה נקייה המשתמשת ברכיבים ויזואליים קטנים (כמו תגיות עם כפתור מחיקה) כדי לדחוס מידע רב בצורה נוחה וידידותית.

פעולות משמעותיות:

- **refreshDialogFields**: מנוע הרינדור של החלונית. בכל פעם שמתבצע שינוי (הוספה או מחיקה של שדה/אופציה), פעולה זו מנקה את התצוגה ובונה אותה מחדש ישירות מה - Database. היא מייצרת לולאות המרכיבות שורות עריכה לכל שדה בנפרד. בנוסף, בעת סגירת החלונית, מופעל ה - onChangeCallback (שהועבר כפונקציית Lambda מה - SmartCartView), אשר דואג לרענן באופן אוטומטי את השאלון המוצג ללקוח ברקע כדי לשקף את השינויים שבוצעו זה עתה.

.AdminView

C AdminView
productService : ProductService userService : UserService orderService : OrderService contentContainer : Div
AdminView(productService : ProductService, userService : UserService, orderService : OrderService) updateContent(selectedTab : Tab, products : Tab, users : Tab, orders : Tab, stats : Tab) beforeEnter(event : BeforeEnterEvent)

תפקיד המחלקה:

מחלקה זו משמשת כמרכז הבקרה (Dashboard) הראשי והבלעדי של מנהל המערכת (Admin). היא מציגה ממשק ניהול מקצועי המבוסס על כרטיסיות (Tabs) כדי לרכז את כל כלי השליטה במסך אחד מסודר. המחלקה מאגדת תחתיה רכיבי ניהול משניים (כגון ניהול מוצרים, ניהול משתמשים, מעקב הזמנות וסטטיסטיקות אמת), ומתפקדת כקונטיינר אב המנתב את תצוגת התוכן המרכזית בהתאם לבחירת המנהל, תוך מניעת הצורך לטעון דפי אינטרנט נפרדים ולשמור על מהירות תגובה גבוהה של הממשק.

פעולות משמעותיות:

- **updateContent**: הפעולה האחראית על הלוגיקה של מערכת הכרטיסיות. בעת לחיצה על לשונית, הפעולה מנקה את מיכל התוכן (contentContainer) וטוענת לתוכו את אובייקט התצוגה הרלוונטי (למשל AdminProductTab או AdminStatsTab). גישה זו חוסכת משאבי רינדור ושומרת על קוד נקי ומודולרי.
- **beforeEnter**: מימוש קריטי של ממשק BeforeEnterObserver המהווה שכבת הגנה. בניגוד לדפים רגילים שרק בודקים אם קיים משתמש כלשהו, פעולה זו מבצעת המרה ומוודאת באופן אקטיבי שהמשתמש המחובר מחזיק בהרשאת Role.ADMIN המפורשת. כל ניסיון גישה של משתמש רגיל או אורח ייחסם מיידיית והלקוח ינותב בחזרה לעמוד הבית הראשי, מה שמבטיח שנתונים עסקיים רגישים לא ייחשפו לגורמים לא מורשים.

.AdminProductTab

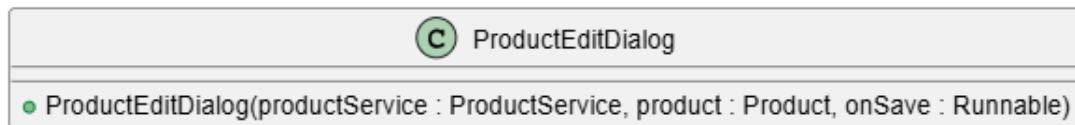


תפקיד המחלקה:

מחלקה זו משמשת כממשק הניהול המרכזי לקטלוג החנות עבור מנהל המערכת. היא מציגה רשימה טבלאית מלאה של כלל הפריטים המאוחסנים במסד הנתונים, ומספקת נקודת גישה אחת ונוחה להוספת מוצרים חדשים, עריכת פרטים קיימים או מחיקת פריטים מהמלאי. העיצוב מבוסס על רכיב ה- Grid של Vaadin, המעניק יכולות מיון פנימיות ושומר על סדר ויזואלי גם כאשר קטלוג החנות הופך לעמוס ומרובה פריטים.

פעולות משמעותיות:

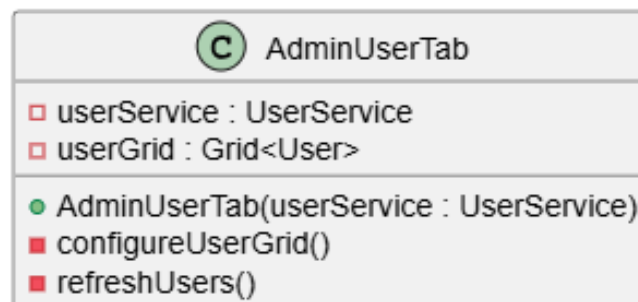
- **configureGrid**: פעולה הבונה את עמודות הטבלה ומגדירה את הפורמט שלהן (כמו תצוגת מחיר מדויקת ומיזוג מערך התגיות למחרוזת קריאה). הפעולה מייצרת עמודות "פעולות" דינמית הכוללת כפתורי עריכה ומחיקה לכל שורה בנפרד, כאשר לחיצה על מחיקה פונה ישירות לשרת ומעדכנת את תצוגת הטבלה בזמן אמת ללא צורך ברענון הדף.
- **openProductDialog**: פעולת עזר המקשרת בין טבלת התצוגה לבין חלונית העריכה (ProductEditDialog). היא מקבלת כאובייקט את המוצר שנבחר (או null במקרה של הוספת מוצר חדש) ופותחת את הדיאלוג המתאים, תוך שהיא מעבירה אליו פונקציית קולבק (Callback) בדמות updateGrid, כדי להבטיח שהטבלה תתעדכן אוטומטית מיד לאחר ביצוע שמירה.

.ProductEditDialog**תפקיד המחלקה:**

מחלקה זו מהווה חלונית צפה (Dialog) המיועדת להזנה ולעריכת הנתונים של פריט בודד בקטלוג המערכת. היא מרכזת לתוכה טופס קלט מקיף הכולל שדות טקסט, מספרים, תיאורים ותגיות, תוך ביצוע המרות נתונים הכרחיות (כמו המרת מחיר מעשרוני לשלם עבור מסד הנתונים, או פיצול מחרוזת תגיות למערך). שימוש בחלונית מרחפת במקום בדף ניווט נפרד שומר על הקשר של המנהל ומאפשר לו לערוך מספר מוצרים ברצף במהירות וביעילות.

פעולות משמעותיות:

- **לוגיקת כפתור השמירה (בתוך הבנאי):** הבלוק המרכזי במחלקה, האוסף את כל המידע שהוזן בשדות הטופס ומעדכן את אובייקט ה- Product. הפעולה עושה שימוש חכם ב- Java Streams כדי לפצל את מחרוזת התגיות (שהופרדו בפסיק), לנקות רווחים מיותרים (trim), ולסנן אלמנטים ריקים, מה שמבטיח איכות נתונים גבוהה לפני השמירה ב- Database. לאחר השמירה המוצלחת מול ProductService ה- ProductService. הפעולה סוגרת את החלונית ומפעילה את רכיב ה- Runnable כדי לעדכן את טבלת האב.

.AdminUserTab**תפקיד המחלקה:**

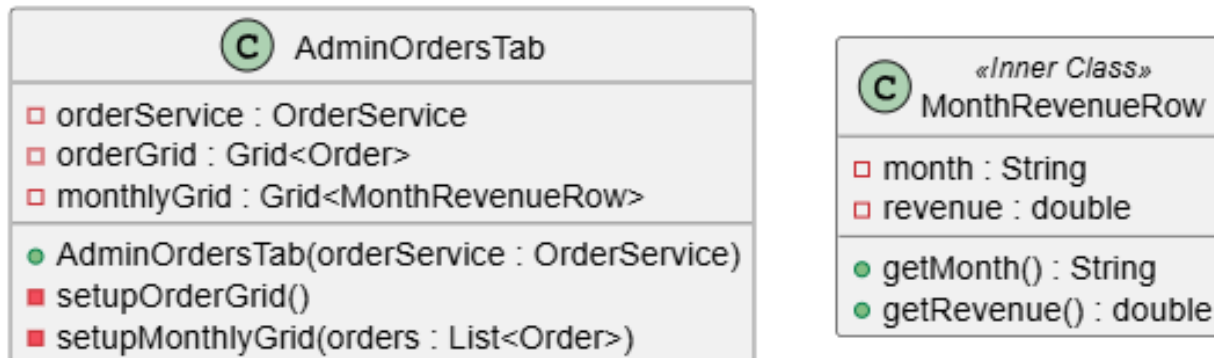
מחלקה זו מספקת למנהל המערכת לוח בקרה לשליטה וניהול בזהויות המשתמשים הרשומים באפליקציה. היא מציגה בטבלה מאורגנת את כלל לקוחות המערכת (שמות, אימיילים והרשאות), ומאפשרת לבצע פעולות תחזוקה כמו הסרת משתמשים ישנים או חסימת גישה (על ידי מחיקתם ממסד הנתונים). הממשק הנקי והפשוט מבטיח שמנהל האתר יוכל למצוא נתוני לקוחות בקלות במקרה של צורך במתן שירות או תמיכה טכנית.

פעולות משמעותיות:

- **configureUserGrid (ומנגנון ההגנה במחיקה):** הפעולה מגדירה את עמודות טבלת המשתמשים ויוצרת את כפתור המחיקה. החלק ההנדסי והחשוב ביותר

בפעולה זו הוא מנגנון ה: Safeguard-לפני ציור כפתור המחיקה, הפעולה משווה בין מזהה המשתמש בשורה הנוכחית לבין מזהה ה - Admin המחובר באותו רגע לסשן. אם קיימת התאמה, כפתור המחיקה מנוטרל, מה שמונע מצב קריטי שבו המנהל מוחק את עצמו בטעות ונועל את עצמו מחוץ למערכת.

.AdminOrdersTab



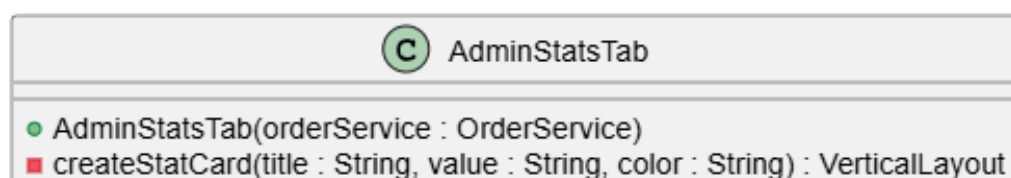
תפקיד המחלקה:

מחלקה זו משמשת כלוח הבקרה הפיננסי (Financial Dashboard) של מנהל האתר. היא אינה מסתפקת רק בהצגת נתונים גולמיים של היסטוריית העסקאות (מי קנה ומה קנה), אלא מבצעת עיבוד נתונים אקטיבי כדי לייצר תובנות עסקיות משמעותיות, כדוגמת פילוח והצגת ההכנסות בחתך חודשי. חלוקת המסך לשתי טבלאות נפרדות (האחת מציגה פירוט של כל הזמנה והשנייה מרכזת רווחים מצטברים) מעניקה למנהל תמונת מצב כלכלית רחבה ומדויקת על ביצועי החנות.

פעולות משמעותיות:

- **setupMonthlyGrid**: פעולה אלגוריתמית מרשימה המדגימה שימוש מתקדם ב Java Stream API לצורך כריית נתונים. הפעולה סורקת את כלל ההזמנות במסד הנתונים, מסננת הזמנות פגומות, ומשתמשת ב - Collectors.groupingBy כדי לקבץ אותן לפי תאריך (חודש ושנה). במקביל, היא סוכמת את מחירי העסקאות עבור כל קבוצת חודש, ממירה את התוצאות לרשימה ממוינת של אובייקטי MonthRevenueRow, ומזינה אותן לטבלת הסיכום הכלכלית.

.AdminStatsTab



תפקיד המחלקה:

מחלקה זו היא מרכז הסטטיסטיקה העסקית של הפרויקט. מטרתה לתרגם עשרות שורות של עסקאות יבשות לכדי ייצוג ויזואלי ומספרים קריאים שעוזרים להבין את מגמות המכירה

באתר. היא מחשבת נתונים כגון סך הפריטים שנמכרו, מזהה את המוצרים הפופולריים ביותר בקטלוג, ומייצרת תצוגה של התפלגות המכירות באמצעות רכיבי UI אינטראקטיביים, ובכך חוסכת למנהל האתר את הצורך לחשב נתונים באופן ידני.

פעולות משמעותיות:

- **הבנאי (לוגיקת עיבוד ופילוח הנתונים):** הבנאי מכיל את ליבת האלגוריתם האנליטי של המחלקה. הוא משתמש ב- flatMap כדי "לשטח" את כל רשימות הפריטים מתוך כל ההזמנות לכדי זרם (Stream) אחד ענק של פריטים בודדים, ואז מקבץ וסופר אותם לפי שם המוצר. בהמשך, הוא ממין את התוצאות, שולף את שלושת המוצרים המובילים, ומייצר עבורם תוצר גרפי דינמי באמצעות שימוש יצירתי ברכיב ה- ProgressBar המציג את אחוז המכירות היחסי של כל פריט מתוך כלל המכירות בחנות.

12.3 מבני נתונים בהם נעשה שימוש

1. מבנה הנתונים: מערך תלת-ממדי (3D Array)

- **מה סוגו? מערך סטטי רב-ממדי של טיפוסים עשרוניים** (`double[][][] dp`).
- **איפה נעשה שימוש?** במחלקת הליבה `SmartCartAlgoService`, בתוך הפעולה `getOptimalBundle`.
- **ולאיזה צורך הוא משמש?** מבנה זה מתפקד כטבלת זיכרון המהווה את לב ליבו של אלגוריתם התכנון הדינמי במערכת ההמלצות. בבעיות אופטימיזציה כמו תרמיל הגב, עץ ההחלטות גדל בצורה מעריכית ויכול לגרום לקריסת המערכת. המערך פותר זאת על ידי שמירת תוצאת הביניים של כל תת-בעיה. כל תא במערך (`dp[i][k][w]`) שומר את ציון ההתאמה המקסימלי שהושג עבור שילוב ספציפי של הפריט הנוכחי, כמות הפריטים שנבחרו, והתקציב שנוצל. על ידי שמירה וקריאה של ערכים אלו מהמערך במקום לחשב אותם מחדש בכל פעם, סיבוכיות זמן הריצה יורדת לזמן פולינומי, מה שמאפשר למערכת לספק ללקוח תוצאה בזמן אמת ללא השהיות בממשק.

2. מבנה הנתונים: רשימה דינמית (ממשק List ומימוש ArrayList)

- **מה סוגו? אנו משתמשים ב-List כממשק (Interface) המגדיר אוסף נתונים ליניארי, וב-ArrayList כמבנה הנתונים בפועל המממש אותו באמצעות מערך שגודלו דינמי.**
- **איפה נעשה שימוש?** במרבית שכבות המערכת. לדוגמה: לאחסון קטלוג המוצרים ב-`ProductService`, לניהול עגלת הקניות ב-`VaadinSession`, ולשמירת שדות השאלון ב-`SmartCartView`.

- **ולאיזה צורך הוא משמש?** השילוב בין השניים משרת שני צרכים הנדסיים. ראשית, ההצהרה על טיפוס List מקדמת פולימורפיזם וקוד גמיש, כך שבעתיד ניתן להחליף את המימוש (למשל ל-LinkedList) ללא שינוי בשאר המערכת. שנית, הבחירה במבנה הנתונים ArrayList נועדה לניהול יעיל של אוספים שגודלם אינו ידוע מראש. בניגוד למערך פרימיטיבי הדורש הקצאת גודל קשיח מראש, ה-ArrayList מקצה זיכרון נוסף באופן אוטומטי מאחורי הקלעים כאשר הוא מתמלא. תכונה זו אידיאלית לאתרי סחר כיוון שאיננו יודעים מראש כמה מוצרים המשתמש יחליט להוסיף לסל הקניות שלו (CartItem) או כמה פריטים יחזרו כתוצאה מסינון דינמי בקטלוג. בנוסף, המבנה מאפשר גישה אקראית ומהירה לאיברים בסיבוכיות של $O(1)$, מה שחיוני לשליפת נתונים מהירה בתצוגת הממשק.

3. מבנה הנתונים: מפת גיבוב (Map / HashMap)

- **מה סוגו? אוסף נתונים המאחסן מידע בצורה של צמדים מקושרים:** מפתח ייחודי וערך תואם (Key-Value Pairs).
- **איפה נעשה שימוש?** במחלקות לוח הבקרה של מנהל האתר: AdminStatsTab ו-AdminOrdersTab.
- **ולאיזה צורך הוא משמש?** ביצוע קיבוץ נתונים וחישובים סטטיסטיים ביעילות מקסימלית. HashMap משתמשת בפונקציית גיבוב כדי לאתר נתונים בסיבוכיות ממוצעת של $O(1)$.
 - בטבלת ההזמנות (AdminOrdersTab), המפה (`<String, Double>Map`) קיבצה אלפי עסקאות פוטנציאליות לפי מפתח של "חודש/שנה", וסיכמה את הערך הכספי שלהן לצורך הצגת רווח חודשי.
 - בטבלת הסטטיסטיקות (AdminStatsTab), המפה סרקה את כל הפריטים שנמכרו אי פעם וספרה מופעים (המפתח הוא שם המוצר והערך הוא כמות המכירות). שימוש במבנה זה מייתר את הצורך בלולאות מקוננות ומסורבלות, ומאפשר הפקת דוחות אנליטיקה ברגע.

4. מבנה הנתונים: קונטיינר בטוח לערכים (Optional)

- **מה סוגו?** מחלקת מעטפת המשמשת כמכל שעשוי להכיל אובייקט קיים או להיות ריק (`<User>Optional`).
- **איפה נעשה שימוש?** בשכבת מאגר הנתונים `UserRepository`, בשירות האימות `UserService`, ובמסך ההתחברות `LoginView`.
- **ולאיזה צורך הוא משמש?** מניעת שגיאות זמן ריצה ובראשן קריסת `NullPointerException`. בעבר, כאשר חיפוש משתמש במסד הנתונים לא העלה דבר (למשל אם הוקלדה סיסמה שגויה), המערכת הייתה מחזירה אובייקט `null` מסוכן. השימוש ב-`Optional` משנה את זה. בכך שהשרת מחזיר "קופסה". שכבת ה-UI מחויבת כעת באופן מוצהר לבדוק האם הקופסה מלאה (באמצעות פונקציית `isPresent()`) לפני שהיא מנסה לשלוף ממנה את נתוני המשתמש. מבנה זה אוסף כתיבת קוד הגנתי, בטוח ומודרני, ומבטיח שהאפליקציה תציג הודעת שגיאה מסודרת למשתמש במקום לקרוס.

12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

SmartCartAlgoService

קוד האלגוריתם:

```
/**
 * מחלקה המייצגת את מנוע האופטימיזציה של המערכת.
 * לפתרון בעיית תרמיל הגב המחלקה מממשת אלגוריתם תכנון דינמי
 * תפקידה הוא לבנות חבילת מוצרים אופטימלית עבור המשתמש בהתבסס על תקציב, הגבלת כמות וציוני התאמה
 * @author DAVID BEN GIGI
 */
@Service
public class SmartCartAlgoService
{
    /**
     * ביצוע אופטימיזציה לבניית חבילת מוצרים
     * האלגוריתם משקלל את ציון ההתאמה של כל מוצר מול המחיר שלו תחת אילוצי כמות ותקציב
     * רשימת כל המוצרים הזמינים לבחירה @param items
     * סך התקציב המוקצה (ביחידות של עשריות שקל) @param budgetUnits
     * מספר פריטים מינימלי הנדרש בחבילה @param minItems
     * מספר פריטים מקסימלי שניתן לכלול בחבילה @param maxItems
     * רשימת מוצרים שנבחרו כחבילה האופטימלית ביותר עבור המשתמש @return
     */
    public List<Product> getOptimalBundle(List<Product> items, int budgetUnits, int minItems, int maxItems)
    {
        int n = items.size();
        // טבלה תלת ממדית: [אינדקס פריט][כמות פריטים שנבחרו][תקציב שנוצל]
        double[][][] dp = new double[n + 1][maxItems + 1][budgetUnits + 1];

        // אתחול הטבלה בערך 1- לסימון מצבים שאינם ברי-השגה
        for (int i = 0; i <= n; i++)
        {
            for (int k = 0; k <= maxItems; k++)
            {
                for (int j = 0; j <= budgetUnits; j++)
                {
                    dp[i][k][j] = -1;
                }
            }
        }

        // חישוב האופטימום
        for (int i = 1; i <= n; i++)
        {
            for (int k = 0; k <= maxItems; k++)
            {
                for (int j = 0; j <= budgetUnits; j++)
                {
                    // בדיקת אפשרות של 0 מוצרים
                    if (j < items[i].price)
                    {
                        dp[i][k][j] = dp[i-1][k][j];
                    }
                    // בדיקת אפשרות של 1 מוצר
                    else if (j >= items[i].price)
                    {
                        int newK = k - items[i].quantity;
                        if (newK < 0) newK = 0;
                        double value = dp[i-1][newK][j - items[i].price] + items[i].value;
                        dp[i][k][j] = Math.max(dp[i-1][k][j], value);
                    }
                }
            }
        }

        // בניית חבילת המוצרים
        List<Product> result = new ArrayList<>();
        int i = n, k = maxItems, j = budgetUnits;
        while (i > 0)
        {
            if (dp[i][k][j] == dp[i-1][k][j])
            {
                i--;
            }
            else
            {
                result.add(items[i]);
                j -= items[i].price;
                k -= items[i].quantity;
                i--;
            }
        }

        return result;
    }
}
```



```

    {
        for (int w = 0; w <= budgetUnits; w++)
        {
            dp[i][k][w] = (k == 0) ? 0 : -1.0;
        }
    }
}

// מילוי טבלת התכנון הדינמי
for (int i = 1; i <= n; i++)
{
    Product current = items.get(i - 1);
    int price = current.getPrice();
    double score = current.getTemporaryScore();

    for (int k = 1; k <= maxItems; k++)
    {
        for (int w = 0; w <= budgetUnits; w++)
        {
            // אופציה 1: אי הכללת הפריט הנוכחי בחבילה
            dp[i][k][w] = dp[i - 1][k][w];

            // אופציה 2: הכללת הפריט (בתנאי שהתקציב מאפשר והמצב הקודם היה חוקי)
            if (w >= price && dp[i - 1][k - 1][w - price] != 1)
            {
                double optionTake = dp[i - 1][k - 1][w - price] + score;
                if (optionTake > dp[i][k][w])
                {
                    dp[i][k][w] = optionTake;
                }
            }
        }
    }
}

// מציאת התוצאה האופטימלית בטווח הכמויות המבוקש
double bestScore = -1;
int bestK = -1;
int bestW = -1;

for (int k = minItems; k <= maxItems; k++)
{
    for (int w = 0; w <= budgetUnits; w++)
    {
        if (dp[n][k][w] > bestScore)
        {
            bestScore = dp[n][k][w];
            bestK = k;
            bestW = w;
        }
    }
}

// במקרה שלא נמצא פתרון חוקי, החזר רשימה ריקה
if (bestScore == -1) return new ArrayList<>();

// של הפריטים שנבחרו בפועל מתוך הטבלה (Backtracking)
List<Product> selected = new ArrayList<>();
int currK = bestK;
int currW = bestW;

for (int i = n; i > 0 && currK > 0; i--)
{
    // בדיקה האם הפריט הנוכחי נבחר (הערך השתנה לעומת השורה הקודמת)
    if (dp[i][currK][currW] != dp[i - 1][currK][currW])
    {

```

```

        Product p = items.get(i - 1);
        selected.add(p);
        currK--;
        currW -= p.getPrice();
    }
}
return selected;
}
}

```

טענת כניסה (Parameters):

1. **items**: רשימת כל המוצרים הזמינים בחנות (כאשר לכל מוצר מחושב ציון התאמה אישי).
2. **budgetUnits**: סך התקציב שעומד לרשות המשתמש (מנורמל ליחידות כספיות).
3. **minItems / maxItems**: אילוצי הכמות המינימלית והמקסימלית שהמשתמש הגדיר לחבילה.

טענת יציאה (Return Value):

האלגוריתם מחזיר `<Product>List` המכיל את תת-קבוצת המוצרים המרכיבים את ה"חבילה האופטימלית" – כלומר, הקבוצה שמשיגה את סך הציונים הגבוה ביותר מבלי לחרוג מהתקציב וממגבלות הכמות.

מה הפעולה מבצעת?

האלגוריתם פותר וריאציה מורכבת של בעיית "תרמיל הגב" (Knapsack Problem). המערכת מחפשת את השילוב הטוב ביותר של פריטים (מבחינת ציון התאמה לסטייל של המשתמש) תחת אילוצי תקציב וכמות פריטים. הפעולה לא רק מוצאת את הציון הגבוה ביותר, אלא מבצעת "שחזור מסלול" (Backtracking) מתוך טבלת ה-DP כדי להחזיר בפועל את רשימת המוצרים שנבחרו.

אופן פעולתו:

1. **מבנה נתונים תלת-ממדי**: האלגוריתם משתמש בטבלה `dp[i][k][w]` ששומרת את הניקוד המקסימלי לכל מצב: `i` פריטים שנשקלו, `k` פריטים שנבחרו, ו-`w` תקציב שנוצל.

2. מילוי טבלה: עבור כל פריט, האלגוריתם מקבל שתי החלטות:

- לא לכלול את הפריט: הניקוד נשאר כפי שהיה בפריט הקודם.
- לכלול את הפריט: מוסיף את ציון הפריט לניקוד המקסימלי שהושג עבור התקציב הנותר.

3. **שחזור (Backtracking):** לאחר מילוי הטבלה, האלגוריתם סורק את השורה האחרונה (n) בטבלה כדי למצוא את הניקוד הגבוה ביותר בטווח הכמויות המבוקש. משם הוא נע אחורה בטבלה: אם הערך ב- $dp[i]$ שונה מ- $dp[i-1]$, סימן שהפריט ה- i נבחר לחבילה.

סיבוכיות זמן ריצה:

סיבוכיות הזמן היא $O(N \times K \times W)$, כאשר:

- N = מספר המוצרים בקטלוג.
- K = כמות הפריטים המקסימלית בחבילה.
- W = התקציב הכולל.

מכיוון ש- K ו- W הם גבולות קבועים יחסית (המשתמש לא יבחר 1,000 פריטים), האלגוריתם פועל ביעילות גבוהה מאוד ומספק פתרון אופטימלי בזמן ריצה קצר, מה שמבטיח שגם בחנויות עם מאות פריטים, החבילה תיווצר תוך שבריר שנייה.

מנוע חישוב ציוני התאמה (ScoringService)

קוד האלגוריתם:

```
/**
 * (Matching Engine) שירות לחישוב ציוני התאמה
 * מחלקה זו אחראית על ניתוח ההעדפות של המשתמש כפי שהוגדרו בשאלון הדינמי
 * ושקלולן מול תגיות המאפיינים של מוצרי החנות לקבלת ציון התאמה באחוזים
 * @author DAVID BEN GIGI
 */
@Service
public class ScoringService
{
    /**
     * ביצוע חישוב ציוני התאמה עבור רשימת מוצרים נתונה
     * האלגוריתם מבצע נורמליזציה של הבחירות ומחשב את הציון לפי יחס ההתאמות בפועל
     * * @param products רשימת המוצרים שיש לדרג
     * * @param selectedTags רשימת התגיות שהמשתמש בחר בשאלון ההעדפות
     */
    public void calculateScoresBySurvey(List<Product> products, List<String>
selectedTags)
    {
        for (Product p : products)
        {
```

```

List<String> productTags = p.getTags();

// במידה ולמוצר אין תגיות, הציון נקבע כ-0 (לא ניתן לבצע התאמה)
if (productTags == null || productTags.isEmpty())
{
    p.setTemporaryScore(0.0);
    continue;
}

double currentScore = 0; // מונה התאמות בפועל
double maxScore = 0;     // מסך האילוצים הממשיים

// מעבר על העדפות המשתמש וסינון העדפות כלליות
for (String userTag : selectedTags)
{
    // סינון ערכים ריקים או העדפה גנרית ("הכל") שאינה דורשת סינון
    if (userTag == null || userTag.trim().isEmpty() ||
userTag.equals("הכל"))
    {
        continue;
    }

    maxScore += 1.0; // עדכון הרף למכנה עבור כל אילוץ משמעותי

    // בדיקה האם התגית שהמשתמש בחר קיימת במוצר הנוכחי
    if (hasTag(productTags, userTag))
    {
        currentScore += 1.0;
    }
}

// חישוב האחוז הסופי (במקרה שאין אילוצים, הציון הוא 100% כברירת מחדל)
double finalResult = (maxScore == 0) ? 100.0 : (currentScore /
maxScore) * 100;

// עיגול התוצאה לדיוק של ספרה אחת אחרי הנקודה ושמירת הציון במוצר
p.setTemporaryScore(Math.round(finalResult * 10.0) / 10.0);
}

/**
 * פונקציית עזר לביצוע השוואת תגיות בטוחה (Case-Insensitive).
 * מתעלמת מרווחים מיותרים ומאותיות גדולות/קטנות כדי למנוע טעויות הקלדה.
 * * @param tags רשימת התגיות הקיימות במוצר.
 * * @param value התגית אותה מחפשים (מהעדפות המשתמש).
 * * @return true במידה ונמצאה התאמה, false אחרת.
 */
public boolean hasTag(List<String> tags, String value)
{
    for (String tag : tags)
    {
        // השוואה עמידה לרווחים ואותיות (Case-Insensitive)
        if (tag != null && tag.trim().equalsIgnoreCase(value.trim()))
        {
            return true;
        }
    }
    return false;
}
}

```

טענת כניסה (Parameters):

1. **products**: רשימת כל מוצרי הביגוד הזמינים במערכת, שכל אחד מהם מחזיק רשימת תגיות מאפיינות.
2. **selectedTags**: רשימת התגיות שהמשתמש בחר בשאלון הדינמי (למשל: "חורף", "אלגנטי", "כחול").

טענת יציאה (Return Value):

הפעולה אינה מחזירה ערך (void), אלא מעדכנת את שדה ה-temporaryScore ישירות על גבי אובייקטי ה-Product הקיימים במערכת, כדי להכין אותם לשלב הדירוג וההמלצות.

מה הפעולה מבצעת?

הפעולה מתרגמת העדפות אנושיות סובייקטיביות לציון אובייקטיבי (0-100%). היא משמשת כמנוע ההמלצות הראשי, המאפשר למערכת "להבין" אילו בגדים הכי מתאימים לטעם האישי של המשתמש, תוך התחשבות במקרים שבהם המשתמש לא בחר העדפה מסוימת.

אופן פעולתו:

1. **עיבוד נתונים מקדים**: האלגוריתם עובר על כל מוצר ומחלץ את תגיות המאפיינים שלו.
2. **לוגיקה מותנית (Business Logic)**: האלגוריתם מזהה ערכים לא רלוונטיים (כמו "הכל" או שדות ריקים) ומדלג עליהם. זו לוגיקה קריטית למניעת "ענישה" של מוצרים במקרים שבהם המשתמש השאיר שאלה פתוחה.
3. **השוואה חכמה**: השימוש בפונקציית עזר hasTag המבצעת equalsIgnoreCase ו-trim() מבטיח שהשוואת הטקסט אינה רגישה לרווחים מיותרים או לאותיות גדולות/קטנות (למשל, "כחול" ו-"כחול " ייחשבו זהים).
4. **נורמליזציה**: הציון מחושב כיחס בין התאמות בפועל לבין סך האילוצים שהוגדרו, ומעוגל לרמת דיוק של ספרה אחת אחרי הנקודה.

סיבוכיות זמן ריצה:

הסיבוכיות היא $O(P \times T \times S)$, כאשר:

• P = מספר המוצרים בקטלוג.

- T = ממוצע התגיות בכל מוצר.

- S = מספר התגיות שהמשתמש בחר בשאלון.

מאחר ו- T ו- S הם ערכים קטנים מאוד וקבועים (לרוב מדובר ב-5-10 תגיות), הסיבוכיות מתנהגת בצורה ליניארית $O(P)$ ביחס למספר המוצרים. זהו פתרון קל משקל שניתן להריץ בלחיצת כפתור בכל פעם שהמשתמש מעדכן את בחירותיו, ללא השהיה מורגשת בחוויית המשתמש.

מנוע אינטגרציה למערכת חיצונית (getTemperature)

קוד הפעולה:

```
public double getTemperature(String city)
{
    try
    {
        String url = "https://api.openweathermap.org/data/2.5/weather?q=" +
city + "&appid=" + API_KEY + "&units=metric";

        String jsonResponse = restTemplate.getForObject(url, String.class);

        ObjectMapper mapper = new ObjectMapper();
        JsonNode root = mapper.readTree(jsonResponse);

        return root.path("main").path("temp").asDouble();
    }
    catch (Exception e)
    {
        System.out.println("שגיאה בשליפת מזג האוויר: " + e.getMessage());
        return 22.0; // גיבוי
    }
}
```

טענת כניסה (Parameters):

1. $city$: מחרוזת המייצגת את שם העיר עבורה נדרש התחזית.

טענת יציאה (Return Value):

הפעולה מחזירה $double$ המייצג את הטמפרטורה הנוכחית בעיר הנבחרת במעלות צלזיוס.

מה הפעולה מבצעת?

הפעולה מחברת את המערכת לשירות מזג אוויר חיצוני בזמן אמת (OpenWeatherMap API). היא מבצעת בקשת HTTP, מפענחת את מבנה ה-JSON המורכב שמתקבל מהשרת המרוחק, ומחלצת מתוכו נתון ספציפי (הטמפרטורה) לצורך התאמת המלצות הביגוד של המשתמש.

אופן פעולתו:

1. **צריכת Web Service:** שימוש ב- RestTemplate של Spring המאפשר לשלוח בקשות HTTP בצורה קלה ומהירה.
2. **עיבוד נתונים אסינכרוני (Parsing):** שימוש בספריית Jackson המודרנית. במקום לקרוא את ה-JSON כמחרוזת פשוטה, הקוד בונה "עץ" (JsonNode) שמאפשר לנווט במבנה הנתונים היררכי ולשלוף רק את הערך הרלוונטי (`(root.path("main").path("temp"))`).
3. **תכנות הגנתי:** הוספת בלוק try-catch המבטיח שאם ה-API החיצוני אינו זמין (למשל בעיית רשת), המערכת לא תקרוס, אלא תחזיר טמפרטורת ברירת מחדל ("טמפרטורה נוחה"), ובכך שומרת על חוויית משתמש רציפה.

סיבוכיות:

הסיבוכיות תלויה בעיקר בביצועי הרשת ובמהירות השרת המרוחק. מבחינה אלגוריתמית, פעולת ניתוח ה-JSON היא O(1) ביחס לגודל התגובה (שכן מבנה ה-JSON קבוע וקטן), מה שהופך את השירות למהיר מאוד.

ניתוח הכנסות חודשי (getMonthlyRevenue)**קוד הפעולה:**

```
/**
 * מחזירה מפה המקבצת את הרווחים לפי חודש ושנה (למשל "05/2026").
 */
public Map<String, Double> getMonthlyRevenue()
{
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MM/yyyy");
    return getAllOrders().stream()
        .filter(o -> o.getOrderDate() != null)
        .collect(Collectors.groupingBy(
            o -> o.getOrderDate().format(formatter),
            Collectors.summingDouble(Order::getTotalPrice)
        ));
}
```

טענת כניסה (Parameters):

אין פרמטרים. הפעולה נשענת על נתוני המערכת הקיימים ב-OrderRepository (דרך המתודה `getAllOrders()`).

טענת יציאה (Return Value):

הפעולה מחזירה אובייקט `Map<String, Double>`, כאשר כל מפתח (Key) מייצג חודש ושנה, וכל ערך (Value) מייצג את סך ההכנסות המצטבר של אותו חודש.

מה הפעולה מבצעת?

הפעולה מהווה כלי המאפשר למנהל המערכת לצפות במגמות מכירה חודשיות. היא הופכת רשימה גולמית של עשרות או מאות הזמנות לסיכום פיננסי תמציתי ומאורגן.

אופן פעולתו:

1. **עיצוב נתונים:** שימוש ב-`DateTimeFormatter` כדי לנרמל את תאריכי ההזמנה לפורמט סטנדרטי (MM/yyyy).
2. **סינון:** הפעלת פילטר כדי לוודא שאין הזמנות "יתומות" ללא תאריך, מה שמבטיח יציבות של האלגוריתם.
3. **קיבוץ וסכימה:** שימוש ב-`Collectors.groupingBy` ליצירת מבנה הנתונים הסטטיסטי, תוך הפעלת פונקציית צמצום (`summingDouble`) המבצעת סיכום של המחיר הכולל בזמן הקיבוץ. זה מונע לולאות מורכבות ומייעל את זמן הביצוע.

סיבוכיות זמן ריצה:

הסיבוכיות היא $O(N)$, כאשר N הוא מספר ההזמנות במערכת. ה-`Stream` עובר על כל הזמנה פעם אחת בלבד. הפעולות בתוך ה-`collect` (קיבוץ וסכימה) מתבצעות בסיבוכיות ממוצעת של $O(1)$ לכל הזמנה, מה שמבטיח עבודה מהירה גם תחת עומס של נתונים רבים.

13. תיאור מסד הנתונים

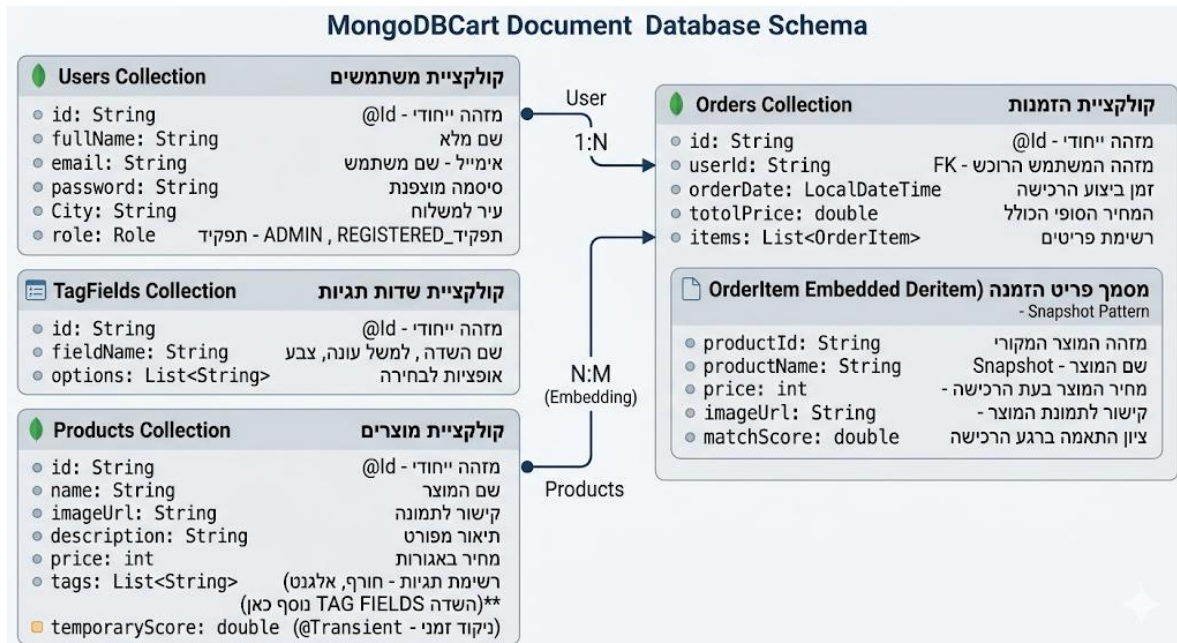
תשתית הנתונים של המערכת מבוססת על מסד נתונים לא-יחסי, תוך שימוש בטכנולוגיית `Mongo DB`. בשונה ממסדי נתונים מסורתיים הכופים מבנה טבלאי קשיח וקבוע, מסד נתונים זה שומר את המידע בתצורה של "אוספים" המכילים "מסמכים" בפורמט טקסטואלי גמיש המדמה עצמים בקוד. בחירה הנדסית זו היא קריטית להצלחת הפרויקט מהסיבות הבאות:

- **גמישות במבנה הנתונים:** קטלוג מוצרי אופנה מתאפיין בהטרוגניות, כלומר לפריטים שונים יש תכונות שונות. מודל המסמכים מאפשר לשמור כל פריט עם המאפיינים הייחודיים לו באותו אוסף, ללא צורך בטבלאות קשר מורכבות או בביצוע פעולות איחוד משאבים יקרות.
- **טיפול במבנים מורכבים:** המערכת נדרשת לאחסן מערכי תגיות עבור כל פריט ועבור כל שדה בשאלון לצורך סינון ואופטימיזציה. מסד הנתונים תומך באופן טבעי

באחסון מערכים דינמיים ואובייקטים פנימיים מוטמעים, ומאפשר ביצוע שאלות שלילפה מהירות במיוחד.

מסד הנתונים מתארח על גבי תשתית ענן מנוהלת. ארכיטקטורה זו מבטיחה זמינות גבוהה של המידע מכל מקום, מנגנוני גיבוי אוטומטיים, ויכולת גדילה נוחה של נפח האחסון והזיכרון ככל שכמות המשתמשים והמוצרים באתר תתרחב.

להלן תרשים ERD:



מודל הנתונים של האפליקציה מורכב מארבעה אוספים עיקריים:

- אוסף משתמשים:** ישות זו מאחסנת את פרופיל המשתמש הרשום. כל מסמך באוסף מכיל פרטים אישיים כגון מזהה ייחודי, שם מלא, כתובת דואר אלקטרוני המשמשת כשם משתמש חד-ערכי, סיסמת גישה, עיר מגורים המשמשת את שירות אקלים מזג האוויר, ושדה המציין האם מדובר בלקוח רגיל או במנהל מערכת בעל הרשאות מורחבות.
- אוסף מוצרים:** מהווה את חומר הגלם המרכזי עבור אלגוריתם הסל החכם. כל מסמך מכיל מידע ויזואלי ולוגיסטי, כמו שם הבגד, תיאור מילולי, קישור לתמונה, ומחיר המיוצג כמספר שלם לצורך דיוק חישובי, לצד מערך מפורט של תגיות טקסטואליות המגדירות את מאפייני הלבוש. ישות זו אינה מכילה שדה ציון קבוע, שכן הציון מחושב באופן דינמי בשרת עבור כל משתמש בנפרד.
- אוסף הזמנות:** אחראי על תיעוד העסקאות שבוצעו בהצלחה בחנות. האוסף ממומש בתבנית של הטמעה פנימית, שבה נשמר בתוך מסמך ההזמנה עצמו צילום מצב מלא של פריטי הלבוש. צילום מצב זה שומר לא רק את שם המוצר ומחירו ברגע הסליקה, אלא מתעד גם את ציון ההתאמה המדויק שהאלגוריתם העניק לפריט. שמירה זו מונעת עיוות נתונים היסטוריים במקרה של שינוי מחירים עתידי, ומאפשרת מעקב אנליטי אחר איכות ההחלטות של הבינה המלאכותית. המסמך כולל גם את מזהה המשתמש, תאריך הרכישה והסכום הכולל.

4. **אוסף שדות תגיות:** מחזיק את המידע המבני לבניית שאלון הסטייל הדינמי של המערכת. כל מסמך מייצג קטגוריית בחירה ומכיל את שם השדה ומערך דינמי של אפשרויות בחירה. אוסף זה מאפשר למנהל המערכת להוסיף, לעדכן או למחוק שדות ואופציות מהשאלון ישירות מממשק הניהול, כאשר שכבת התצוגה פונה לאוסף זה ובונה את הטופס אוטומטית ללא צורך בשינוי קוד.

קשרים לוגיים בין הישויות במסד:

- **קשר יחיד לרבים בין משתמשים להזמנות:** ממומש באמצעות שמירת מזהה המשתמש כמפתח מקשר בתוך מסמך ההזמנה, המאפשר שליפה מהירה של כלל הרכישות עבור לקוח ספציפי בדף ההיסטוריה.
- **קשר רבים לרבים בין הזמנות למוצרים:** ממומש באמצעות הטמעת רשימת הפרטים כאובייקטים פנימיים בתוך מסמך ההזמנה, מה שחוסך פעולות קישור מורכבות בעת קריאת היסטוריית הרכישות.
- **אוסף שדות התגיות:** מתפקד כישות מטא-דאטה עצמאית. רכיבי הלוגיקה נעזרים בה לצורך יצירת שדות הבחירה, ללא קשר ישיר למשתמש או למוצר במסד הנתונים עצמו.

14. מדריך למשתמש

מדריך זה נועד להדריך את הגולש צעד אחר צעד בשימוש במערכת "הסל החכם" (SmartCart), החל משלב ההרשמה ועד לקבלת חבילת הלבוש האופטימלית דרך האלגוריתם החכם.

1. כניסה למערכת והרשמה

עם הכניסה לאתר, תגיעו לדף הבית בתצוגת "אורח". במסך זה תוכלו לראות הסבר קצר על המערכת.

- **משתמשים חדשים:** לחצו על כפתור "התחברות / הרשמה" ולאחר מכן על הקישור "צרו חשבון עכשיו". מלאו את הפרטים בטופס (שם מלא, אימייל, סיסמה ועיר מגורים - חשוב להזנת עיר מדויקת כדי לקבל המלצות מזג אוויר בהמשך) ולחצו על "הירשם עכשיו".
- **משתמשים רשומים:** הזינו את כתובת האימייל והסיסמה בחלונית ההתחברות ולחצו על "התחבר". הודעה ירוקה תאשר את כניסתכם ותעביר אתכם ללוח הבקרה האישי.

2. ניווט במערכת (סרגל הכלים העליון)

לאחר ההתחברות, יופיע בראש המסך סרגל ניווט שילווה אתכם לאורך כל הגלישה. דרכו תוכלו להגיע לכל אזור:

- **חנות:** מעבר לקטלוג המוצרים המלא.
- **סל חכם:** כניסה למנוע בניית החבילות מבוסס הבינה המלאכותית.
- **עגלת קניות:** לחיצה על סמל העגלה (עליו מופיע עיגול אדום עם מספר הפריטים שבחרתם) תוביל אתכם לתשלום.
- **אזור אישי (סמל הפרופיל):** לחיצה על שמכם תפתח תפריט המאפשר צפייה ב"היסטוריית הזמנות", החלפת חשבון או התנתקות מאובטחת מהמערכת.

3. דף הבית והמלצות מזג אוויר

דף הבית האישי שלכם כולל גישה מהירה לשירותי הליבה (חנות, סל חכם, היסטוריה). מתחת לכפתורים אלו, המערכת תזהה את עיר מגוריכם, תבדוק את הטמפרטורה הנוכחית באמצעות שירות מטאורולוגי, ותציג עבורכם באופן אוטומטי "קרוסלת" מוצרים מומלצים התואמים לאקלים (למשל סריגים וז'קטים ביום קר). תוכלו לגלול ימינה ושמאלה ולהוסיף פריטים לסל ישירות מכאן.

4. חיפוש ידני בחנות הבגדים

אם ברצונכם לחפש פריטים בעצמכם:

1. נווטו למסך "חנות".
2. בצד המסך תמצאו סרגל סינון. לחצו על המגדר המבוקש ("גברים" או "נשים") כדי לחשוף את תת-הקטגוריות (חולצות, מכנסיים, שמלות וכו').
3. ניתן להשתמש בשורת החיפוש העליונה כדי לחפש פריט לפי שם ספציפי (התוצאות יתעדכנו תוך כדי הקלדה).
4. מצאתם פריט שאהבתם? לחצו על "הוסף לסל" שמופיע על כרטיס המוצר. מספר הפריטים בבר העליון יתעדכן מיד.

5. שימוש בבונה החבילות – "הסל החכם" (מומלץ!)

גולת הכותרת של המערכת היא היכולת לבנות עבורכם חבילת לבוש אידיאלית בתקציב נתון.

1. נווטו למסך "סל חכם" דרך הבר העליון או דף הבית.

2. **מילוי השאלון:** ענו על השאלות הדינמיות המופיעות במסך (כגון עבור מי הבגד, לאיזה אירוע, סגנון רצוי וצבע מועדף). תגיות אלו יעזרו לאלגוריתם להבין בדיוק מה הטעם שלכם.
 3. **הגדרת אילוצים:** הזינו את התקציב המקסימלי שאתם מוכנים להוציא (בשקלים), ובחרו את כמות הפריטים המינימלית והמקסימלית שתמצאו לקבל בחבילה.
 4. לחצו על "חשב סל אופטימלי".
 5. חלונית התקדמות תופיע ותציג את שלבי החישוב. בסיום, תוצג בפניכם רשימת הפריטים המנצחת שמרכיבה את החבילה המשתלמת ביותר עבורכם, יחד עם סיכום העלות. תוכלו להוסיף את הפריטים הללו ישירות לעגלה.
- 6. עגלת קניות ותשלום**
1. לחצו על סמל עגלת הקניות בפינה העליונה.
 2. במסך זה תראו את כל הפריטים שאספתם. אם התחרטתם על פריט מסוים, לחצו על סמל פח האשפה האדום לידו כדי להסירו (הסכום הכולל יתעדכן אוטומטית).
 3. לחצו על הכפתור הירוק "מעבר לתשלום מאובטח".
 4. המערכת תדמה עיבוד תשלום, ובסיומו תופיע חלונית עם "וי" ירוק המאשרת שההזמנה נשמרה בהצלחה. העגלה שלכם תתרוקן.
- 7. היסטוריית הזמנות**
- כדי לעקוב אחר הרכישות הקודמות שלכם, פתחו את תפריט הפרופיל העליון ובחרו ב"היסטוריית הזמנות". במסך זה תוצג טבלה מסודרת המכילה את כל ההזמנות שביצעתם בעבר, כולל תאריך העסקה, סכום לתשלום ותמונות מוקטנות של כל הפריטים שרכשתם. ריכוף עם העכבר מעל כל תמונה יציג את שם הפריט המלא.
- הערה למנהלי מערכת (Admins):** משתמשים בעלי הרשאות ניהול יראו כפתור נוסף בסרגל העליון בשם "ניהול אתר". לחיצה עליו תוביל לפאנל הניהול בו ניתן להוסיף או להסיר מוצרים מהחנות, לנהל משתמשים, ולצפות בגרפים וניתוחים סטטיסטיים של הכנסות האתר. בנוסף, למנהלים קיימת גישה לעריכת שדות "הסל החכם" דרך כפתור מיוחד המופיע בראש מסך בונה החבילות.

15. בדיקות והערכה

תהליך הבדיקות בפרויקט SmartCart התבצע לאורך כל מחזור החיים של הפיתוח, תוך בדיקת התנהגות המערכת בהתאם לקלט ובדיקת התקשורת בין ה-UI, ה-Services ומד MongoDB הנתונים.

פירוט סוגי הבדיקות:

- 1. בדיקות יחידה:** כל לוגיקה עסקית מורכבת (כמו אלגוריתם ה-Knapsack וחישוב ציוני ההתאמה ב-ScoringService) נבדקה בנפרד. יצרתי תרחישי קצה (כגון חוסר בתגיות, תקציב אפסי או חריגה מכמות פריטים) כדי לוודא שהאלגוריתמים עמידים ואינם גורמים לקריסת המערכת.
- 2. בדיקות אינטגרציה:** בוצעו בדיקות מלאות של תהליך הרכישה, החל מבחירת מוצרים ב-ShopView, דרך הרצת האלגוריתם ב-SmartCartView, ועד לשמירת ההזמנה ב-OrderService ווידוא עדכון מסד הנתונים.
- 3. בדיקות ממשק משתמש:** בדיקות ידניות לבחינת רספונסיביות הממשק. נבדק כי הניווט בין המסכים תקין, כי הרשאות המנהל (AdminView) אינן נגישות למשתמש רגיל, וכי הודעות המערכת (כמו "התשלום בוצע בהצלחה") מוצגות בצורה ברורה למשתמש.
- 4. טיפול בשגיאות:** המערכת נבדקה מול מקרי קצה של תקשורת מול ה-API החיצוני (מזג האוויר). הוספנו ערכי ברירת מחדל (Default Values) כדי להבטיח שהאפליקציה תמשיך לעבוד גם במקרה של ניתוק מהרשת.

תוצאות הערכה:

הערכת המערכת הראתה עמידה בכל היעדים שהוגדרו בתחילת הפרויקט. זמן תגובת האלגוריתם המורכב (Knapsack) הוא מהיר בצורה בלתי רגילה, מה שמבטיח חוויית משתמש חלקה גם תחת עומס.

16. מסקנות

סיכום חוויית הפיתוח:

העבודה על פרויקט SmartCart אפשרה לי ליישם את עקרונות הנדסת התוכנה שלמדתי בצורה מעשית ומורכבת. ההתמודדות עם בעיות אמיתיות כמו ניהול מצבים ב-Vaadin, אופטימיזציה של שאילתות ב-MongoDB, והפרדת שכבות היוותה שיעור חשוב בדרך לפיתוח מערכת זו.

מסקנות מקצועיות:

- 1. חשיבות הארכיטקטורה:** למדתי שללא הפרדה ברורה בין שכבת התצוגה (UI) לשכבת הלוגיקה העסקית (Services), הפרויקט הופך ל"קוד ספגטי" קשה לתחזוקה. המעבר שביצעתי במהלך הפיתוח העברת החישובים מה-UI ל-Services הוכיח כי קוד נקי הוא המפתח ליציבות המערכת.

2. **ניהול מורכבות אלגוריתמית:** הוכח כי פתרון בעיות NP-Hard (כמו בעיית התרמיל) דורש חשיבה תכנונית מעמיקה לפני כתיבת שורת קוד אחת. היכולת להשתמש בתכנון דינמי פתרה בעיה עסקית מורכבת בצורה יעילה ומהירה.
3. **למידה מול טעויות:** במהלך הפרויקט נתקלתי באתגרים משמעותיים, כגון זליגת לוגיקה ל-UI וניהול סנכרון תהליכונים (Threads) ב-Vaadin. הפתרון היה תמיד חזרה לעקרונות היסוד של הנדסת תוכנה.

שורה תחתונה:

פרויקט SmartCart עומד בדרישות המערכת שהוגדרו, ומספק פתרון ייחודי, יעיל ומקצועי למשתמש. הידע שצברתי בתהליך זה מהווה בסיס איתן להמשך דרכי המקצועית בתחום פיתוח התוכנה.

17. פיתוחים עתידיים

הגרסה הנוכחית של המערכת מהווה "הוכחת היתכנות" ליכולת לבצע אופטימיזציה של סל קניות תחת אילוצים. עם זאת, החזון הטכנולוגי והעסקי של הפרויקט רחב הרבה יותר. להלן כיווני הפיתוח המרכזיים המתוכננים לגרסאות הבאות, אשר לא נכללו בגרסה זו מפאת מסגרת הזמן:

1. מעבר לארכיטקטורת Multi-Vendor

החזון האסטרטגי הוא להרחיב את המערכת מחנות בודדת לפלטפורמת "Super-Aggregator" המאגדת קטלוגים ממספר רב של קמעונאי אונליין (כגון ASOS, ZARA ועוד) באמצעות ממשקי API בזמן אמת. שינוי זה יהפוך את הליבה האלגוריתמית למורכבת ועשירה יותר: המנוע ידרש לשקלל לא רק את מחיר וניקוד הפריט, אלא גם משתנים לוגיסטיים כגון דמי משלוח משתנים וזמני אספקה מספקים שונים. התוצאה תהיה "סל היברידי" אופטימלי, המאפשר למשתמש לרכוש את העסקה המשתלמת ביותר ברשת במקום אחד.

2. פרסונליזציה מבוססת למידת מכונה

בגרסה הנוכחית, ה"ניקוד" לכל פריט הוא סטטי. בפיתוח העתידי ישולב מודל למידת מכונה (Machine Learning) שיחליף את הניקוד הקבוע בניקוד דינמי ומותאם אישית. המערכת תלמד את דפוסי הגלישה, הרכישות והסגנון של כל משתמש, ותייצר פונקציית הערכה (Scoring Function) ייחודית עבורו. כך, שני משתמשים שונים יקבלו המלצות שונות לחלוטין עבור אותו מאגר מוצרים, מה שישיפר דרמטית את אחוזי ההמרה.

3. התמודדות עם עומסים באמצעות אלגוריתמי קירוב

פתרון ה-Knapsack הנוכחי מבוסס על תכנות דינמי ומבטיח פתרון מדויק, אך עשוי להיות כבד חישובית בקלטים גדולים מאוד. כהרחבה טכנולוגית, יפותחו במערכת "אלגוריתמי קירוב" ושיטות חמדניות. מנגנון זה ייכנס לפעולה אוטומטית במקרי קיצון בהם גודל התקציב או כמות הפריטים חורגים מהטווח היעיל, ויספק למשתמש פתרון "כמעט אופטימלי" בזמן אמת, תוך שמירה על ביצועי מערכת גבוהים.

4. אופטימיזציה ירוקה

הרחבת המודל המתמטי כך שישלול מימד נוסף של "ציון אקולוגי" לכל פריט, המבוסס על נתונים כגון הרכב הבד, תהליכי ייצור ומרחק שינוע. המערכת תאפשר למשתמש להוסיף אילוץ ערכי (למשל: "הבטח סל עם דירוג ירוק מעל 80%"). פיתוח זה דורש פתרון בעיית אופטימיזציה רב-ממדית. מציאת האיזון המושלם בין מקסימום סטייל, מינימום מחיר ומינימום פגיעה סביבתית.

18. ביבליוגרפיה

- [1] ויקיפדיה, "בעיית תרמיל הגב", [לערך המלא](#).
- [2] SCIRP, "Research on the Algorithms of MDPK," Journal of Computer Science and Applications, 2018. [למאמר](#).
- [3] GeeksforGeeks, "0/1 Knapsack Problem - Dynamic Programming," Technical Tutorial Series. [למדריך](#).
- [4] Spring.io, "Spring Boot Reference Guide," Official Documentation. [לתיעוד](#).
- [5] Martin Fowler, "Software Development Lifecycle", [למאמר](#).
- [6] MongoDB Documentation, "Document Model", [לתיעוד](#).
- [7] Martin Fowler, "Layered Architecture", [למאמר](#).